GriD-LMIA
User Manual

Version v1.3.7

August 10, 2026

Abstract

GriD-LMIA is a MATLAB/YALMIP package for modelling and certifying a Linear Matrix Inequality (LMI) condition that needs to hold over a multi-dimensional parameter hyper-rectangle, known as Parameter-Dependent Linear Matrix Inequality (PD-LMI). If furthermore, it contains the derivative of decision variables, it is known as Differentiable Parameter-Dependent Linear Matrix Inequality (DPD-LMI). The package name,GriD-LMIA, stands for: Gridding-Based DPD-LMI Assembler. The package represents coefficient-backed known matrices and continuous decision matrices as cell-wise tensor-product Bernstein polynomials on an user-defined partitioned physical parameter grid. On each cell, the sufficient certificate finds a set of LMIs to turn the continuous condition into inspectable YALMIP constraints.

The intended workflow is to define the tensor grid and coefficient-backed known data with `pdmat`, create continuous polynomial decisions with `pdvar`, use `rhodiff` when scheduling-rate terms are present, form an affine residual comparison, and optionally replace the default Direct certificate by one of the five supported alternatives. The user then calls `toYalmip`, solves with YALMIP, and recovers values after checking the solver status. The manual develops each step from runnable examples through the underlying tensor convolution, degree alignment, rate-row, and Gram-assembly mathematics.

The representation keeps four ideas distinct. The tensor grid exactly partitions the parameter hyper-rectangle. The chosen cell-wise polynomial decision space is subject to user to determine its degree and gridding. Rate-vertex reduction is exact as rate dependence is affine. Each finite certificate is a sufficient condition over the parameter continuum.

Contents

Version History

The entries below identify the current source and documentation snapshot, the latest tagged GitHub Release, and the final documentation snapshot or latest release from each completed minor line.

v1.3.7 — August 10, 2026 — current documentation
: Aligns the printable and Web manuals with the source-indexed 192-symbol public API, restores their shared generated inventory contract, and uses explicit Cartesian-product traversal for every complete local-label sum. It retains the example-led class chapters, solver-status boundaries, and mathematical appendices of v1.3.6. The MATLAB API is unchanged, and v1.3.0 remains the latest tagged GitHub Release.

v1.3.0 — August 8, 2026 — latest tagged GitHub Release
: Consolidates the public API around `bernTable`, `elevate`, `usePolya`, `usePutinar`, `useSpPut`, `useSpBox`, and `useFullBox`. It also consolidates elevation, product, Direct, Pólya, and Bernstein–Gram assembly into reusable numeric plans while preserving represented functions, existing YALMIP variables, certificate families, constraint order, Positive Semidefinite (PSD) block dimensions, and cone partition, apart from ordinary floating-point summation differences. SparsePutinar and SparseFullBox now follow their implemented sliding tensor-window constructions in the documentation.

v1.2.4 documentation — August 5, 2026 — final v1.2 documentation snapshot
: Adds the SparsePutinar tensor-window certificate to the public API and documents its overlapping-window coefficient accumulation, canonical endpoints, validation rules, and relationship to dense Putinar and SparseFullBox. It also expands the Sum-Of-Squares (SOS) and Gram background and adds the software-paper citation. This documentation-only snapshot remains outside the tagged GitHub Releases.

v1.1.6 documentation — July 28, 2026 — final v1.1 snapshot
: Unifies tensor-grid, cell, Bernstein-degree, residual, target, and Gram notation across the printable and Web manuals. It also expands the Bernstein convolution and Gram/SOS foundations, adds the Math Concepts navigation and interactive `pdmat` addition, multiplication, and elevation explorers, and fits the Welcome three-dimensional grid at every supported rotation. The MATLAB API is unchanged.

v1.0.0 — July 19, 2026
: Marks the first stable public release of the documented package workflow. It consolidates installation and test-suite verification, the `pdbase`/`pdmat`/`pdvar`/`pdlmi` object model, continuous arbitrary-degree decision storage and rate-vertex differentiation, MATLAB-style matrix operations and indexing, and direct, Pólya, Putinar, and full-box finite certificate assembly. The 1.0 documentation also makes the affine-expression, tensor-box, certificate-sufficiency, and solver-ownership boundaries explicit. Runtime behavior remains the implementation covered by the current code and tests.

v0.4.7 — July 18, 2026
: Completed the v0.4 manual and Web explorer refresh around the LPV-to-cell-wise-Bernstein-to-solver workflow, including installer guidance, the public API reference, mathematical cross-references, and storage-transformation examples. Released by commit `c6a2e25`.

v0.3.6 — July 16, 2026
: Completed the public rename to `pdbase`/`pdmat`/`pdvar`/`pdlmi`, corrected forward Bernstein coordinates, added FullBox and Putinar certificates with independent SOS validation, hardened API boundaries, and defined entry-wise inequality behavior. Released by commit `fe74a65`.

v0.2.12 — July 13, 2026

Completed the `pdmat`/`pdvar`/`pdlmi` layers, rate-aware differentiation, display, plotting, structural algebra, direct finite assembly, Pólya selection, and deterministic reference examples and tests. Released by commit `98b63e5`.

v0.1.0 — July 3, 2026

Introduced the original grid-and-storage foundation and the first package manual, including Bernstein traversal, validation, and helper contracts. Released by commit `63c06ee`.

Chapter 1

From a Parameter Hyper-rectangle to Finite Constraints

PD-LMI requires a matrix inequality to hold at every point of a continuous parameter domain. The package represents the known data and decision variables on a hyper-rectangle with cell-wise Bernstein polynomials, then converts the resulting continuous condition into finite constraints for YALMIP. An Linear Parameter-Varying (LPV) induced-$\mathcal{L}_2$-gain problem provides the first example. A general Differentiable Parameter-Dependent Linear Matrix Inequality (DPD-LMI) problem is formulated with its finite certificate construction. The supporting algebra is developed in Appendices A and B, and the API contracts are presented in Chapters 4–6.

1.1 Motivation: LPV induced-$\mathcal{L}_2$-Gain

Consider a Linear Parameter-Varying (LPV) plant

$$\begin{aligned}\dot{x}(t) &= A(\boldsymbol{\rho}(t))x(t) + B_w(\boldsymbol{\rho}(t))w(t),\\ z(t) &= C_z(\boldsymbol{\rho}(t))x(t) + D_{zw}(\boldsymbol{\rho}(t))w(t).\end{aligned} \tag{1.1}$$

where $x \in \mathbb{R}^{n_x}$, $w \in \mathbb{R}^{n_w}$, $z \in \mathbb{R}^{n_z}$. The system matrices depend on the scheduling parameter $\boldsymbol{\rho} \in \mathbb{R}^{\ell}$, which is constrained to a hyper-rectangle $\mathcal{P} \subset \mathbb{R}^{\ell}$. Its derivative is constrained to another hyper-rectangle $\mathcal{R} \subset \mathbb{R}^{\ell}$.

Under zero initial conditions, the induced-$\mathcal{L}_2$ of such LPV system is defined as the smallest $\gamma > 0$ such that

$$\|z\|_{\mathcal{L}_2} < \gamma \|w\|_{\mathcal{L}_2}$$

A well known sufficient condition to find such γ, is to search for a differentiable parameter dependent matrix $P(\boldsymbol{\rho}) = P(\boldsymbol{\rho})^\top$ such that the following conditions hold for all $(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) \in \mathcal{P} \times \mathcal{R}$[1]:

$$P(\boldsymbol{\rho}) \succ 0, \tag{1.2}$$

$$\begin{bmatrix} \dot{P}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) + \mathrm{He}(P(\boldsymbol{\rho})A(\boldsymbol{\rho})) & P(\boldsymbol{\rho})B_w(\boldsymbol{\rho}) & C_z(\boldsymbol{\rho})^\top \\ B_w(\boldsymbol{\rho})^\top P(\boldsymbol{\rho}) & -\gamma I_{n_w} & D_{zw}(\boldsymbol{\rho})^\top \\ C_z(\boldsymbol{\rho}) & D_{zw}(\boldsymbol{\rho}) & -\gamma I_{n_z} \end{bmatrix} \prec 0. \tag{1.3}$$

where $\mathrm{He}(X) = X + X^\top$ and

$$\dot{P}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) = \sum_{s=1}^{\ell} \dot{\rho}_s \frac{\partial P}{\partial \rho_s}(\boldsymbol{\rho}).$$

This application is one instance of the general derivative-dependent PD-LMI below. The strict signs are analysis notation. The package uses non-strict semidefinite comparisons and leaves the numerical margin realization to the user.

1.2 General Problem and Assumptions

Let $\boldsymbol{y}(\boldsymbol{\rho}) = \begin{bmatrix} y_1(\boldsymbol{\rho}) & \dots & y_N(\boldsymbol{\rho}) \end{bmatrix}^\top$ be a parameter-dependent N-dimensional decision vector. The goal of this package is to formulate a finite certificate for the following DPD-LMI problem:

$$\mathcal{F}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}; \boldsymbol{y}) = F_0(\boldsymbol{\rho}) + \sum_{k=1}^{N} F_k(\boldsymbol{\rho}) y_k(\boldsymbol{\rho}) + \sum_{k=1}^{N} \sum_{s=1}^{\ell} T_{k,s}(\boldsymbol{\rho}) \dot{\rho}_s \frac{\partial y_k}{\partial \rho_s}(\boldsymbol{\rho}) \preceq 0, \quad \forall (\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}) \in \mathcal{P} \times \mathcal{R}. \tag{1.4}$$

The package uses the following assumptions throughout the modeling and certification process.

- The scheduling set and rate set are hyper-rectangles.

$$\mathcal{P} = \prod_{s=1}^{\ell} [\underline{\rho}_s, \overline{\rho}_s], \qquad \mathcal{R} = \prod_{s=1}^{\ell} [\underline{v}_s, \overline{v}_s], \qquad \mathcal{V}_{\mathcal{R}} = \prod_{s=1}^{\ell} \{\underline{v}_s, \overline{v}_s\}.$$

- The decision vector $\boldsymbol{y}$ is continuous and is represented on the chosen grid by cell-wise polynomials of the selected degree. Its values agree on shared faces, although its one-sided derivatives may differ.
- The known data F_0, F_k, and $T_{k,s}$ are continuous and are represented on the chosen grid by cell-wise polynomials of appropriate degree.
- MATLAB and YALMIP use non-strict semidefinite comparisons. A theoretical strict condition therefore requires an explicit user-selected margin, for example $-\mathcal{F}(\boldsymbol{\rho}, \dot{\boldsymbol{\rho}}; \boldsymbol{y}) - \varepsilon I \succeq 0$. Failure of a finite certificate leaves feasibility of (1.4) inconclusive.

1.3 Gridding the Parameter Domain

For each parameter direction $s = 1, \dots, \ell$, choose an independent, possibly nonuniform axis grid

$$\mathcal{G}_s = \{\rho_s^{(1)}, \dots, \rho_s^{(k_s)}\}, \qquad \underline{\rho}_s = \rho_s^{(1)} < \dots < \rho_s^{(k_s)} = \overline{\rho}_s.$$

Their Cartesian product is

$$\mathcal{G} = \mathcal{G}_1 \times \dots \times \mathcal{G}_\ell,$$

which contains $\prod_{s=1}^{\ell} k_s$ physical nodes. The multi-index $\boldsymbol{c} = (c_1, \dots, c_\ell)$, with $c_s \in \{1, \dots, k_s - 1\}$, defines one physical cell and its width in parameter direction s:

$$\mathcal{H}_{\boldsymbol{c}} = \prod_{s=1}^{\ell} [\rho_s^{(c_s)}, \rho_s^{(c_s+1)}], \qquad h_s^{\boldsymbol{c}} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)} > 0.$$

Thus $\mathcal{G}$ has $\prod_{s=1}^{\ell} k_s$ physical nodes and determines $\prod_{s=1}^{\ell} (k_s - 1)$ physical cells whose union is exactly $\mathcal{P}$. Gridding remains a modeling approximation as it restricts the search for $\boldsymbol{y}$ from a large $\mathcal{P}$ to a bunch of selected smaller physical cells $\mathcal{H}_{\boldsymbol{c}}$.

For example, take the following nonuniform two-dimensional grid on $[0, 1]^2$:

$$\mathcal{G}_1 = \{0, 0.4, 1\}, \qquad \mathcal{G}_2 = \{0, 0.25, 1\}.$$

Their tensor grid $\mathcal{G} = \mathcal{G}_1 \times \mathcal{G}_2$ has nine physical nodes and exactly partitions $[0,1]^2$ into four physical cells. The cell indices are $(1,1)$, $(1,2)$, $(2,1)$, and $(2,2)$.

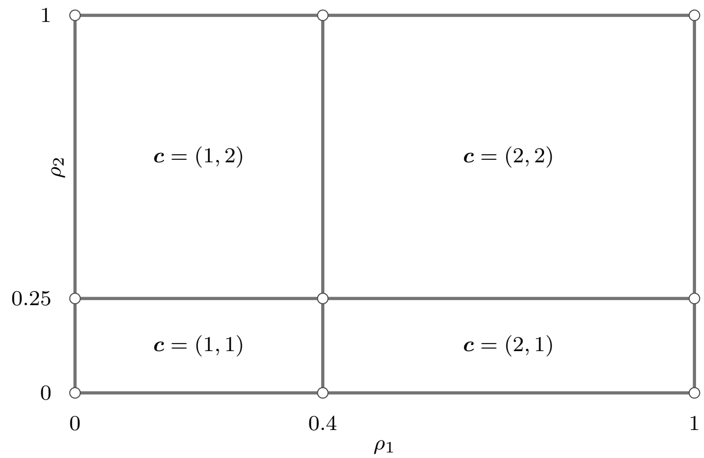


Figure 1.1: A nonuniform two-dimensional tensor grid with nine physical nodes and four physical cells. Every point of the box belongs to at least one cell, and the lines are exact cell boundaries of the domain.

Each physical cell has its own local coordinate $\boldsymbol{\alpha} \in [0,1]^\ell$ which will be defined in the next section, and a separate finite coefficient set.

1.4 Bernstein Basis on Each Cell

Fix a physical cell $\mathcal{H}_{\boldsymbol{c}}$. As different physical cells may have different side lengths, we need to first express every cell on the same domain. Define the forward affine map

$$\phi_{\boldsymbol{c}} : [0,1]^\ell \longrightarrow \mathcal{H}_{\boldsymbol{c}}, \qquad \phi_{\boldsymbol{c},s}(\alpha_s) = \rho_s^{(c_s)} + h_s^{\boldsymbol{c}} \alpha_s.$$

Its inverse defines the local coordinate $\boldsymbol{\alpha} = \phi_{\boldsymbol{c}}^{-1}(\boldsymbol{\rho})$, where $\alpha_s = \left[\rho_s - \rho_s^{(c_s)}\right] / h_s^{\boldsymbol{c}}$, and maps every physical hyper-rectangle to the unit box $[0,1]^\ell$. Since $\boldsymbol{c}$ is arbitrary, this normalization gives a uniform coordinate domain for all cell-wise expressions.

On this common unit box, we use the normalized tensor-product Bernstein basis

$$B_{\boldsymbol{i}}^{\boldsymbol{m}}(\boldsymbol{\alpha}) = \prod_{s=1}^{\ell} B_{i_s}^{m_s}(\alpha_s) = \prod_{s=1}^{\ell} \binom{m_s}{i_s} (1-\alpha_s)^{m_s - i_s} \alpha_s^{i_s}, \qquad \boldsymbol{i} \in \prod_{s=1}^{\ell} \{0, \ldots, m_s\},$$

where the vector $\boldsymbol{m} = (m_1, \ldots, m_\ell)$ specifies the modeled degree in each parameter direction.

For any cell-wise quantity X, write its pullback as $X^{(\boldsymbol{c})} = X \circ \phi_{\boldsymbol{c}}$. A cell-wise polynomial is then represented by

$$X^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i} \in \prod_{s=1}^{\ell} \{0, \ldots, m_s\}} B_{\boldsymbol{i}}^{\boldsymbol{m}}(\boldsymbol{\alpha}) X^{(\boldsymbol{c})}[\boldsymbol{i}]. \tag{1.5}$$

Each cell stores $\prod_{s=1}^{\ell}(m_s + 1)$ coefficients $X^{(\boldsymbol{c})}[\boldsymbol{i}]$. These local labels identify coefficient positions inside the cell and are separate from the physical sample points. The unequal components of $\boldsymbol{m}$ allow different polynomial degrees in different parameter directions.

The polynomial expressed in the Bernstein basis in (1.5) is called a ***Bernstein polynomial***. On a boundary of the unit box, only the coefficients on the corresponding face are active. Continuity on the shared face of two adjacent cells therefore reduces to equality of the corresponding coefficients. The Bernstein representation also supports exact cell-wise differentiation, which is required by the derivative terms in a DPD-LMI.

To make the cell-wise storage concrete, return to the two-dimensional grid in Fig 1.1 and consider

$$a(\rho_1, \rho_2) = 1 + \rho_1^3 + \rho_2 + \rho_1\rho_2.$$

Choose the anisotropic degree $\boldsymbol{m} = (3, 1)$, written as `Degree=[3 1]` in MATLAB. Every physical cell then has eight local labels $\boldsymbol{i} = (i_1, i_2) \in \{0, 1, 2, 3\} \times \{0, 1\}$. The following input constructs one known `pdmat` on all four cells and inspects the coefficients stored for cell $[1, 1]$.

```
grid = {[0 0.4 1], [0 0.25 1]};
a = pdmat(grid, @(rho1, rho2) ...
    1 + rho1.^3 + rho2 + rho1.*rho2, ...
    Degree=[3 1]);

labelCoefficientMap = [a.lbls(), ...
    cell2mat(a.LocalValues{1}{1}).']
```

```
labelCoefficientMap = 8×3 double
         0         0    1.0000
         0    1.0000    1.2500
    1.0000         0    1.0000
    1.0000    1.0000    1.2833
    2.0000         0    1.0000
    2.0000    1.0000    1.3167
    3.0000         0    1.0640
    3.0000    1.0000    1.4140
```

In Fig 1.1, cell $[1, 1]$ is $\mathcal{H}_{(1,1)} = [0, 0.4] \times [0, 0.25]$. Its forward map is $\phi_{(1,1)}(\boldsymbol{\alpha}) = (0.4\alpha_1, 0.25\alpha_2)$, so the pullback of a to the unit box is

$$a^{(1,1)}(\alpha_1, \alpha_2) = 1 + 0.064\alpha_1^3 + 0.25\alpha_2 + 0.1\alpha_1\alpha_2.$$

Substituting the eight stored coefficients into (1.5) gives the Bernstein form

$$\begin{aligned} a^{(1,1)}(\alpha_1, \alpha_2) = B_0^3(\alpha_1)B_0^1(\alpha_2) &+ \frac{5}{4}B_0^3(\alpha_1)B_1^1(\alpha_2) \\ &+ B_1^3(\alpha_1)B_0^1(\alpha_2) + \frac{77}{60}B_1^3(\alpha_1)B_1^1(\alpha_2) \\ &+ B_2^3(\alpha_1)B_0^1(\alpha_2) + \frac{79}{60}B_2^3(\alpha_1)B_1^1(\alpha_2) \\ &+ \frac{133}{125}B_3^3(\alpha_1)B_0^1(\alpha_2) + \frac{707}{500}B_3^3(\alpha_1)B_1^1(\alpha_2). \end{aligned}$$

Each row of `labelCoefficientMap` gives one term in this expansion. Its first two columns contain the local label (i_1, i_2), and its third column contains the corresponding coefficient from `a.LocalValues{1}{1}`. Thus in the cell $\mathcal{H}_{(1,1)}$, `pdmat` stores the coefficients for labels $(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1), (3, 0), (3, 1)$ in that order. The two nested indices select the physical cell, while the eight leaf columns select local Bernstein labels inside that cell. These eight values represent the polynomial over the complete cell and leave the physical grid unchanged. Sections A.3 and A.6 to A.8 give the complete traversal, higher-degree, elevation, and multiplication rules.

1.5 From a Continuous Cell Condition to Direct

Fix any physical cell $\mathcal{H}_{\boldsymbol{c}}$. After all local operations have been aligned to the assembled residual degree $\boldsymbol{M}$, the pullback of the (1.4) has the general form on each cell:

$$\mathcal{F}^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i}\in\prod_{s=1}^{\ell}\{0,\ldots,M_s\}} B_{\boldsymbol{i}}^{\boldsymbol{M}}(\boldsymbol{\alpha})\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}] \preceq 0.$$

Here $\boldsymbol{\alpha} = \phi_{\boldsymbol{c}}^{-1}(\boldsymbol{\rho})$, $B_{\boldsymbol{i}}^{\boldsymbol{M}}$ is the direction-wise degree-$\boldsymbol{M}$ tensor-product Bernstein basis, and $\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}]$ is the corresponding residual coefficient. Define the sign-normalized target by

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = -\mathcal{F}^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = -\sum_{\boldsymbol{i}\in\prod_{s=1}^{\ell}\{0,\ldots,M_s\}} B_{\boldsymbol{i}}^{\boldsymbol{M}}(\boldsymbol{\alpha})\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}].$$

The theoretical target is $S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) \succeq 0$ on the unit box. The Direct certificate imposes

$$-\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}] \succeq 0 \quad \forall \boldsymbol{i} \in \prod_{s=1}^{\ell}\{0,\ldots,M_s\}.$$

Because every $B_{\boldsymbol{i}}^{\boldsymbol{M}}$ is nonnegative on the unit box and the basis functions sum to one,

$$\left[-\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}] \succeq 0 \text{ for all } \boldsymbol{i}\right] \quad \Longrightarrow \quad S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) \succeq 0 \quad \text{for all } \boldsymbol{\alpha} \in [0,1]^{\ell}.$$

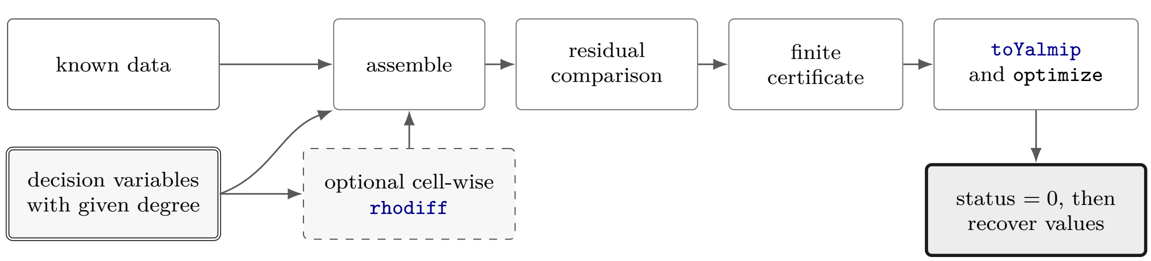
Repeating this implication for every physical cell and every stored rate row produces finitely many YALMIP constraints that suffice for the continuous-domain condition. The implication is only sufficient because a positive semidefinite matrix polynomial may have an indefinite Bernstein coefficient. Therefore, failure of Direct is inconclusive. The following alternatives provide additional sufficient certificates that may succeed when Direct fails.

1.5.1 Opt-in alternatives

Direct is the default finite certificate. The package also provides opt-in Pólya, Putinar, SparsePutinar, SparseFullBox, and FullBox alternatives. Their orders, Gram constructions, support rules, and validation boundaries belong to the detailed `pdlmi` chapter and Appendix B. Each alternative supplies a sufficient continuum certificate that is separate from parameter sampling and the solver call.

1.6 The Complete Path to YALMIP

The full workflow is as follows: represent known data, create continuous decision functions, differentiate them cell by cell when rate terms are present, assemble and compare a residual, select a finite certificate, export it, solve, validate the status, and only then recover numerical values.



```
yalmip('clear')
grid = {[0 0.5 1]};
rateBounds = [-0.2 0.2];

a = pdmat(grid, @(rho) 1 + rho + rho.^2, ...
    Degree=2, RateBounds=rateBounds);
P = pdvar(1, grid, Degree=2, RateBounds=rateBounds);
dP = rhodiff(P);

residual = P - 0.1 * dP - a;
certificate = residual >= 1e-6;
F = certificate.toYalmip();
opts = sdpsettings('solver', 'mosek', 'verbose', 0);
sol = optimize(F, [], opts);
assert(sol.problem == 0, sol.info)
solvedP = value(P);
```

Here `rhodiff` evaluates the affine derivative term at the vertices of the stored rate box. That vertex reduction is exact. The comparison creates a Direct `pdlmi` by default, and an opt-in certificate selector could be inserted before `toYalmip`. Solver settings and objectives remain ordinary YALMIP choices. Recovery of `solvedP` requires `sol.problem == 0`. Any infeasible, numerical, unknown, or otherwise nonzero status requires diagnosis before decisions or objectives are retained. A failed sufficient certificate leaves feasibility of the original continuum condition inconclusive.

1.7 Manual Outline

Chapter 2 covers installation and verification. The `pdbase` chapter explains shared backend geometry, after which Chapters 4 and 5 develop known data and decision functions. Chapter 6 gives the complete certificate-selection and YALMIP-export contracts, the worked examples assemble application-specific residuals and objectives, and Appendices A and B derive the Bernstein and certificate mathematics.

Chapter 2

Install and Verify the Package

2.1 One-Call Installation

Start MATLAB in the downloaded repository and run command `install_pd_lmi` once. The installer adds only the repository root to the end of the MATLAB path. MATLAB resolves the package's class, package, and private directories from that root. Dependency installation and `startup.m` configuration remain user-managed operations.

```
report = install_pd_lmi();
```

The returned structure records the package root, the YALMIP root discovered on the current MATLAB path, the working Semidefinite Program (SDP) solver, the paths added by this run, and whether the path was persisted:

```
report.PackageRoot
report.YALMIPRoot
report.Solver
report.AddedPaths
report.Persisted
```

YALMIP must already be on the active MATLAB path. The installer validates `yalmip`, `sdpvar`, `sdpsettings`, `optimize`, and `getavailablesolvers`. It then probes available SDP solvers in the fixed commercial-first order MOSEK, COPT, SeDuMi, SDPT3, and LMILAB using a tiny bounded SDP. A failed probe falls through to the next candidate. If any precondition, shadow check, solver probe, or path persistence operation fails, the installer restores the original MATLAB path and stops with one of these stable errors:

```
install_pd_lmi:MissingYALMIP
install_pd_lmi:IncompleteYALMIP
install_pd_lmi:NoWorkingSDPSolver
install_pd_lmi:PathConflict
install_pd_lmi:PathSaveFailed
```

Repeated successful calls are idempotent. When the repository root is already present, the installer preserves the single path entry and returns an empty `AddedPaths`.

2.2 Validate the Installation

After the installer has selected and recorded a working solver, run the current MATLAB test entry point from the repository root:

```
results = tests.run_all();
```

The entry point covers installer rollback and persistence behavior, helper utilities, `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. YALMIP-backed tests and the solver smoke tests use the same automatic commercial-first solver policy as the installer. Their diagnostics identify the actual solver used.

2.3 Run a Minimal Solver Example

The installer already rejects shadowed package classes before it persists the path. To inspect the selected solver and exercise the solver-facing assembly boundary, use the report together with a small known-data example:

```
report.Solver
A = pdmat({[0 1]}, {1, 3}, Degree=1);
T = bernTable(A, "oneLine");
disp(T)
```

```
    CellSubscript           Expression
    _____________    _______________________

        {[1]}        "(1-alpha)*1 + alpha*3"
```

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=4);
C = P >= 0;
F = toYalmip(C);
Cputinar = C.usePutinar();
Fputinar = toYalmip(Cputinar);
CspPutinar = C.useSpPut(2);
FspPutinar = toYalmip(CspPutinar);
sparsePutinarState = [CspPutinar.UseSparsePutinar, ...
    CspPutinar.SparsePutinarOrder, CspPutinar.CliqueSize, ...
    numel(CspPutinar.Constraints)]
Csp = C.useSpBox(2);
Fsp = toYalmip(Csp);
Cbox = C.useFullBox();
Fbox = toYalmip(Cbox);
```

```
sparsePutinarState = 1x4 double
     1     2     2     8
```

`F` is the direct coefficient certificate. For the same stored residual, `Fputinar` is dense Putinar, `FspPutinar` is clique-size-two SparsePutinar, `Fsp` is width-two SparseFullBox, and `Fbox` is FullBox. Any one can be passed to ordinary YALMIP calls. Use the selected solver from the report when an explicit solver setting is useful:

```
opts = sdpsettings('solver', report.Solver, 'verbose', 0);
sol = optimize(F, [], opts);
```

2.4 Citing GriD-LMIA

If GriD-LMIA supports published work, cite the software paper [18]:

> Yicheng Xu and Faryar Jabbari, "GriD-LMIA: A Gridding-Based Assembler for Solving Differentiable Parameter-Dependent Linear Matrix Inequalities," arXiv:2608.03175, 2026. https://arxiv.org/abs/2608.03175.

The corresponding reusable BibTeX entry uses citation key `xu2026gridlmia`:

```
@article{xu2026gridlmia,
  title   = {GriD-LMIA: A Gridding-Based Assembler for Solving
             Differentiable Parameter-Dependent Linear Matrix Inequalities},
  author  = {Xu, Yicheng and Jabbari, Faryar},
  journal = {arXiv preprint arXiv:2608.03175},
  year    = {2026},
  url     = {https://arxiv.org/abs/2608.03175}
}
```

2.5 Navigate the Reference

Category	Entries
Backend	`pdbase`, constructor, `cells`, `lbls`, `coeffs`, `ncell`, `ncoeff`, `npar`, shape protocols, shared matrix operations
Known data	`pdmat`, construction, `evaluate`, `bernTable`, `disp/display`, `plot`, operators, matrix methods
Decision variables	`pdvar`, construction, `rhodiff`, affine/mixed algebra, matrix methods, comparisons
LMIs	`pdlmi`, construction, direct coefficient assembly, `usePolya`, `usePutinar`, `useSpPut`, `useSpBox`, `useFullBox`, `toYalmip`, solver examples

Chapter 3

pdbase: Grid and Storage Backend

pdbase is the developer-facing backend parent for pdmat and pdvar. Ordinary users should model known data with pdmat and decision expressions with pdvar. This section documents pdbase because it owns the shared storage, traversal, inspection, matrix-shape, unary-transformation, reduction, and reordering implementations inherited by both public modeling classes. Central implementation preserves these methods as public pdmat and pdvar behavior. Their class-specific syntax, examples, validation, and limitations remain documented in sections 4.10 and 5.9.

Its traversal contract is the concrete form of the notation in section A.7: `cells()` enumerates physical indices $\boldsymbol{c}$, `lbls()` enumerates local labels $\boldsymbol{i}$, and `coeffs(c)` returns $C^{(\boldsymbol{c})}[\boldsymbol{i}]$ in that label order. If a cell contains derivative data, its rows enumerate the constructed vertices of $\mathcal{R}$, while the physical cell and local label remain fixed. Public elevate and coefficient algebra preserve this separation. The consolidated sparse elevation and planned product kernels are derived in sections A.6 and A.8, while this chapter keeps public calls and the protected table, mapping, indexing, and grid-refinement context needed to understand their behavior.

3.1 API Map

Task	Function or field
Create backend object	`pdbase`
Inspect cells/labels	`cells`, `lbls`, `coeffs`
Evaluate a stored function	`evaluate`
Differentiate a rate box	`rhodiff`
Inspect traversal counts	`ncell`, `ncoeff`, `npar`
Inspect matrix shape	`size/height/width/length/ndims/numel`
Transform matrix payloads	unary signs, transpose, diagonal, reshape, reductions, reordering, repetition, and concatenation dispatch
Elevate stored coefficients	`elevate`
Backend algebra mechanisms	`bernTbl/mapVals/matSubs`, `mergeGrid`
Read storage fields	`GridInfo`, `LocalValues`, metadata fields

3.2 Construct pdbase Objects

Syntax

```
obj = pdbase(gridVectors, matrixSize, degree)
obj = pdbase(gridVectors, matrixSize, degree, localValues)
obj = pdbase(..., IsContinuous=tf)
obj = pdbase(..., ContainsDecision=tf)
obj = pdbase(..., NumRateRows=nRows)
obj = pdbase(..., RateBounds=rb)
obj = pdbase(..., SourceSummary=txt)
obj = pdbase(..., ValidationMode=mode)
```

Arguments

- `gridVectors` is a cell vector with one strictly increasing node vector per parameter, such as `{[0 1 2], [10 20]}`.
- `matrixSize` is a positive two-element matrix size, such as `[2 2]`.
- `degree` is one finite nonnegative integer scalar shorthand or an ℓ-element direction-wise vector. It is stored as a 1-by-ℓ row $\boldsymbol{m} = \begin{bmatrix} m_1 & \dots & m_\ell \end{bmatrix}$, and a scalar expands uniformly.
- `localValues` is the nested physical-cell storage. When omitted, `pdbase` creates a zero coefficient-backed object.
- `IsContinuous` and `ContainsDecision` are logical metadata scalars. They default to false, and subclasses set them to describe their payload contract.
- `NumRateRows` is a nonnegative integer that declares explicit rate-vertex rows in each supplied coefficient cell. Zero selects ordinary one-row storage. A positive value must equal the number of distinct vertices generated by `RateBounds`.
- `RateBounds` is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as `RateBounds=[-1 1; -2 2]`. Bounds may be stored as metadata while `NumRateRows` remains zero.
- `SourceSummary` is inspectable origin text and defaults to `"coefficient-backed"`.
- `ValidationMode` is transient scalar text `"fast"` or `"strict"`, case-insensitively, with default `"fast"`. For a raw public `pdbase` construction in fast mode, repeated coefficient structure is checked only in the first supplied physical cell, so malformed coefficient counts, payloads, or rate-row layouts in later supplied cells can remain undetected. Strict mode audits every supplied cell. Grid, metadata, and options are validated globally in both modes, and the mode remains transient call state. The public `pdmat` and `pdvar` constructors normalize their source data before applying their class-specific guarantees, so their fast-mode contracts follow the class-specific descriptions in their constructor sections.

Examples and output

The following example constructs a three-parameter parent object with anisotropic degree $\boldsymbol{m} = (1, 3, 0)$. The grid has two physical intervals in the first parameter direction and one interval in each remaining direction.

```
obj = pdbase({[0 1 2], [10 20], [5 9]}, [2 2], [1 3 0]);
objectClass = class(obj)
matrixSize = obj.MatrixSize
degree = obj.Degree
cellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
objectSize = size(obj)
```

```
objectClass = 'pdbase'
matrixSize = 1×2 double
     2     2

degree = 1×3 double
     1     3     0
cellCount = 2
coefficientCount = 8
parameterCount = 3
objectSize = 1×2 double
     2     2
```

The printed `ncoeff` value is $\prod_{s=1}^{\ell}(m_s + 1) = (1 + 1)(3 + 1)(0 + 1) = 8$. The printed `npar` value is 3, which means $\ell = 3$. This count is part of the backend inspection contract and appears directly as an operation result.

Remarks

- Malformed grids and node vectors raise `pdbase:InvalidGrid` or `pdbase:InvalidGridVector`.
- Invalid matrix sizes, degrees, rate bounds, rate-row counts, or validation modes raise `pdbase:InvalidMatrixSize`, `pdbase:InvalidDegree`, `pdbase:InvalidRateBounds`, `pdbase:InvalidOptions`, or `pdbase:InvalidValidationMode`.
- The nested tree must contain one cell per physical cell and $\prod_{s=1}^{\ell}(m_s + 1)$ coefficient columns per cell. Violations raise `pdbase:InvalidLocalValues` or `pdbase:InvalidCoefficientCell`.
- Payload matrices must have the declared size and be finite, real, and numeric or supported real YALMIP expressions. Malformed payloads raise `pdbase:InvalidCoefficientPayload`.
- A positive `NumRateRows` requires nonempty `RateBounds`, must match the number of distinct rate-box vertices, and requires every supplied cell to use that many rows.

3.3 Public Properties

The members of pdbase are public, read-only properties. Users inspect them through dot access, such as `obj.Degree` or `obj.GridInfo.Vectors`, but cannot assign new values to them because their set access is private. Constructors and package operations return new objects when stored information must change.

Property	Stored information
`GridInfo`	Stores the tensor-grid geometry in a scalar structure. • `Vectors` contains one axis grid vector per parameter • `Points` contains all physical grid nodes in combination order. • `Bounds` stores the lower and upper endpoint of each parameter range • `NumNodes` stores the number of axis grid nodes in each parameter direction for traversal and validation
`MatrixSize`	Stores the two-element row $[n_r, n_c]$ that gives the row and column dimensions of every matrix coefficient or matrix expression in the object.
`Degree`	Stores the local Bernstein degree as $[m_1, \ldots, m_\ell]$. Its s-th entry is the degree used inside every physical cell along parameter direction s.
`LocalValues`	Stores the cell-wise coefficient data as an ℓ-level nested cell tree. The indices $i_1, i_2, \ldots, i_\ell$ select physical cell $[i_1, i_2, \ldots, i_\ell]$. Its cell stores coefficients in `lbls(obj)` order. An ordinary cell is one flat row, while a derivative cell can contain one row per rate vertex.
`IsContinuous`	Stores a logical continuity flag. A true value records that adjacent physical cells represent matching values on their shared faces. At the pdbase level this is metadata. The subclass that constructs the object owns the corresponding boundary-sharing guarantee.
`ContainsDecision`	Stores a logical flag that is true when at least one coefficient payload contains YALMIP decision variables and false for known coefficient data.
`NumRateRows`	Stores the number of explicit rate-vertex rows in each `LocalValues` cell. Zero denotes ordinary coefficient storage, including objects that store `RateBounds` only as metadata.
`RateBounds`	Stores the parameter-rate box as an ℓ-by-2 matrix. Row s contains the lower and upper bounds for $\dot{\rho}_s$. These bounds are stored separately from `LocalValues` and are used by `rhodiff` and `pdlmi`.
`SourceSummary`	Stores a short text label describing how the coefficient data originated, such as `coefficient-backed`, `decision`, or `derivative`.

The following object has three physical cells in the first parameter direction, two in the second, and four local coefficients for vector degree $\boldsymbol{m} = (1, 1)$ in every cell. Its coefficients are the four corner values of

$$p(\rho_1, \rho_2) = \rho_1 + \rho_2$$

in `lbls(obj)` order. Adjacent cells reuse the coefficients on their shared edge, so the stored piecewise polynomial is continuous.

```
localValues = { ...
    {{10, 20, 11, 21}, {20, 30, 21, 31}}, ...
    {{11, 21, 12, 22}, {21, 31, 22, 32}}, ...
    {{12, 22, 13, 23}, {22, 32, 23, 33}}};

obj = pdbase({[0 1 2 3], [10 20 30]}, ...
    [1 1], [1 1], localValues, IsContinuous=true);
```

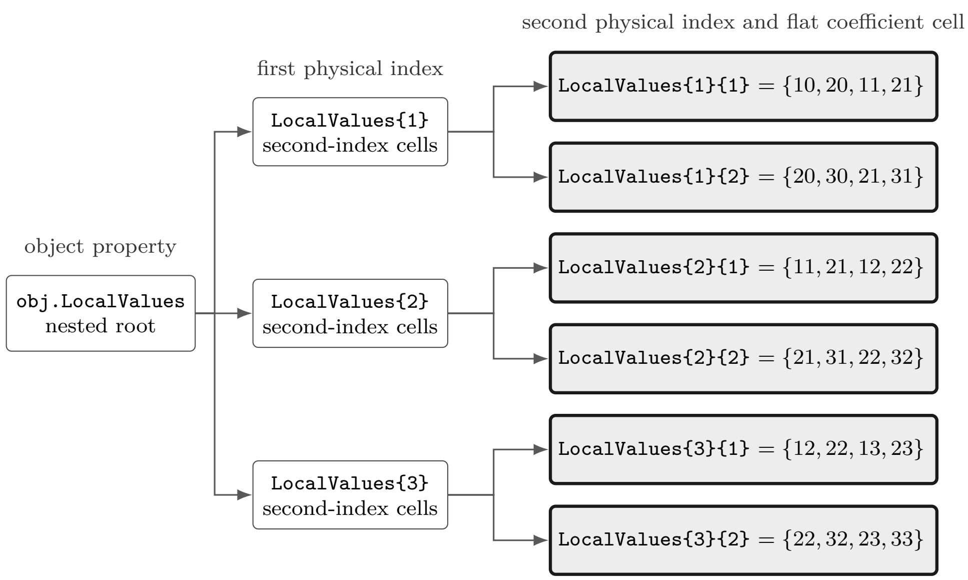


Figure 3.1: Concrete nested `LocalValues` storage for the 3×2 physical-cell grid in the example. The first cell index selects one of the three outer entries, and the second selects one of its two cells. Every cell contains four coefficients in the `lbls(obj)` order $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$. Repeated values on adjacent cells are the shared-edge coefficients.

Within physical cell $[i_1, i_2]$, let α_s be the normalized local coordinate in parameter direction s, and write $B^{(1,1)}_{(r_1,r_2)} = B^1_{r_1}(\alpha_1)B^1_{r_2}(\alpha_2)$. The next diagram rearranges the four terms from the `Expression` column of `bernsteinTable(...,"oneLine")` into their physical 2×2 corner layout. The upper row contains labels $(0, 1)$ and $(1, 1)$, and the lower row contains $(0, 0)$ and $(1, 0)$. This spatial display differs from the flat `lbls(obj)` storage order $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$, but represents the same polynomial. Every cell resets (α_1, α_2) to $[0, 1]^2$.

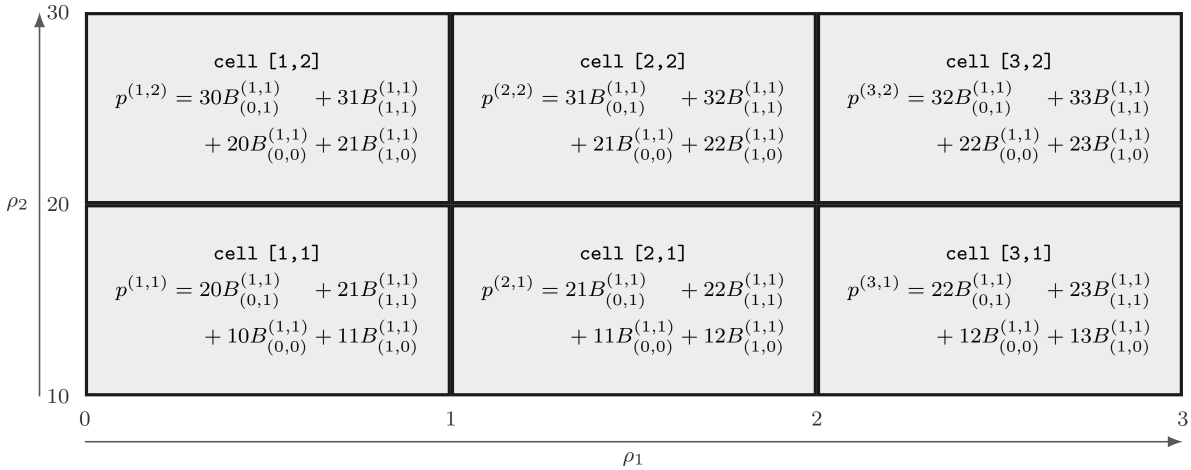


Figure 3.2: The six physical cells and their stored Bernstein polynomials of vector degree $\boldsymbol{m} = (1,1)$, with the four terms placed according to their physical corners. Cells $[1,1]$ and $[2,1]$ both use 11 and 21 on their common vertical edge, while cells $[1,1]$ and $[1,2]$ both use 20 and 21 on their common horizontal edge.

The shared coefficients give identical edge polynomials. For example,

$$\begin{aligned} p^{(1,1)}(1,\alpha_2) &= 11(1-\alpha_2) + 21\alpha_2 = p^{(2,1)}(0,\alpha_2), \\ p^{(1,1)}(\alpha_1,1) &= 20(1-\alpha_1) + 21\alpha_1 = p^{(1,2)}(\alpha_1,0). \end{aligned}$$

The same check applies to the remaining shared edges, so the object represents the continuous function $p(\rho_1,\rho_2) = \rho_1 + \rho_2$.

`LocalValues` uses cell-wise access, and direct nested access is equivalent to `coeffs`. For any valid physical-cell subscript,

$$\texttt{obj.LocalValues\{i1\}\{i2\}}\cdots\texttt{\{i}\ell\texttt{\}} = \texttt{obj.coeffs([i1, i2, ..., i}\ell\texttt{])}.$$

For example, `obj.LocalValues{3}{2}` and `obj.coeffs([3, 2])` both return `{22, 32, 23, 33}` in Fig 3.1.

3.4 Inspect `cells`, `lbls`, and `coeffs`

These three methods expose the two traversal levels in `LocalValues`. The pair `cells` and `coeffs` locates a physical-cell cell in the nested cell array, while `lbls` identifies the local Bernstein coordinate attached to each coefficient column in that cell.

3.4.1 Physical-Cell Traversal with `cells` and `coeffs`

Syntax

```
subs = cells(obj)
subs = obj.cells()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments

- `obj` is a `pdbase` object or a `pdmat`/`pdvar` subclass instance using the shared cell-wise storage convention.
- `cellSubs` selects one physical cell for `coeffs`. Supply a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `{2,1}`.
- `subs` has one row for each physical cell and one column for each parameter. Row `subs(k,:)` is the physical-cell coordinate used at traversal step `k`.
- `c` is the coefficient cell stored at the selected physical cell. An ordinary cell is a flat cell row, while a rate-dependent cell has one row per active rate vertex.

Each row of `subs` gives the complete path into the nested storage. If `cellSubs = subs(k,:)`, its entries are applied from left to right as successive indices of `obj.LocalValues`, and `coeffs(obj,cellSubs)` returns the cell at the end of that path. Thus `cells` supplies the physical-cell coordinates for traversal, and `coeffs` follows one coordinate to its stored cell.

Remarks

`coeffs` requires one positive integer physical-cell index per parameter, within the corresponding cell count. Invalid numeric or cell-array selectors raise `pdbase:InvalidCellSubs`.

Examples and output

This object has two physical cells. The second row returned by `cells` gives the path `[2 1]`, so passing that row to `coeffs` returns `obj.LocalValues{2}{1}`.

```
localValues = {{{10, 20, 11, 21}}, ...
    {{11, 21, 12, 22}}};
obj = pdbase({[0 1 2], [10 20]}, [1 1], ...
    [1 1], localValues);
physicalCells = cells(obj)
cellSubs = physicalCells(2, :)
selectedcell = coeffs(obj, cellSubs);
selectedCoefficients = cell2mat(selectedcell)
```

```
physicalCells = 2×2 double
     1     1
     2     1

cellSubs = 1×2 double
     2     1

selectedCoefficients = 1×4 double
    11    21    12    22
```

3.4.2 Local Bernstein Coordinates with `lbls`

Syntax

```
labels = lbls(obj)
labels = obj.lbls()
```

Arguments

`lbls(obj)` returns the local Bernstein labels in lexicographical order. Row `j` gives the local multi-index for coefficient column `j` of every cell returned by `coeffs`.

Examples and output

Each row of `labelToCoefficient` pairs one local Bernstein label with the coefficient stored at the same position in `selectedcell`.

```
labels = lbls(obj)
labelToCoefficient = [labels, cell2mat(selectedcell(:))]
```

```
labels = 4×2 double
     0     0
     0     1
     1     0
     1     1

labelToCoefficient = 4×3 double
     0     0    11
     0     1    21
     1     0    12
     1     1    22
```

3.4.3 Traversal Counts with `ncell`, `ncoeff`, and `npar`

Syntax

```
n = ncell(obj)
n = obj.ncell()
n = ncoeff(obj)
n = obj.ncoeff()
n = npar(obj)
n = obj.npar()
```

Arguments

- `ncell(obj)` returns the number of physical cells traversed by `cells`, which is the number of rows in the array returned by `cells`, i.e., $\prod_{s=1}^{\ell}(k_s - 1)$.
- `ncoeff(obj)` returns the number of local Bernstein coefficient positions listed by `lbls`, which is the number of columns in the array returned by `lbls`, i.e., $\prod_{s=1}^{\ell} m_s$.
- `npar(obj)` returns the number of parameter coordinates in each traversal row or local label, which is the number of columns in the arrays returned by `cells` and `lbls`, i.e., ℓ.

Examples and output

For the same result in section 3.4.2, `cells` has two rows, `lbls` has four rows, and both arrays have two columns.

```
physicalCellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
```

```
physicalCellCount = 2
coefficientCount = 4
parameterCount = 2
```

3.5 `evaluate` Stored Coefficients

Syntax

```
val = evaluate(obj, point)
val = obj.evaluate(point)
```

Arguments

- `point` is a finite real vector that has ℓ entries and lies within the grid bounds.
- `val` is the evaluated value of `obj` at that point. It has the same size as `obj.MatrixSize`. If `obj.NumRateRows>0`, then `val` is a 1-by-N cell array in stored row order, where N is the number of rate rows.

Examples and output

```
obj = pdbase({[0 2]}, [1 1], 2, {{1, 3, 9}});
valueAtOne = obj.evaluate(1)
rateObj = pdbase({[0 2]}, [1 1], 1, ...
    {{1, 3; 10, 14}}, NumRateRows=2, ...
    RateBounds=[-1 1]);
rateRows = cell2mat(rateObj.evaluate(0.5))
```

```
valueAtOne = 4
rateRows = 1×2 double
    1.5000   11.0000
```

Remarks

Malformed points raise `class(obj) + ":InvalidPoint"` and out-of-bounds points raise `class(obj) + ":PointOutOfBounds"`. `pdmat` overrides the function-backed path so exact stored handles remain exact, while coefficient-backed `pdmat` and all `pdvar` data use this row-aware backend reconstruction. Evaluation preserves the object, certificate state, and YALMIP assignments.

3.6 Differentiate Stored Coefficients with `rhodiff`

Syntax

```
D = rhodiff(A)
D = rhodiff(A, rateBounds)
```

Arguments

`rhodiff` differentiates coefficient-backed storage cell by cell and returns the same dynamic class as `A`. The form `rhodiff(A)` uses `A.RateBounds`. Empty stored bounds require explicit `rateBounds`. Explicit bounds supply missing metadata, but when `A.RateBounds` is already stored they must match it exactly. Let $\boldsymbol{e}_s \in \mathbb{R}^\ell$ denote an indicator vector, in which the s-th entry is one and zero otherwise. On physical cell $\boldsymbol{c}$, the partial coefficient along a nonzero-degree axis s is

$$\left(\partial_s A^{(\boldsymbol{c})}\right)[\boldsymbol{i}] = \frac{m_s}{h_s^{c}}\left(A^{(\boldsymbol{c})}[\boldsymbol{i}+\boldsymbol{e}_s] - A^{(\boldsymbol{c})}[\boldsymbol{i}]\right), \qquad i_s = 0, \ldots, m_s - 1.$$

`rhodiff` processes the parameter directions in increasing order $s = 1, \ldots, \ell$. For every direction with $m_s > 0$, it forms the cell-wise partial above, which has degree $\boldsymbol{m} - \boldsymbol{e}_s$, while a direction with

$m_s = 0$ contributes the zero partial. In one parameter, this reduced degree is retained, so $\boldsymbol{m} = (m_1)$ returns $(m_1 - 1)$ when $m_1 > 0$, while (0) remains (0). In two or more parameters, each nonzero partial is elevated exactly to the common tensor degree $\boldsymbol{m}$ = `A.Degree` before the directions are combined, so `D.Degree=A.Degree`. Next, `rhodiff` applies `helper.rateVerts` to the selected rate bounds. This helper enumerates the distinct rate vertices in lower-then-upper `combRows` order, with a fixed-bound direction contributing only one value. `rhodiff` then processes the returned vertices row by row. For each rate vertex $\boldsymbol{\nu}$, it multiplies the s-th partial by ν_s, adds the direction contributions in increasing s order, and stores the result as the corresponding coefficient row in each physical-cell cell. If every degree component is zero, each stored rate-vertex row is a complete zero row. The result has `IsContinuous=false` and derivative source metadata. Sections A.9 and A.9.1 derive the degree reduction, common-degree restoration, and row ordering.

Examples and output

```
A = pdbase({[0 2]}, [1 1], 1, {{1, 5}}, ...
    RateBounds=[-2 3]);
D = rhodiff(A);
derivativeClass = class(D)
derivativeDegree = D.Degree
derivativeRows = cell2mat(D.coeffs(1))
```

```
derivativeClass = 'pdbase'
derivativeDegree = 0
derivativeRows = 2×1 double
    -4
     6
```

Remarks

- Function-only storage raises `pdbase:FunctionOnlyDiff`. An object that already has explicit rate rows raises `pdbase:InvalidDiff`.
- Missing, malformed, or inconsistent bounds raise `pdbase:MissingRateBounds`, `pdbase:InvalidRateBounds`, or `pdbase:RateBoundsMismatch`, respectively. Repeated differentiation is rejected because the first derivative already contains rate rows.

See also. Class-specific `pdmat` and `pdvar` differentiation is documented in sections 4.6 and 5.6.

3.7 Inspect Matrix Shape

Syntax

```
sz = size(obj)
d = size(obj, dim)
[m, n] = size(obj)
n = height(obj)
n = width(obj)
n = length(obj)
n = ndims(obj)
n = numel(obj)
```

Arguments

- `obj` is a `pdbase` object or subclass instance.
- `dim` is a positive integer matrix dimension for `size`. Dimensions after row and column are singleton, and multiple outputs split the stored two-dimensional shape with trailing outputs equal to one.
- `size`, `height`, `width`, `length`, `ndims`, and `numel` report the matrix payload: respectively its two-element shape, row count, column count, largest dimension, the value two, and the product of the two dimensions.

Examples and output

The object below has a 2-by-3 matrix payload.

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
matrixSize = size(obj)
rowCount = size(obj, 1)
columnCount = size(obj, 2)
thirdDimension = size(obj, 3)
rowCountByName = height(obj)
columnCountByName = width(obj)
longestDimension = length(obj)
elementCount = numel(obj)
dimensionCount = ndims(obj)
```

```
matrixSize = 1×2 double
     2     3

rowCount = 2
columnCount = 3
thirdDimension = 1
rowCountByName = 2
columnCountByName = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
```

Remarks

These methods describe the stored matrix payload rather than the traversal counts in section 3.4.3.

3.8 Apply Shared Matrix Operations

The methods in this section are implemented once by `pdbase`. Each operation maps the same MATLAB matrix transformation over every local coefficient, every active rate row, and both terms of legacy rate-affine payloads while preserving the physical grid, degree, cell/label order, and metadata. A direct backend input returns a direct `pdbase`. The `pdmat` and `pdvar` reconstruction hooks retain their respective classes and class-specific semantics. Function-only `pdmat` data contain handle evaluation exclusively and therefore raise `pdmat:FunctionOnlyAlgebra` for coefficient mapping.

3.8.1 Unary Operations, Transpose, Vectorization, Diagonals, And Reshape

Syntax

```
B = +A
B = -A
T = transpose(A)
H = ctranspose(A)
T = A.'
H = A'
v = vec(A)
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
R = reshape(A, m, n)
R = reshape(A, [m n])
S = squeeze(A)
```

Description

Unary plus returns the same value. Unary minus, transpose, conjugate transpose, vectorization, diagonal extraction or construction, trace, reshape, and squeeze map coefficient payloads while preserving the parameter representation.

- `vec` uses MATLAB column-major order.
- `trace` returns a 1-by-1 object and sums the main diagonal even for a row or column vector.
- `diag` follows MATLAB's built-in behavior. The offset `k` is a finite integer and defaults to zero. For a nonvector matrix, including a nonsquare matrix, `diag(A, k)` extracts the diagonal at offset `k` and returns it as a column vector. For a row or column vector, `diag(v, k)` constructs a square matrix whose diagonal at offset `k` contains the elements of `v`.

- reshape supports exactly two positive dimensions, permits at most one inferred [] dimension, and preserves the element count.

- squeeze is the identity because the storage contract remains two-dimensional.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {{[1 2; 3 4], [10 20; 30 40]}});
T = A.';
v = vec(A);
d = diag(A);
t = trace(A);
R = reshape(A, 1, 4);
transposeCoefficients = T.coeffs(1);
vectorCoefficients = v.coeffs(1);
diagonalCoefficients = d.coeffs(1);
traceCoefficients = t.coeffs(1);
transposeSize = size(T)
transposeFirst = transposeCoefficients{1}
vectorFirst = vectorCoefficients{1}
diagonalFirst = diagonalCoefficients{1}
traceFirst = traceCoefficients{1}
reshapeSize = size(R)
```

```
transposeSize = 1×2 double
     2     2
transposeFirst = 2×2 double
     1     3
     2     4
vectorFirst = 4×1 double
     1
     3
     2
     4
diagonalFirst = 2×1 double
     1
     4
traceFirst = 5
reshapeSize = 1×2 double
     1     4
```

Remarks

Malformed or empty diagonal selections raise `class(A) + ":InvalidDiag"`. Unsupported reshape forms, multiple inferred dimensions, nonpositive dimensions, or an element-count mismatch raise `class(A) + ":InvalidReshape"`. These identifiers therefore appear as `pdbase:*`, `pdmat:*`, or `pdvar:*` according to the public input class.

3.8.2 Triangular Parts

Syntax

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
```

Description

`tril` and `triu` follow MATLAB's diagonal-offset convention. For an entry in row i and column j, its diagonal offset is $j - i$. Therefore, `k=0` selects the main diagonal, a positive `k` shifts the boundary upward to a superdiagonal, and a negative `k` shifts it downward to a subdiagonal. The default is `k=0`.

- `tril(A, k)` retains entries with $j - i \leq k$. Thus, `k=1` includes the first superdiagonal together with the main diagonal and all entries below it, while `k=-1` excludes the main diagonal and retains the first subdiagonal and all entries below it.
- `triu(A, k)` retains entries with $j - i \geq k$. Thus, `k=1` excludes the main diagonal and retains the first superdiagonal and all entries above it, while `k=-1` includes the first subdiagonal together with the main diagonal and all entries above it.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {{[1 2; 3 4], [10 20; 30 40]}});
L = tril(A);
lowerCoefficients = L.coeffs(1);
lowerFirst = lowerCoefficients{1}
```

```
lowerFirst = 2×2 double
     1     0
     3     4
```

Remarks

Invalid triangular offsets raise `class(A) + ":InvalidTriangularPart"`.

3.8.3 Reductions

Syntax

```
S = sum(A)
S = sum(A, dim)
S = sum(A, vecdim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, vecdim)
M = mean(A, "all")
C = cumsum(A)
C = cumsum(A, dim)
C = cumsum(A, direction)
C = cumsum(A, dim, direction)
```

Description

- `sum` and `mean` accept a positive integer scalar `dim`, a nonempty vector `vecdim` of unique positive integer dimensions, or the case-insensitive text `"all"`.
- A scalar `dim` reduces each coefficient matrix along that dimension. A `vecdim` reduces it along every listed dimension, so `sum(A, [1 2])` sums over both rows and columns. The option `"all"` reduces over both matrix dimensions and returns a scalar coefficient for each basis term.
- `cumsum` accepts an optional positive integer scalar dimension and an optional case-insensitive `direction`. The default direction is `"forward"`, which accumulates entries from the first index to the last index along the selected dimension. The option `"reverse"` accumulates from the last index to the first index.
- A `direction` supplied by itself uses the default dimension. When the dimension is omitted, these methods use the first nonsingleton matrix dimension, or dimension one when the coefficient matrix is scalar. A dimension above two cells every matrix payload unchanged.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {{[1 2; 3 4], [10 20; 30 40]}});
S = sum(A, [1 2]);
M = mean(A, "all");
C = cumsum(A, 2, "reverse");
sumCoefficients = S.coeffs(1);
meanCoefficients = M.coeffs(1);
cumsumCoefficients = C.coeffs(1);
sumFirst = sumCoefficients{1}
meanFirst = meanCoefficients{1}
reverseCumsumFirst = cumsumCoefficients{1}
```

```
sumFirst = 10
meanFirst = 2.5000
reverseCumsumFirst = 2×2 double
     3     2
     7     4
```

Remarks

Malformed reduction forms raise `class(A) + ":InvalidSum"`, `class(A) + ":InvalidMean"`, or `class(A) + ":InvalidCumsum"`. The supported reduction flags are exactly the dimension, `"all"`, and cumulative-direction forms listed above.

3.8.4 Reordering, Rotation, And Repetition

Syntax

```
F = flip(A)
F = flip(A, dim)
F = flipud(A)
F = fliplr(A)
R = rot90(A)
R = rot90(A, k)
Q = repmat(A, m)
Q = repmat(A, m, n)
Q = repmat(A, [m n])
```

Description

- `flip`, `flipud`, and `fliplr` reverse entries inside each coefficient matrix and never reverse the physical parameter grid. `flip(A)` uses the first nonsingleton dimension, while `flip(A, dim)` requires a positive integer scalar dimension.
- `rot90` applies a finite integer number of counterclockwise quarter-turns and reduces the count modulo four.
- `repmat` accepts a positive scalar interpreted as `[m m]`, two positive counts, or a two-element count vector. It changes only the payload shape and cells the physical parameter grid unchanged.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {{[1 2; 3 4], [10 20; 30 40]}});
F = flipud(A);
R = rot90(A);
Q = repmat(A, [1 2]);
flippedCoefficients = F.coeffs(1);
rotatedCoefficients = R.coeffs(1);
```

```
flippedFirst = flippedCoefficients{1}
rotatedFirst = rotatedCoefficients{1}
repeatedSize = size(Q)
```

```
flippedFirst = 2×2 double
     3     4
     1     2
rotatedFirst = 2×2 double
     2     4
     1     3
repeatedSize = 1×2 double
     2     4
```

Remarks

Invalid flip dimensions, rotation counts, or repetition forms raise `class(A) + ":InvalidFlip"`, `class(A) + ":InvalidRot90"`, or `class(A) + ":InvalidRepmat"`. N-dimensional repetition is outside the two-dimensional payload contract.

3.8.5 Concatenation Dispatch

Syntax

```
C = horzcat(A, B, ...)
C = [A, B, ...]
C = vertcat(A, B, ...)
C = [A; B; ...]
C = cat(dim, A, B, ...)
```

Description

- The inherited `horzcat` and `vertcat` entry points dispatch compatible derived objects to the class-specific `cat` implementation. Horizontal concatenation uses matrix dimension two, while vertical concatenation uses matrix dimension one.
- The `pdmat` and `pdvar` implementations perform coefficient-wise concatenation, grid and degree alignment, numeric promotion, validation, and result construction as documented in sections 4.10.7 and 5.9.7.
- A direct `pdbase` operand represents a scalar backend container and cannot be concatenated. The operation raises `pdbase:UnsupportedConcatenation`.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = [A, A];
resultClass = class(B)
resultSize = size(B)
```

```
resultClass = 'pdmat'
resultSize = 1×2 double
     1     2
```

Remarks

Any direct `pdbase` operand in `cat`, `horzcat`, or `vertcat` raises `pdbase:UnsupportedConcatenation`. Derived classes support matrix dimensions one and two and report their own compatibility errors. `blkdiag` belongs to the class-specific `pdmat` and `pdvar` implementations.

3.9 Transform Bernstein Storage

Whole-object elevation changes the coefficient representation while preserving the represented function and object metadata. Grid refinement and other protected storage transformations remain backend context, and their use preserves the meanings of `ncell`, `ncoeff`, `npar`, and `size` described above.

3.9.1 `elevate`

Syntax

```
out = obj.elevate(degreeIncrement)
out = obj.elevate(degreeIncrement, validationMode)
```

Description

`degreeIncrement` is one finite nonnegative integer scalar shorthand or an ℓ-element vector. A scalar expands uniformly, while a vector adds direction-wise. `out` is the same dynamic class and has degree `obj.Degree + degreeIncrement`. It preserves the represented polynomial, physical cells, metadata, existing YALMIP variables, rate-row order, subclass properties, and decision freedom. The optional positional `validationMode` controls the coefficient checks performed before the shared elevation plan is applied. It accepts the following case-insensitive scalar text options, and the selected mode is call-local and remains transient.

- `"fast"` is the default and checks one representative coefficient cell.
- `"strict"` checks every coefficient cell.

Section A.6 derives the sparse operator and packed application.

Examples and output

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
B = A.elevate(1);
sourceDegree = A.Degree
elevatedDegree = B.Degree
elevatedCoefficients = cell2mat(B.coeffs(1))
sameValue = [A.evaluate(0.25), B.evaluate(0.25)]
```

```
sourceDegree = 1
elevatedDegree = 2
elevatedCoefficients = 1×3 double
     1     2     3
sameValue = 1×2 double
    1.5000    1.5000
```

Remarks

Invalid increments raise `pdbase:InvalidDegreeIncrement`, and invalid validation modes raise `pdbase:InvalidValidationMode`. A strict check reports malformed cell structure through `pdbase:InvalidCoefficientCell` and malformed or inconsistent payloads through `pdbase:InvalidCoefficientPayload`. Function-only `pdmat` data contain handle evaluation exclusively and raise `pdbase:MissingCoefficientEvidence`. Constructing the function with a verified explicit `Degree` supplies the needed evidence. `elevate` returns a new object and preserves the source.

3.9.2 `bernTbl`, `mapVals`, and `matSubs`

Internal/protected mechanisms. These utilities belong to `pdbase` protected implementation context. The public `+helper` namespace and supported user call forms are listed separately in Chapter 7.

Syntax

```
T = bernTbl(obj, errId, valFcn, exprFcn, rateVerts, ...)
mapped = mapVals(vals, fcn, grid)
[rows, cols] = matSubs(subs, sz, errId)
```

Description

These protected utilities provide the shared backend operations for coefficient inspection, payload mapping, and matrix subscripting.

- `bernTbl` builds the detailed and one-line inspection tables used by `pdmat.bernTable` and `pdvar.bernTable`. It traverses `cells`, `lbls`, and `coeffs`, accepts at most one physical-cell selector and the case-insensitive `"oneLine"` option, and preserves the caller's error namespace.
- `mapVals` applies a coefficient mapping to every payload in every nested physical-cell cell while preserving the tree and label order. The owning class remains responsible for validating matrix, symbolic, and rate semantics.

- matSubs normalizes exactly two row and column selectors. It accepts :, finite positive integer vectors, or matching logical vectors, and rejects linear indexing, empty or out-of-range numeric indices, mismatched logical selectors, and array growth.

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
T = A.bernTable("oneLine");
column = A(:, 2);
tableHeight = height(T)
columnSize = size(column)
```

```
tableHeight = 1
columnSize = 1×2 double
     2     1
```

Remarks

Users reach bernTbl through bernTable, mapVals through inherited unary operations, and matSubs through pdmat/pdvar indexing and assignment. Invalid table selectors and rate-row mismatches raise the caller-owned errId, while indexing uses the class-specific identifier supplied by subsref or subsasgn. Shared helper utilities are documented in Chapter 7.

3.9.3 mergeGrid

Internal/protected mechanism. This signature documents the backend grid-refinement algorithm used by public algebra. User calls enter through the public algebra methods.

Syntax

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments

- errId is the error identifier used when parameter counts or physical bounds are incompatible.
- other1, other2, ... are coefficient-backed operands, including function-backed operands constructed with an explicit Degree. They must have equal parameter counts and matching first and last nodes on every parameter axis.
- grid has the sorted union of the interior nodes. Public algebra re-expresses operands on these cells before degree elevation, coefficient-wise addition, or product convolution.

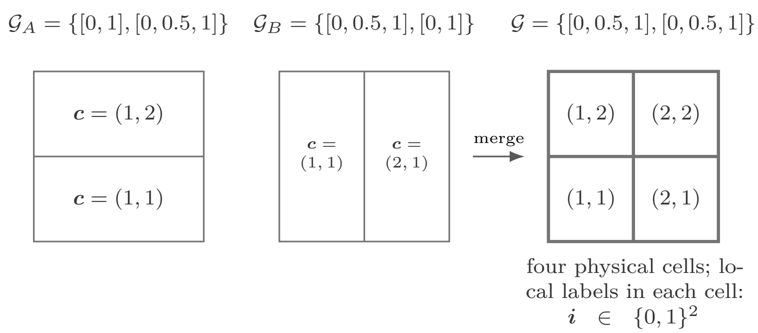
Public algebra methods invoke this protected helper through a common four-stage alignment: form the sorted union grid, use exact subdivision to re-express every operand on its physical cells, elevate direction-wise to the componentwise maximum degree, and then operate on matching local coefficients. Grid alignment preserves the represented functions and outer parameter box. Section A.6 gives the degree step, while section A.7 fixes the physical-cell and tensor-label ordering.

Examples and output

```
gridA = {[0 1], [0 0.5 1]};
gridB = {[0 0.5 1], [0 1]};
A = pdmat(gridA, @(rho1, rho2) rho1 + 2*rho2, Degree=[1 1]);
B = pdmat(gridB, @(rho1, rho2) 2*rho1 - rho2, Degree=[1 1]);
C = A + B;
rho1Grid = C.GridInfo.Vectors{1}
rho2Grid = C.GridInfo.Vectors{2}
```

```
rho1Grid = 1×3 double
        0    0.5000    1.0000

rho2Grid = 1×3 double
        0    0.5000    1.0000
```

The example invokes the protected helper through public addition, which is the supported user entry point. Here $\ell = 2$, the physical-cell index is $\boldsymbol{c} = (c_1, c_2)$ with $c_s \in \{1, 2\}$, and the degree-one local Bernstein label is $\boldsymbol{i} = (i_1, i_2) \in \{0, 1\}^2$ inside every cell.



Each axis uses the sorted union of its input nodes. The result is the unique minimal same-bound tensor grid containing both inputs.

3.10 Boundaries and Related APIs

`pdmat`, `pdvar`, `cells`, `coeffs`, `lbls`, `evaluate`, `ncell`, `ncoeff`, `npar`, `size`, `height`, `width`, `length`, `ndims`, `numel`, `end`, `numArgumentsFromSubscript`, unary `+`/`-`, `transpose`, `ctranspose`, `trace`, `vec`, `diag`, `reshape`, `squeeze`, `tril`, `triu`, `sum`, `mean`, `cumsum`, `flip`, `flipud`, `fliplr`, `rot90`, `repmat`, `cat`, `horzcat`, `vertcat`, `elevate`, `bernTbl`, `mapVals`, `matSubs`, and `mergeGrid`.

Chapter 4

pdmat: Known Parameter-Dependent Matrices

`pdmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle and reserve `bernTable` and coefficient algebra for objects with coefficient evidence. Shared matrix-shape, unary, structural, reduction, reordering, and repetition methods are implemented centrally by `pdbase` and remain complete public `pdmat` behavior. This chapter documents their known-data syntax, numeric examples, validations, and boundaries.

For coefficient-backed data, one storage cell is the pullback $A^{(\boldsymbol{c})} = A \circ \phi_{\boldsymbol{c}}$, with local expansion

$$A^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i} \in \prod_{s=1}^{\ell} \{0,\ldots,m_s\}} B_{\boldsymbol{i}}^{\boldsymbol{m}}(\boldsymbol{\alpha}) A^{(\boldsymbol{c})}[\boldsymbol{i}] \tag{4.1}$$

with $\boldsymbol{\alpha} = \phi_{\boldsymbol{c}}^{-1}(\boldsymbol{\rho})$. Construction chooses the source of those coefficients. `evaluate` reconstructs the matrix value. `elevate` changes the coefficient representation while preserving the polynomial. Matrix multiplication applies the binomial-weighted convolution, with details given in section A.8. A function-backed object keeps exact handle evaluation separate from its finite coefficient evidence, while a function-only object supports exact evaluation exclusively.

The chapter follows these three source families through one common question: which operations are justified by the stored evidence? A global or nested coefficient source supports coefficient inspection and algebra, a function with an explicit degree retains both exact evaluation and validated coefficient evidence, and a function constructed with the degree omitted supports exact evaluation only. The examples therefore inspect the source state before using elevation, plotting, algebra, or comparison.

4.1 API Map

Task	Entry
Construct data	`pdmat`
Public properties	`GridInfo`/`MatrixSize`/`Degree`/`LocalValues` and metadata
Inspect backend storage	`cells`/`lbls`/`coeffs` and counts
Evaluate	`evaluate`
Differentiate	`rhodiff`
Inspect/display	`bernTable`, `disp`, `display`
Plot	`plot`
Arithmetic	`+`, `-`, unary `+`/`-`, `*`
Compare/certify	logical `==` and known-data `<=`/`>=`
Matrix methods	shape and structural methods
Elevate Bernstein storage	`elevate`
Index/assign entries	indexing and assignment

4.2 Construct `pdmat` Objects

Syntax

```
A = pdmat(gridVector, source)
A = pdmat(gridVector, source, Degree=degree)
A = pdmat(gridVector, source, RateBounds=rb)
A = pdmat(gridVectors, source)
A = pdmat(gridVectors, source, Degree=degree, RateBounds=rb)
A = pdmat(..., ValidationMode=mode)
```

Arguments

- `gridVector` is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- `gridVectors` is a cell vector with one numeric grid vector per parameter, such as `{[0 1], [10 20]}`.
- `source` may be:
  1. A function handle, e.g., `@(rho) [rho rho^2]` is a one-parameter 2×1 matrix function
  2. A global cell grid of numeric Bernstein coefficients, e.g., `{[1 2], [3 4]}` is a two-cell degree-one grid with two 1×1 coefficients per cell.
  3. A explicit nested `LocalValues`, e.g., `{1, 2, 3}` is scalar degree-two data on one cell.
- `Degree=degree` accepts one finite nonnegative integer scalar shorthand or an ℓ-element vector and is stored as a 1-by-ℓ row $\boldsymbol{m} = (m_1, \dots, m_\ell)$. A scalar expands uniformly. An explicitly supplied scalar on a multidimensional grid emits `pdmat:ScalarDegreeExpansion` once. Omitted degree inference, function-only defaults, and one-dimensional forms remain warning-free. For

function handles, omitting `Degree` creates a function-only object. Supplying `Degree` validates and stores Bernstein coefficient evidence while retaining the exact function handle for `evaluate`.

- `RateBounds=rb` is an ℓ-by-2 finite real matrix of lower and upper scheduling-rate bounds and is accepted with every source family. With an ordinary one-row source it is metadata used by `rhodiff`. An explicit nested leaf may instead contain either one ordinary row or all $N_v =$ `size(helper.rateVerts(rb),1)` distinct-vertex rows, uniformly across every physical cell, in `helper.rateVerts(rb)` order. A fixed rate direction contributes one value, so $N_v \le 2^\ell$.
- `ValidationMode=mode` accepts scalar text `"fast"` or `"strict"`, case-insensitively, and defaults to `"fast"`. It controls only repeated post-normalization structural checks and remains transient call state. Public source validation remains complete in both modes.

Both `pdmat` and `pdvar` constructors share the same normalization expression as in (4.1), which inherited from `pdbase`. A `pdmat` validates known source data, while a `pdvar` creates and shares YALMIP control matrices. The basis and storage mathematics are given in section A.7.

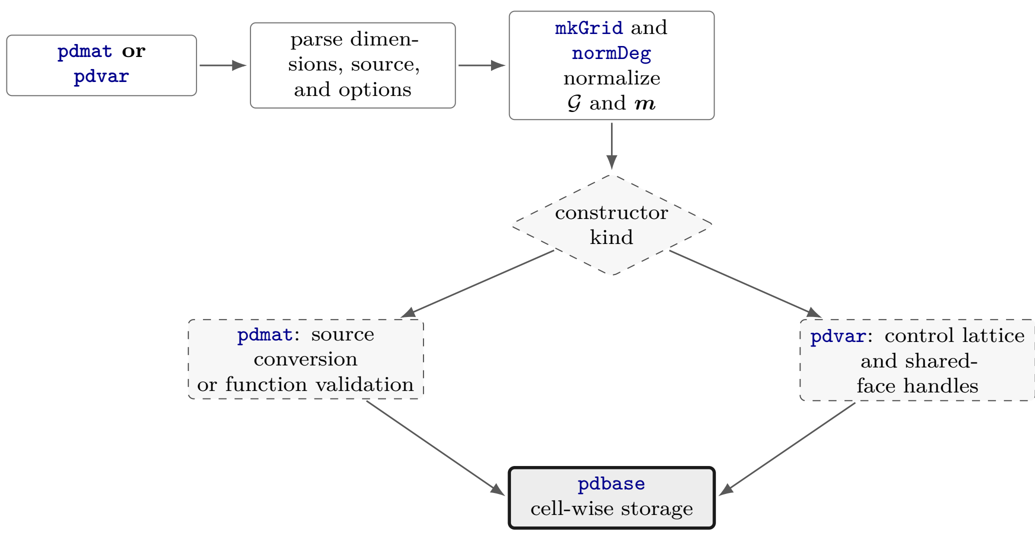


Figure 4.1: Constructor call path. The public parser chooses known-source conversion or decision-control construction after the common grid and vector-degree normalization.

Examples and output

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

```
A = pdmat([0 1 2], {10, 20, 30}, Degree=1);
disp(A)
matrixSize = size(A)
degree = A.Degree
sourceSummary = A.SourceSummary
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed, rate rows false

matrixSize = 1×2 double
```

```
     1     1

degree = 1
sourceSummary = "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. This independent example uses $\boldsymbol{m} = (2, 1)$, so the first parameter has quadratic Bernstein basis and the second has linear basis. The selected-cell `oneLine` table prints the represented expression directly.

```
B = pdmat({[0 1], [0 1]}, ...
    @(rho1, rho2) rho1.^2 + 2*rho2, Degree=[2 1]);
T = bernTable(B, [1 1], "oneLine");
disp(T)
```

```
    CellSubscript
    _____________

       {[1 1]}


   Expression
   ___________________________________________________________________________________________________________________

   "(1-α1)^2 * (1-α2)*0 + (1-α1)^2 * α2*2 + 2(1-α1)α1 * (1-α2)*0 + 2(1-α1)α1 * α2*2 + α1^2 * (1-α2)*1 + α1^2 * α2*3"
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

```
localVals = {{[1 0], [0 1]}, {[0 1], [0 2]}};
C = pdmat({[0 1 2]}, localVals, Degree=1);
disp(C)
matrixSize = size(C)
secondCellCoefficients = cell2mat(C.coeffs(2))
```

```
pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed, rate rows false

matrixSize = 1×2 double
     1     2

secondCellCoefficients = 1×4 double
     0     1     0     2
```

When `source` is a function handle and `Degree` is omitted, the object stores the exact evaluator exclusively.

```
F = pdmat({[0 1]}, @(rho) [rho rho^2]);
disp(F)
sourceSummary = F.SourceSummary
evaluatedValue = F.evaluate(0.5)
```

```
pdmat [1 2] over 1-D grid, degree 1, source function, rate rows false

sourceSummary = "function"
evaluatedValue = 1×2 double
    0.5000    0.2500
```

Add `Degree` for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

```
G = pdmat({[0 1]}, @(rho) rho.^2, Degree=2);
disp(G)
sourceSummary = G.SourceSummary
firstCellCoefficients = cell2mat(G.coeffs(1))
```

```
pdmat [1 1] over 1-D grid, degree 2, source function-bernstein, rate rows false

sourceSummary = "function-bernstein"
firstCellCoefficients = 1×3 double
     0     0     1
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

Remarks

`FunctionHandle` retains the supplied function handle for function-only and function-Bernstein sources. It is empty for cell-backed sources. Its set access is private.

- Constructor option errors use `pdmat:InvalidOptions`, `pdmat:UnsupportedOption`, or `pdmat:UnknownOption`. Grid errors use `pdmat:InvalidGrid` or `pdmat:InvalidGridVector`.
- Source and function validation uses `pdmat:InvalidSource`, `pdmat:InvalidFunctionHandle`, `pdmat:InvalidFunctionOutput`, or `pdmat:InvalidData`.
- Degree, rate, and storage failures use `pdmat:InvalidDegree`, `pdmat:InvalidRateBounds`, `pdmat:InvalidLocalValues`, `pdmat:InvalidCoefficientCell`, or `pdmat:InvalidCoefficientPayload`. Explicit multidimensional scalar degree expansion is accepted with warning `pdmat:ScalarDegreeExpansion`.
- Invalid validation modes raise `pdmat:InvalidValidationMode`. With explicit `Degree`, nonrepresentable functions raise `pdmat:NonBernsteinPolynomial`. Invalid validation probes raise `pdmat:InvalidFunctionValue`.
- Mismatched shared faces are retained as discontinuous local data and emit warning `pdmat:DiscontinuousLocalValues`.

4.3 Public Properties

Every pdmat property is public and read-only with private set access. Dot access exposes the known-data representation, while constructors and algebra return new values when stored information changes.

Property	Stored information
GridInfo	A scalar tensor-grid structure. Vectors is a 1-by-ℓ cell row of axis grids, Points is a $\prod_{s=1}^{\ell} k_s$-by-ℓ numeric matrix of physical grid nodes in combination order, Bounds is the ℓ-by-2 parameter box, and NumNodes is the 1-by-ℓ row $[k_1, \dots, k_\ell]$.
MatrixSize	The two-element row $[n_r, n_c]$ giving the size of every known matrix coefficient and every evaluated matrix.
Degree	The 1-by-ℓ row $[m_1, \dots, m_\ell]$ of local Bernstein degrees used in every physical cell.
LocalValues	The ℓ-level nested physical-cell tree with $k_s - 1$ cells along direction s. Each ordinary leaf is a 1-by-$\prod_{s=1}^{\ell}(m_s + 1)$ cell row of finite real numeric matrices in lbls(A) order. An explicit-rate leaf is NumRateRows-by-$\prod_{s=1}^{\ell}(m_s + 1)$ in stored rate-vertex order. Function-only objects contain placeholder storage that provides zero Bernstein coefficient evidence.
IsContinuous	A logical scalar recording whether adjacent coefficient-backed cells match on shared faces. Function-backed data with verified Bernstein evidence are continuous, while accepted mismatched nested local values set this flag false.
ContainsDecision	Always false for pdmat, because its stored coefficient payloads are known numeric data.
NumRateRows	A nonnegative integer scalar giving the number of explicit rate-vertex rows in each leaf. Zero denotes ordinary one-row storage even when RateBounds is present only as metadata.
RateBounds	Empty or an ℓ-by-2 matrix whose row s gives the lower and upper bounds for $\dot{\rho}_s$. Distinct rate vertices use the package-wide lower-then-upper combination order.
SourceSummary	Scalar origin text such as "coefficient-backed", "function", "function-bernstein", or "derivative".
FunctionHandle	The supplied function handle for function-only and function-Bernstein sources. It is empty for cell-backed sources and for numeric results whose construction clears exact function state.

Examples and output

```
A = pdmat({[0 1]}, @(rho) [rho.^2, 1+rho], ...
    Degree=2, RateBounds=[-1 1]);
gridBounds = A.GridInfo.Bounds
nodeCounts = A.GridInfo.NumNodes
matrixSize = A.MatrixSize
degree = A.Degree
firstLeafSize = size(A.LocalValues{1})
isContinuous = A.IsContinuous
containsDecision = A.ContainsDecision
rateRowCount = A.NumRateRows
rateBounds = A.RateBounds
sourceSummary = A.SourceSummary
hasFunction = ~isempty(A.FunctionHandle)
```

```
gridBounds = 1×2 double
     0     1
nodeCounts = 2
matrixSize = 1×2 double
     1     2
degree = 2
firstLeafSize = 1×2 double
     1     3
isContinuous =
  logical
   1
containsDecision =
  logical
   0
rateRowCount = 0
rateBounds = 1×2 double
    -1     1
sourceSummary = "function-bernstein"
hasFunction =
  logical
   1
```

Direct assignments such as `A.Degree = 1` or `A.FunctionHandle = []` raise MATLAB's private-set property error. Construct a new `pdmat`, or use a public operation that returns one, to change the represented value.

Related APIs. Use section 4.4 to traverse `LocalValues`, section 4.5 to reconstruct a matrix, and section 3.3 for the backend implementation contract.

4.4 Inspect Inherited Bernstein Storage

Syntax

```
cellSubs = cells(A)
cellSubs = A.cells()
labels = lbls(A)
labels = A.lbls()
```

```
coefficients = coeffs(A, cellSubs)
coefficients = A.coeffs(cellSubs)
n = ncell(A)
n = A.ncell()
n = ncoeff(A)
n = A.ncoeff()
n = npar(A)
n = A.npar()
```

Description

The inherited implementation follows the shared `pdbase` traversal order. `cells` returns an `ncell(A)`-by-ℓ numeric matrix with one physical-cell subscript per row and component $c_s \in \{1, \dots, k_s - 1\}$, where $k_s =$ `A.GridInfo.NumNodes`(s). `lbls` returns an `ncoeff(A)`-by-ℓ numeric matrix of tensor labels with component $i_s \in \{0, \dots, m_s\}$. Both use the package-wide combination order. `coeffs` returns the selected flat leaf in that label order: an ordinary known-data leaf is a 1-by-`ncoeff(A)` cell row of numeric matrices, while an explicit-rate leaf is `A.NumRateRows`-by-`ncoeff(A)` in stored rate-vertex order.

The counts are `ncell(A)`$= \prod_{s=1}^{\ell}($`A.GridInfo.NumNodes`$(s) - 1)$, `ncoeff(A)`$= \prod_{s=1}^{\ell}($`A.Degree`$(s) + 1)$, and `npar(A)`$= \ell =$ `numel(A.GridInfo.Vectors)`. Function-only data still expose their grid, labels, counts, and placeholder leaf. Coefficient-dependent operations such as `bernTable`, elevation, and algebra require actual coefficient evidence.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
physicalCells = A.cells()
labels = A.lbls()
sourceCoefficients = cell2mat(A.coeffs(1))
counts = [A.ncell(), A.ncoeff(), A.npar()]
```

```
physicalCells = 1
labels = 2×1 double
     0
     1
sourceCoefficients = 1×2 double
     1     2
counts = 1×3 double
     1     2     1
```

Remarks

Invalid physical-cell selectors raise `pdbase:InvalidCellSubs`.

Related APIs. Use section 4.3 for the stored tree and metadata, section 4.7.1 for formatted coefficient inspection, and sections 3.4 and 3.4.3 for the backend implementation details.

4.5 `evaluate`

Syntax

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments

- `A` is a `pdmat` object.
- `point` is a finite real scalar for a one-parameter object, such as `0.25`, or a finite real row vector with one entry per parameter, such as `[0.25 12]`. Every entry must lie within the corresponding grid bounds.
- `val` is a numeric matrix with `size(A)`. Coefficient-backed objects use local Bernstein interpolation. Function-backed objects call their stored function handle.

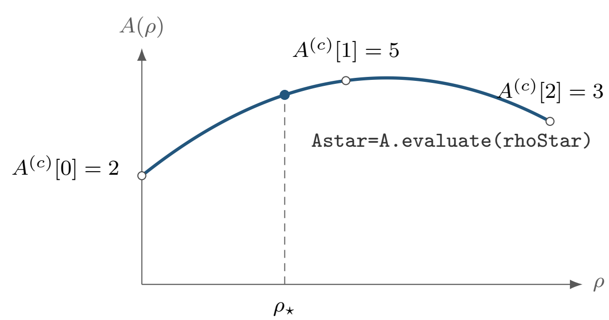
Remarks

A point with the wrong type, length, or finite-real shape raises `pdmat:InvalidPoint`. A point outside any grid bound raises `pdmat:PointOutOfBounds`. If a stored function fails or returns a nonfinite, complex, empty, or size-inconsistent matrix at the requested point, `evaluate` raises `pdmat:InvalidFunctionValue`.

For a degree-one scalar coefficient-backed object on $[\rho_1^{(1)}, \rho_1^{(2)}]$,

$$A(\rho) = (1-\alpha)A^{(c)}[0] + \alpha A^{(c)}[1], \qquad \alpha = \frac{\rho_1 - \rho_1^{(1)}}{\rho_1^{(2)} - \rho_1^{(1)}}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.



Evaluation follows the degree-two Bernstein curve with coefficients $[2, 5, 3]$. *The dashed line locates the requested physical point.*

Examples and output

Use the function-call form when the object is naturally one input among several MATLAB expressions.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

```
val = 2.5000
```

At $\rho = 0.25$, the public local coordinate is $\alpha = 0.25$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$.

The dot-call form is equivalent and can be convenient in command-window exploration.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

```
val = 3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

```
F = pdmat({[0 pi]}, @(rho) sin(rho));
val = F.evaluate(pi/2)
```

```
val = 1
```

4.6 Differentiate Known Data with `rhodiff`

Syntax

```
D = rhodiff(A)
D = rhodiff(A, rateBounds)
```

Arguments

`rhodiff` differentiates coefficient-backed `pdmat` data cell by cell and returns a `pdmat` with numeric rate-vertex rows. Explicit `rateBounds` supplies missing metadata, and when `A.RateBounds` is already stored, the explicit bounds must match it exactly. Empty bounds from both sources raise `pdmat:MissingRateBounds`. Along axis s, the local derivative coefficients are $\frac{m_s}{h^{\boldsymbol{c}}_s}(A^{(\boldsymbol{c})}[\boldsymbol{i}+\boldsymbol{e}_s] - A^{(\boldsymbol{c})}[\boldsymbol{i}])$. For one parameter, the input degree is still the vector $\boldsymbol{m} = (m_1)$. If $m_1 > 0$, the result degree is $(m_1 - 1)$, while $\boldsymbol{m} = (0)$ remains (0). For more than one parameter, each nonzero-axis partial is elevated exactly from $\boldsymbol{m} - \boldsymbol{e}_s$ back to the common tensor degree $\boldsymbol{m} =$ `A.Degree` before summation. A zero-degree axis contributes zero, and an all-zero tensor degree produces complete zero rate rows. The result is rate-dependent and generally discontinuous across cell faces. Sections A.9 and A.9.1 derive the reduction, re-elevation, and deterministic rate-row order. A function-only source raises `pdmat:FunctionOnlyDiff`. Malformed or inconsistent bounds raise `pdmat:InvalidRateBounds` or `pdmat:RateBoundsMismatch`, and data that already contain explicit rate rows, including a previous derivative, raise `pdmat:InvalidDiff`.

The implementation constructs the distinct rate vertices first. It then forms each cell partial, re-elevates multivariate partials exactly to the common degree, combines them with each rate vertex, and returns the owning value class through its `mkRhodiff` method. The derivative formula and rate-row layout are given in sections A.9 and A.9.1.

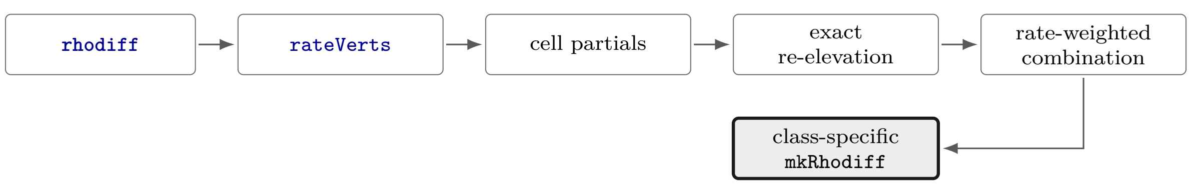


Figure 4.2: Differentiation call path. Cell partials are combined only after their tensor degrees and the rate-vertex order are fixed.

Examples and output

```
A = pdmat([0 1], {0, 2}, Degree=1, RateBounds=[-1 2]);
D = rhodiff(A);
T = bernTable(D, 1, "oneLine");
disp(T)
```

```
    CellSubscript    RateVertexIndex    RateVertex    Expression
    -------------    ---------------    ----------    ----------

        {[1]}               1              {[-1]}       "1*-2"
        {[1]}               2              {[ 2]}       "1*4"
```

In several parameters, each partial is restored to the common tensor degree before the rate-weighted sum. Here the four rate rows are constant over all four local labels, but the stored degree remains $[1, 1]$.

```
A = pdmat({[0 1], [0 1]}, ...
    @(rho1, rho2) rho1 + 2*rho2, ...
    Degree=[1 1], RateBounds=[−1 1; −2 3]);
D = rhodiff(A);
T = bernTable(D, [1 1], "oneLine");
disp(T)
```

```
    CellSubscript    RateVertexIndex    RateVertex                                                 Expression
    _____________    _______________    __________
      _______________________________________________________________________________________________________________________

     {[1 1]}               1            {[-1 -2]}     "(1-α1)^2 * (1-α2)*-4 + (1-α1)^2 * α2*-4 + 2(1-α1)α1 * (1-α2)*-5 + 2(1-α1)α1 * α2*-5 + α1^2 * (1-α2)*-6 + α1^2 * α2*-6"
     {[1 1]}               2            {[ -1 3]}     "(1-α1)^2 * (1-α2)*6 + (1-α1)^2 * α2*6 + 2(1-α1)α1 * (1-α2)*5 + 2(1-α1)α1 * α2*5 + α1^2 * (1-α2)*4 + α1^2 * α2*4"
     {[1 1]}               3            {[ 1 -2]}     "(1-α1)^2 * (1-α2)*-4 + (1-α1)^2 * α2*-4 + 2(1-α1)α1 * (1-α2)*-3 + 2(1-α1)α1 * α2*-3 + α1^2 * (1-α2)*-2 + α1^2 * α2*-2"
     {[1 1]}               4            {[  1 3]}     "(1-α1)^2 * (1-α2)*6 + (1-α1)^2 * α2*6 + 2(1-α1)α1 * (1-α2)*7 + 2(1-α1)α1 * α2*7 + α1^2 * (1-α2)*8 + α1^2 * α2*8"
```

4.7 Inspect Coefficients and Plot Values

4.7.1 `bernTable`

Syntax

```
T = bernTable(A)
T = bernTable(A, cellSubs)
T = bernTable(A, "oneLine")
T = bernTable(A, cellSubs, "oneLine")
```

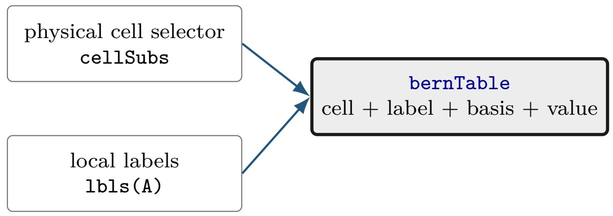
Arguments

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell and rate vertex with a compact Bernstein expression.
- The detailed default form uses seven metadata columns. An explicit-rate-row object additionally includes `RateVertexIndex` and `RateVertex`. Their deterministic distinct-vertex order is `helper.rateVerts(A.RateBounds)` order.

For one parameter, write the stored degree vector as $\boldsymbol{m} = (m_1)$. The `Basis` column reports

$$\binom{m_1}{j}(1-\alpha)^{m_1-j}\alpha^j.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate, using `alpha1`, `alpha2`, and so on. `LocalIndex` is the local Bernstein label. `CoeffSubscript` maps the local label back to the global coefficient subscript on the physical grid. `IsPhysicalNode` is true when that coefficient lies on a physical grid node.



The table reports stored coefficient evidence exclusively. Function sampling and solver-constraint creation belong to `evaluate` and `pdlmi`.

Examples and output

```
A = pdmat({[0 0.2 1]}, {[0 1], [1 1], [1 2]}, Degree=1);
T = bernTable(A, "oneLine");
disp(T)
T1 = bernTable(A, 1, "oneLine");
disp(T1)
```

```
    CellSubscript              Expression
    -------------    -------------------------------

        {[1]}        "(1-alpha)*[0 1] + alpha*[1 1]"
        {[2]}        "(1-alpha)*[1 1] + alpha*[1 2]"

    CellSubscript              Expression
    -------------    -------------------------------

        {[1]}        "(1-alpha)*[0 1] + alpha*[1 1]"
```

Use the full table when the metadata columns are the point of the example:

```
A = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
T = bernTable(A);
disp(T)
```

```
    TermIndex    CellSubscript    CoeffSubscript    LocalIndex          Basis           IsPhysicalNode    Value
    ---------    -------------    --------------    ----------    -----------------    --------------    -----

        1            {[1]}            {[1]}            {[0]}      "(1-alpha)^2"             true         {[1]}
        2            {[1]}            {[2]}            {[1]}      "2(1-alpha)alpha"         false        {[2]}
        3            {[1]}            {[3]}            {[2]}      "alpha^2"                 true         {[3]}
```

Remarks

A function-only object stores exact handle evaluation exclusively and raises `pdmat:FunctionOnlyBernsteinTable`. Unknown text options, repeated selectors, or invalid physical-cell selectors raise `pdmat:InvalidBernsteinTableInput`.

For an explicit-rate-row object, `bernTable(A,"oneLine")` preserves the alternatives by returning one expression for every stored vertex:

```
A = pdmat([0 1], {{1, 3; 10, 14}}, ...
```

```
    Degree=1, RateBounds=[−1 2]);
T = bernTable(A, "oneLine");
disp("vertexIndices =")
disp(T.RateVertexIndex)
disp("rateVertices =")
disp(vertcat(T.RateVertex{:}))
disp("expressionCount =")
disp(height(T))
```

```
vertexIndices =
     1
     2

rateVertices =
    -1
     2

expressionCount =
     2
```

4.7.2 disp And display

Syntax

```
disp(A)
display(A)
A
```

Arguments

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used for an expression with assignment echo enabled. It prints grid, degree, coefficient, and source metadata.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed, rate rows false
A =
  pdmat with 1-by-1 matrix values
  Parameters: 1
    rho_1: [0, 1], 2 nodes
  Degree: 1
  Physical cells: 1
  Coefficients per cell: 2
  Continuous: true
  Rate metadata: false
  Explicit rate rows: false
  Source: coefficient-backed
  Function handle: false
```

4.7.3 plot

Syntax

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., RateVertex=k)
h = plot(A, ..., Name, Value)
```

Arguments

- `dims` is one or two parameter indices. For one-parameter objects, the default is `1`. Otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is `15`. Use `SamplesPerCell=4` for a coarse diagnostic plot.
- `RateVertex=k` selects one one-based stored rate row in `helper.rateVerts(A.RateBounds)` order. Rate-row objects default to row one. Supplying this option for ordinary data or selecting beyond `A.NumRateRows` raises `pdmat:InvalidRateVertex`.
- Remaining name-value pairs pass through to MATLAB graphics. One-dimensional plots pass them to `plot`. Two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Remarks

An invalid parameter index, a repeated dimension, or a request for more than two plotting dimensions raises `pdmat:InvalidPlotDimensions`. Malformed plotting options or a nonpositive integer `SamplesPerCell` raise `pdmat:InvalidPlotOptions`. Errors from MATLAB graphics options remain MATLAB graphics errors. Every unselected parameter is fixed at its lower grid bound, `A.GridInfo.Bounds(:,1)`. A three-parameter object therefore produces a one- or two-dimensional slice plot.

For explicit rate rows, select the vertex as part of the plotting call:

```
A = pdmat([0 1], {0, 2}, Degree=1, RateBounds=[-1 2]);
D = rhodiff(A);
plot(D, RateVertex=2, SamplesPerCell=20, LineWidth=2);
```

Examples and output

Each source listing below is the complete source used by `generate_plot_figures.m` for the adjacent result.

One-dimensional entries.

```
A = pdmat({[-1 1]}, @(rho) [rho, rho.^2]);
h = plot(A, SamplesPerCell=80, LineWidth=2);
set(h(1), "Color", [35 86 125]/255, "LineStyle", "-")
set(h(2), "Color", [0.25 0.25 0.25], "LineStyle", "--")
grid on
xlabel("rho")
ylabel("matrix entry")
title("One-dimensional pdmat entries")
legend(["A(1,1)", "A(1,2)"], "Location", "eastoutside")
```

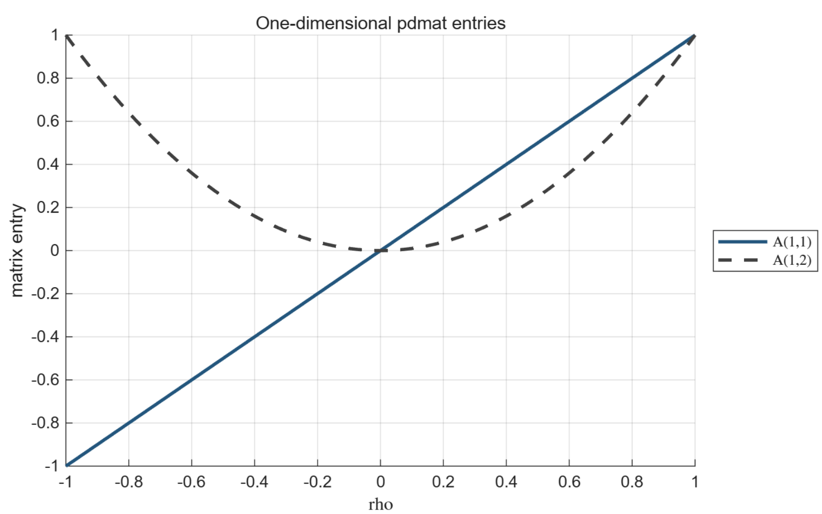


Figure 4.3: Result of the one-dimensional source above. The method returns one line handle per matrix entry.

Two-dimensional surface.

```
A = pdmat({[-1 1], [0 2]}, ...
    @(rho1, rho2) 1 + rho1.^2 + 0.5*rho2 + rho1.*rho2);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.78);
set(h, "FaceColor", [35 86 125]/255)
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two-dimensional pdmat surface")
```

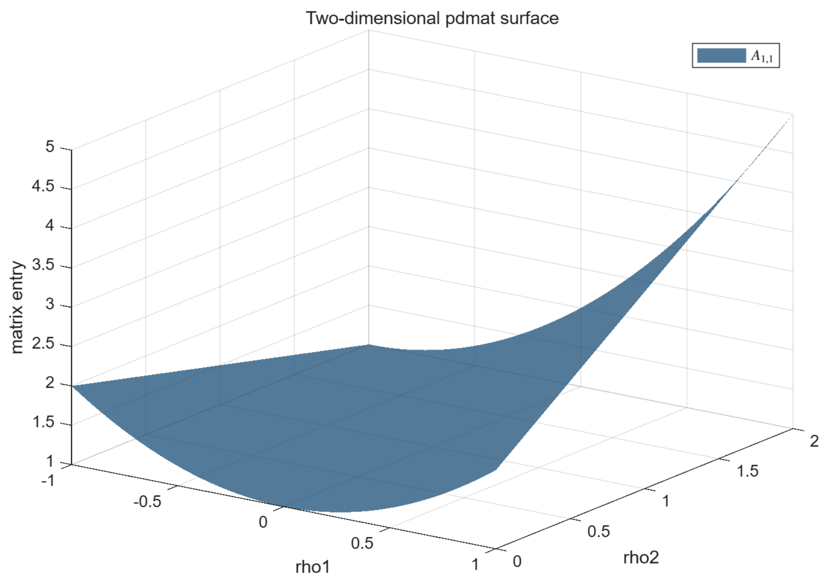


Figure 4.4: Result of the two-dimensional source above.

Two-dimensional matrix-valued surface.

```
A = pdmat({[-1 1], [0 2]}, @(rho1, rho2) [ ...
    1 + 0.8*rho1 + 0.25*rho2; ...
    1 - 0.8*rho1 + 0.25*rho2]);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.62);
set(h(1), "FaceColor", [35 86 125]/255)
set(h(2), "FaceColor", [0.55 0.55 0.55])
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two entries of a 2-by-1 pdmat cross at rho1 = 0")
legend(h, ["A(1,1)", "A(2,1)"], ...
    "Location", "eastoutside")
```

The two entries are equal for every ρ_2 along $\rho_1 = 0$. Their ordering reverses on opposite sides: the first entry is lower when $\rho_1 < 0$ and higher when $\rho_1 > 0$. The crossing therefore comes from the matrix function itself.

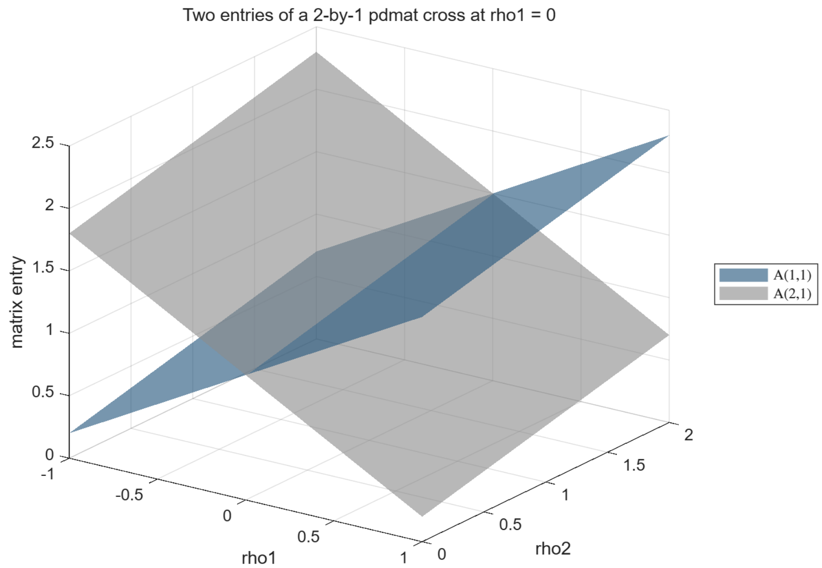


Figure 4.5: Two entry surfaces of a 2-by-1 matrix-valued function. The outside legend distinguishes the entries while their intersection remains visible along $\rho_1 = 0$.

4.8 Combine Known Matrices

A `pdmat` operation acts on complete matrix coefficients while the physical grid remains attached to the result. The blocks below deliberately show one operation family at a time: purpose, supported syntax, input, immediate output, and—when the result is spatial—a before/after picture.

Addition, subtraction, concatenation, block assembly, indexing assignment, and related binary operations use the same alignment path. The path exactly refines both operands to the union grid and elevates them to the componentwise maximum degree before it assembles the requested coefficient rows. The grid and elevation mathematics are given in sections A.6 and 3.9.3.

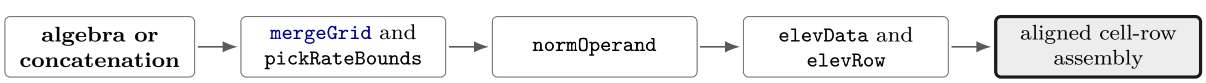


Figure 4.6: Shared alignment call path. Every parameter direction is refined and elevated independently before the public operation combines coefficients.

4.8.1 Signs, Addition, And Subtraction

Use signs, sums, and differences to build known matrix expressions. For two parameter-dependent operands, the backend first checks matching outer bounds, forms the sorted union of the interior nodes on every axis, and exactly subdivides/re-expresses both coefficient trees on that common grid. It then elevates both operands componentwise to $\max(\boldsymbol{m}_A, \boldsymbol{m}_B)$ and combines matching coefficients. Numeric and affine constant operands are promoted on the active grid before the same coefficientwise operation. The corresponding MATLAB overload names are `plus`, `minus`, `uplus`, and uminus. sections A.6 and 3.9.3 document the exact grid and degree mechanisms.

Syntax

```
S = A + B
S = A + M
S = M + A
D = A - B
D = A - M
D = M - A
P = +A
N = -A
```

Here M is a finite real scalar or size-compatible matrix. A scalar broadcasts to the matrix payload size. The first example makes the degree alignment visible.

Examples and output

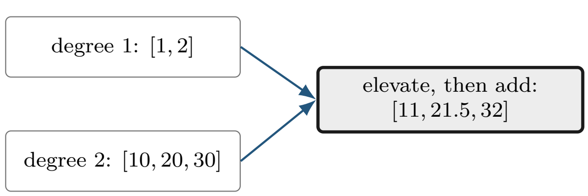
```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B;
T = bernTable(S, 1, "oneLine");
disp(T)
```

```
    CellSubscript                  Expression
    -------------    --------------------------------------

        {[1]}        "(1-α)^2*11 + 2(1-α)α*21.5000 + α^2*32"
```

The middle coefficient of the elevated A is $(1+2)/2 = 1.5$, so the middle sum is $1.5 + 20 = 21.5$. Subtraction and unary signs act immediately on the same local coefficients.

```
D = 5 - A;
N = -A;
differenceCoefficients = cell2mat(D.coeffs(1))
negativeAtOne = N.evaluate(1)
positiveClass = class(+A)
```

```
differenceCoefficients = 1×2 double
     4     3

negativeAtOne = -2
positiveClass = 'pdmat'
```



When grids have the same outer bounds but different interior nodes, the union grid is used before addition or subtraction. The next example combines this refinement with unequal degrees. The two input objects remain unchanged.

```
A = pdmat({[0 1]}, @(rho) rho, Degree=1);
B = pdmat({[0 0.5 1]}, @(rho) 2*rho, Degree=2);
S = A + B;
mergedGrid = S.GridInfo.Vectors{1}
T1 = bernTable(S, 1, "oneLine");
T2 = bernTable(S, 2, "oneLine");
disp(T1)
disp(T2)
```

```
mergedGrid = 1×3 double
        0    0.5000    1.0000

    CellSubscript                Expression
    -------------    --------------------------------------

        {[1]}        "(1-α)^2*0 + 2(1-α)α*0.7500 + α^2*1.5000"

    CellSubscript                  Expression
    -------------    ------------------------------------------

        {[2]}        "(1-α)^2*1.5000 + 2(1-α)α*2.2500 + α^2*3"
```

4.8.2 Ordered Matrix Multiplication

Use mtimes for a matrix product. The inner payload dimensions must match. Two parameter-dependent factors produce the componentwise degree sum. A numeric factor is constant and contributes degree zero in every component.

Syntax

```
K = A * B
K = A * M
K = M * A
```

The result preserves the common physical grid and has componentwise degree `A.Degree + B.Degree`. Payload matrices multiply in the written order. Section A.8 derives the numeric, known–affine, and generic planned kernels.

The multiplication backend first uses the shared alignment path in Fig 4.6. It then enumerates the ordered coefficient pairs whose labels sum to each output label and applies the matching numeric, known-affine, or generic kernel from section A.8.

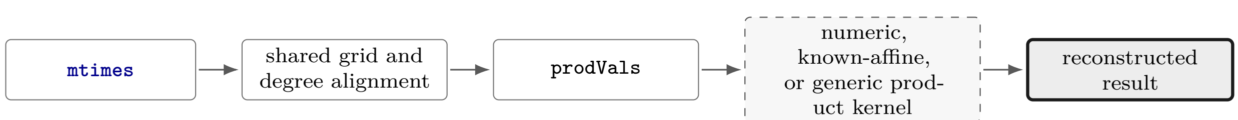


Figure 4.7: Multiplication call path. The kernel computes the ordered binomial-weighted convolution while preserving matrix multiplication order.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
K = A * B;
T = bernTable(K, 1, "oneLine");
disp(T)
```

```
    CellSubscript                          Expression
    -------------    ---------------------------------------------------------

       {[1]}         "(1-α)^3*10 + 3(1-α)^2α*20 + 3(1-α)α^2*36.6667 + α^3*60"
```

For tensor data, degrees add independently in each parameter direction. Two degree-one factors in two parameters therefore produce degree $[2, 2]$ and nine coefficients per physical cell.

Remarks

Incompatible operands for `+`, `-`, and `*` raise `pdmat:InvalidAddition`, `pdmat:InvalidSubtraction`, or `pdmat:InvalidMultiplication`. Parameter counts or outer bounds that reject a common grid raise `pdmat:MixedGrid`. Coefficient algebra on a function-only object raises `pdmat:FunctionOnlyAlgebra`. Construct the function with an explicit verified `Degree` when coefficient evidence is required.

4.9 Compare and Certify Known Data

Syntax

```
tf = A == B
C = A <= B
C = A >= B
C = A <= M
C = A >= M
```

Arguments

For two `pdmat` operands, `A==B` is a scalar logical coefficient-evidence comparison. Subtraction performs the compatibility checks, grid alignment, and degree elevation. The result is true exactly when the aligned coefficient difference is provably zero. This equality returns a logical scalar, while `<=` and `>=` create a `pdlmi` from a compatible coefficient-backed known residual. A function-only object supplies exact handle evaluation exclusively. Numeric `M` follows the ordinary finite-real size and scalar-broadcast rules.

Examples and output

```
A = pdmat([0 1], {1, 3}, Degree=1);
B = pdmat([0 1], {1, 2, 3}, Degree=2);
same = A == B
different = A == 2*A

yalmip('clear')
K = pdmat([0 1], {{1, 2; 3, 4}}, ...
    Degree=1, RateBounds=[-1 2]);
direct = K >= 0;
directCertificate = toYalmip(direct)
polya = direct.usePolya(1);
polyaCertificate = toYalmip(polya)
```

```
same = logical
   1

different = logical
   0

directCertificate = logical
   1

polyaCertificate = logical
   1
```

Remarks

Equality with an operand outside pdmat raises pdmat:InvalidEquality. Between two pdmat objects, subtraction owns compatibility checks and representation alignment, so incompatible operands propagate the corresponding subtraction, grid, continuity, or rate-bound error. A known pdmat equality returns a scalar logical result. Direct and Pólya known-data exports reduce the complete family to one scalar logical certificate using the wrapper's global inequality classification and inclusive absolute tolerance 10^{-10}. In semidefinite mode, every coefficient matrix is symmetrized as $(C + C^\top)/2$ and its eigenvalues are tested against the oriented tolerance, while element-wise mode tests every scalar matrix entry. A false result emits pdlmi:InconclusiveCertificate when exported and leaves continuous violation or indefiniteness inconclusive. Putinar, SparsePutinar, SparseFullBox, and FullBox still return YALMIP constraints and may introduce auxiliary Gram variables even when the residual is known.

4.10 Reshape and Transform Known Matrices

These methods transform every local matrix coefficient in the same way. They are inherited from the centralized pdbase implementation, while the pdmat reconstruction hook preserves the pdmat class, known-data payloads, metadata, and physical grid. Except for the documented evaluator, display, and shape paths, structural methods require coefficient evidence.

4.10.1 Shape, Equality, And Display Protocols

Shape queries describe the matrix payload, while `GridInfo` describes the grid. `isequal` compares object representations after compatible grid refinement and degree elevation. Scalar logical operator equality is documented in section 4.9. The protocol methods support ordinary MATLAB indexing syntax and dot access through the existing storage API.

Syntax

```
sz = size(A)
d = size(A, dim)
n = height(A)
n = width(A)
n = length(A)
n = numel(A)
n = ndims(A)
tf = isequal(A, B, ...)
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
matrixSize = size(A)
rowCount = height(A)
columnCount = width(A)
longestDimension = length(A)
elementCount = numel(A)
dimensionCount = ndims(A)
equalsUnaryPlus = isequal(A,+A)
```

```
matrixSize = 1×2 double
     2     3

rowCount = 2
columnCount = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
equalsUnaryPlus =
  logical
   1
```

A function-only object can use shape queries and `isequal` compares its stored metadata and function handle. A coefficient-backed comparison that rejects a common grid raises `pdmat:InvalidEquality`. The detailed multi-line `display` transcript is shown in section 4.7.2.

4.10.2 Transpose And Conjugate Transpose

Both transpose forms act on every coefficient. Because `pdmat` payloads are finite and real, conjugation preserves their values.

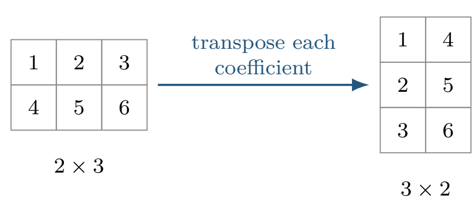
Syntax

```
T = transpose(A)
H = ctranspose(A)
T = A.’
H = A’
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
T = A.';
H = A';
transposeSize = size(T)
formsEqual = isequal(T,H)
transposeAtZero = T.evaluate(0)
```

```
transposeSize = 1×2 double
     3     2

formsEqual =
  logical
   1
transposeAtZero = 3×2 double
     1     4
     2     5
     3     6
```



4.10.3 Vectorization, Reshape, And Squeeze

Use these methods to change the matrix view while preserving column-major element order, the parameter grid, the degree, and every coefficient value. `reshape` accepts exactly two positive dimensions with at most one inferred `[]` dimension. `squeeze` retains the package’s two-dimensional matrix convention.

Syntax

```
V = vec(A)
R = reshape(A, [m n])
R = reshape(A, m, n)
S = squeeze(A)
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
V = vec(A);
R = reshape(A, [3 2]);
S = squeeze(A);
vectorSize = size(V)
reshapeSize = size(R)
squeezeSize = size(S)
vectorAtZero = V.evaluate(0).'
```

```
vectorSize = 1×2 double
     6     1
reshapeSize = 1×2 double
     3     2
squeezeSize = 1×2 double
     2     3
vectorAtZero = 1×6 double
     1     4     2     5     3     6
```

Malformed or element-count-changing reshape requests raise `pdmat:InvalidReshape`.

4.10.4 Diagonals And Trace

`diag` extracts a selected diagonal from a matrix, or builds a square diagonal matrix from a vector. A finite integer offset `k` must select a nonempty diagonal. `trace` returns a 1-by-1 `pdmat`.

Syntax

```
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
d = diag(A);
D = diag(d);
t = trace(A);
diagonalAtZero = d.evaluate(0)
rebuiltAtZero = D.evaluate(0)
traceAtZero = t.evaluate(0)
```

```
diagonalAtZero = 2×1 double
     1
     4
rebuiltAtZero = 2×2 double
     1     0
     0     4
traceAtZero = 5
```

Invalid offsets or empty selections raise `pdmat:InvalidDiag`.

4.10.5 Triangular Parts And Reductions

Triangular extraction preserves or zeros entries relative to a selected diagonal. Reductions act along matrix dimensions. `"all"` is supported by `sum` and `mean`.

Syntax

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
S = sum(A)
S = sum(A, dim)
S = sum(A, vecdim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, vecdim)
M = mean(A, "all")
C = cumsum(A)
C = cumsum(A, dim)
C = cumsum(A, direction)
C = cumsum(A, dim, direction)
```

First inspect the two triangular results.

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
L = tril(A);
U = triu(A);
lowerAtZero = L.evaluate(0)
upperAtZero = U.evaluate(0)
```

```
lowerAtZero = 2×2 double
     1     0
     3     4
upperAtZero = 2×2 double
     1     2
     0     4
```

Then apply reductions to the same coefficient matrix.

```
rowSum = sum(A, 2);
allMean = mean(A, "all");
running = cumsum(A, 2);
rowSumAtZero = rowSum.evaluate(0)
allMeanAtZero = allMean.evaluate(0)
runningAtZero = running.evaluate(0)
```

```
rowSumAtZero = 2×1 double
     3
     7
allMeanAtZero = 2.5000
runningAtZero = 2×2 double
     1     3
     3     7
```

`vecdim` is a nonempty vector of unique positive integer dimensions. Dimensions above two are identity operations. `direction` is `"forward"` or `"reverse"`, case-insensitively. A direction supplied by itself uses the first nonsingleton matrix dimension, and `cumsum` along a dimension above two returns the same coefficient payloads. Invalid triangular offsets raise `pdmat:InvalidTriangularPart`. Invalid reduction calls raise `pdmat:InvalidSum`, `pdmat:InvalidMean`, or `pdmat:InvalidCumsum`. Supported reduction flags are exactly the dimension, `"all"`, and cumulative-direction forms listed above.

4.10.6 Reordering And Rotation

These methods reorder rows or columns inside every coefficient while preserving the scheduling-grid order.

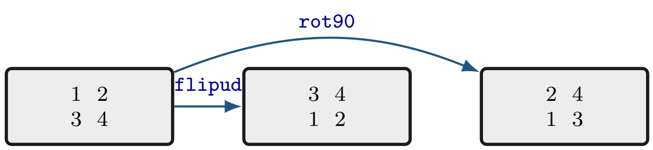
Syntax

```
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
F = flipud(A);
R = rot90(A);
flippedAtZero = F.evaluate(0)
rotatedAtZero = R.evaluate(0)
```

```
flippedAtZero = 2×2 double
     3     4
     1     2
rotatedAtZero = 2×2 double
     2     4
     1     3
```



Invalid dimensions or quarter-turn counts raise `pdmat:InvalidFlip` or `pdmat:InvalidRot90`.

4.10.7 Concatenation And Block Assembly

Concatenation joins payload matrices after compatible grid refinement and exact elevation to the componentwise maximum degree. Numeric blocks are promoted to constant known data. `cat` supports only dimensions 1 and 2.

Syntax

```
H = horzcat(A, B, ...)
V = vertcat(A, B, ...)
H = [A, B]
V = [A; B]
C = cat(dim, A, B, ...)
D = blkdiag(A, B, ...)
```

Examples and output

```
a = pdmat({[0 1]}, {1, 2}, Degree=1);
b = pdmat({[0 1]}, {10, 20}, Degree=1);
H = [a, b];
V = [a; b];
D = blkdiag(a, 5);
horizontalAtZero = H.evaluate(0)
verticalAtZero = V.evaluate(0)
blockDiagonalAtZero = D.evaluate(0)
```

```
horizontalAtZero = 1×2 double
     1    10
verticalAtZero = 2×1 double
     1
    10
blockDiagonalAtZero = 2×2 double
     1     0
     0     5
```

Incompatible blocks or syntax raise `pdmat:InvalidConcatenation`. Unsupported dimensions raise `pdmat:UnsupportedCatDimension`. Invalid block-diagonal inputs raise `pdmat:InvalidBlkdiag`. Function-only operands raise `pdmat:FunctionOnlyAlgebra`.

4.10.8 Repetition

`repmat` tiles the matrix payload. It accepts a positive scalar `m`, interpreted as `[m m]`, two positive counts, or a two-element count vector. Grid and Bernstein degree remain unchanged.

Syntax

```
R = repmat(A, m)
R = repmat(A, m, n)
R = repmat(A, [m n])
```

Examples and output

```
v = pdmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
R = repmat(v, [2 2]);
repeatedSize = size(R)
repeatedDegree = R.Degree
repeatedAtZero = R.evaluate(0)
```

```
repeatedSize = 1×2 double
     2     4
repeatedDegree = 1
repeatedAtZero = 2×4 double
     1     2     1     2
     1     2     1     2
```

Invalid repetition shapes raise `pdmat:InvalidRepmat`.

4.11 Elevate `pdmat` Bernstein Storage

Syntax

```
B = elevate(A, degreeIncrement)
B = A.elevate(degreeIncrement)
B = elevate(A, degreeIncrement, validationMode)
B = A.elevate(degreeIncrement, validationMode)
```

Description

`elevate` accepts a scalar shorthand or direction-wise increment and returns a new `pdmat` at the componentwise degree sum while preserving the represented known function, metadata, physical cells, and stored rate-row order. The optional positional validation mode and its errors follow the canonical contract in section 3.9.1. The sparse elevation map and its reuse are derived in section A.6.

For a source degree $\boldsymbol{m}$ and increment $\boldsymbol{d}$, `elevate` forms the target $\boldsymbol{M} = \boldsymbol{m} + \boldsymbol{d}$. The numeric ratios in `bernConvRatios` define a sparse exact change of Bernstein basis, so the returned coefficients preserve the represented known function exactly. See section A.6 for the operator formula.

Figure 4.8: Elevation call path. The row kernel reuses exact binomial ratios for every matching source-target degree pair.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = A.elevate(1);
T = bernTable(B, 1, "oneLine");
disp(T)
```

```
    CellSubscript                  Expression
    _____________    ___________________________________

        {[1]}        "(1-α)^2*1 + 2(1-α)α*1.5000 + α^2*2"
```

A direction-wise increment changes only the requested axes. This two-parameter example elevates the first axis while retaining the second.

```
A = pdmat({[0 1], [0 1]}, ...
    @(rho1, rho2) rho1 + 2*rho2, Degree=[1 1]);
B = A.elevate([1 0]);
T = bernTable(B, [1 1], "oneLine");
disp(T)
```

The exact output table is split at its column boundary so the expression can wrap while preserving its text.

```
    CellSubscript
    _____________

       {[1 1]}

   Expression
   ________________________________________________________________________________________________________________________

   "(1-α1)^2 * (1-α2)*0 + (1-α1)^2 * α2*2 + 2(1-α1)α1 * (1-α2)*0.5000 + 2(1-α1)α1 * α2*2.5000 + α1^2 * (1-α2)*1 + α1^2 * α2*3"
```

Remarks

Invalid increments raise `pdbase:InvalidDegreeIncrement`, and invalid validation modes raise `pdbase:InvalidValidationMode`. Function-only data support exact `evaluate` exclusively. Calls to `elevate`, algebra, or `bernTable` reject their placeholder storage as coefficient evidence, and the elevation path raises `pdbase:MissingCoefficientEvidence`. A function constructed with an explicit verified `Degree` retains both its exact handle and Bernstein evidence.

4.12 Index and Assign `pdmat` Entries

Parenthesis indexing selects a matrix block from every coefficient. Assignment writes the selected block into every coefficient after the same common-grid refinement and componentwise maximum-degree elevation used by addition: the left object and parameter-dependent right-hand side are exactly re-expressed on the sorted union grid, both coefficient families are elevated, and then only the selected payload block is replaced at every aligned label. Physical-cell inspection uses `coeffs`. See sections A.6 and 3.9.3.

Syntax

```
B = A(rows, cols)
B = A(1:end, end)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end);
A(:, 2) = 5;
selectedSize = size(lastCol)
assignedSize = size(A)
selectedAtZero = lastCol.evaluate(0)
assignedAtZero = A.evaluate(0)
```

```
selectedSize = 1×2 double
     2     1
assignedSize = 1×2 double
     2     3
selectedAtZero = 2×1 double
     3
     6
assignedAtZero = 2×3 double
     1     5     3
     4     5     6
```

Assignment also aligns a parameter-dependent right-hand side with a different grid and degree.

```
A = pdmat({[0 1]}, @(rho) [1 rho; 2 3*rho], Degree=1);
B = pdmat({[0 0.5 1]}, @(rho) [rho.^2; 1+rho], Degree=2);
A(:, 2) = B;
assignedGrid = A.GridInfo.Vectors{1}
assignedDegree = A.Degree
assignedValue = A.evaluate(0.25)
```

```
assignedGrid = 1×3 double
         0    0.5000    1.0000

assignedDegree = 2

assignedValue = 2×2 double
    1.0000    0.0625
    2.0000    1.2500
```

`subsref`, `subsasgn`, `end`, and `numArgumentsFromSubscript` implement this two-index MATLAB protocol. Invalid selectors raise `pdmat:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdmat:UnsupportedAssignment`. An incompatible right-hand side raises `pdmat:InvalidAssignment`. Slicing and assignment require coefficient evidence and otherwise raise `pdmat:FunctionOnlyAlgebra`.

4.13 Boundaries and Related APIs

1. Function-only objects created with an omitted `Degree` store exact handle evaluation exclusively. `bernTable`, coefficient algebra, differentiation, and inequalities require Bernstein coefficient evidence.

2. Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.

3. Ordinary one-row operands broadcast across explicit rate rows. Two explicit-rate-row operands require exactly matching grids and nonempty rate bounds. Multiplication supports explicit rate rows on at most one side and rejects rate-row by rate-row products.

4. Algebra, concatenation, indexing, and assignment preserve compatible rate metadata and stored row order. Mismatched rate boxes, grids, row counts, or rate-dependent product forms raise the owning operation's class-specific error.

Related APIs. `pdmat`, `evaluate`, `rhodiff`, `bernTable`, `plot`, `pdlmi`, `pdbase`.

Chapter 5

pdvar: Decision Matrix Functions

`pdvar` creates continuous cell-wise Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. Solver selection and the call to `optimize` belong to the caller's optimization workflow. Comparison overloads create `pdlmi` objects, whose `toYalmip` method later converts them to YALMIP constraints. Shared matrix-shape, unary, structural, reduction, reordering, and repetition methods are implemented centrally by `pdbase` and remain complete public `pdvar` behavior. This chapter documents their decision-expression syntax, stable shape examples, validations, and rate-aware boundaries.

A `pdvar` with direction-wise degree $\boldsymbol{m} = (m_1, \ldots, m_\ell)$ represents a decision such as $P(\boldsymbol{\rho})$ or $y_k(\boldsymbol{\rho})$. When a component m_s is positive, neighboring cells reuse every decision handle on the corresponding shared face, which enforces value continuity through shared variables and zero auxiliary equality constraints. A zero component makes the decision constant along that direction, and $\boldsymbol{m} = \mathbf{0}$ reuses one decision matrix over the complete grid. `rhodiff` applies cell-wise forward differences scaled by $1/h_s^{\boldsymbol{c}}$ and stores one row per vertex in $\mathcal{V}_{\mathcal{R}}$, while `value` recovers numeric coefficient payloads only after the caller has validated YALMIP's solver status. Products remain affine only when the other factor is known numeric or `pdmat` data.

5.1 API Map

Task	Entry
Construct decisions	`pdvar`
Public properties	`GridInfo`/`MatrixSize`/`Degree`/`LocalValues` and metadata
Inspect backend storage	`cells`/`lbls`/`coeffs` and counts
Evaluate	`evaluate`
Differentiate rates	`rhodiff`
Inspect coefficient table	`bernTable`
Affine/mixed algebra	`+`, `-`, `*` with known data
Matrix methods	structural methods
Elevate Bernstein storage	`elevate`
Index/assign entries	indexing and assignment
Convert assigned coefficients	`value`
Build LMIs	`<=`, `>=`, `==`

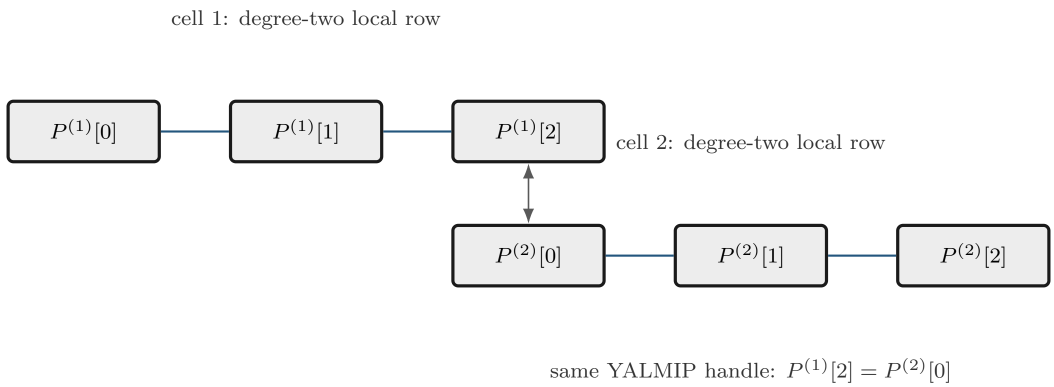
5.2 Construct `pdvar` Decisions

Syntax

```
P = pdvar(n, gridVectors)
P = pdvar(n, n, gridVectors)
P = pdvar(n, p, gridVectors)
P = pdvar(..., "full")
P = pdvar(..., "symmetric")
P = pdvar(..., Degree=0)
P = pdvar(..., Degree=1)
P = pdvar(..., Degree=degree)
P = pdvar(..., RateBounds=rb)
P = pdvar(..., ValidationMode=mode)
```

Arguments

- `pdvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand. For example, `pdvar(2, {[0 1]})`. Square variables default to symmetric YALMIP coefficients.
- `pdvar(n, n, gridVectors)` is the explicit square-size form. For example, `pdvar(2, 2, {[0 1]})`. It has the same default symmetric structure as the shorthand.
- `pdvar(n, p, gridVectors)` creates an $n \times p$ variable. Rectangular variables default to full YALMIP coefficients. For example, `pdvar(2, 3, {[0 1]})`.
- `"full"` and `"symmetric"` explicitly choose the YALMIP coefficient structure. `"symmetric"` requires a square size.
- `Degree=degree` accepts one finite nonnegative integer scalar shorthand or an ℓ-element vector and is stored as a 1-by-ℓ row $\boldsymbol{m} = (m_1, \ldots, m_\ell)$. The omitted default is **1**. An explicitly supplied scalar on a multidimensional grid expands uniformly and emits `pdvar:ScalarDegreeExpansion` once. Omitted defaults and one-dimensional forms remain warning-free. An all-zero vector creates one parameter-independent decision matrix shared across the grid. Otherwise neighboring cells share complete control faces. A zero component means constancy in that parameter direction.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter. Use a two-row matrix for two parameters, as shown in the rate-metadata example below.
- `ValidationMode=mode` accepts scalar text `"fast"` or `"strict"`, case-insensitively, and defaults to `"fast"`. It affects repeated validation of package-generated storage only. Public options, grid, structure, affine safety, continuity, and source semantics remain global in both modes. The mode is transient constructor input.



For positive degree, neighboring physical cells reuse complete boundary-face YALMIP handles. Shared handles enforce continuity with zero auxiliary continuity LMIs.

Examples and output

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object's `RateBounds` property is empty.

```
yalmip('clear')
Ps = pdvar(2, {[0 1]});
P= Ps.coeffs([1]);
symmetricCoefficient = ishermitian(P{1})
variableClass = class(Ps)
matrixSize = size(Ps)
degree = Ps.Degree
rateRowCount = Ps.NumRateRows
```

```
symmetricCoefficient = 1
variableClass = 'pdvar'
matrixSize = 1×2 double
     2     2

degree = 1

rateRowCount = 0
```

Use two dimension arguments for a rectangular full variable.

```
yalmip('clear')
Pf = pdvar(2, 3, {[0 1]});
variableClass = class(Pf)
matrixSize = size(Pf)
degree = Pf.Degree
```

```
variableClass = 'pdvar'
matrixSize = 1×2 double
     2     3

degree = 1
```

The structure flag makes the coefficient structure explicit. Use `"full"` when a square variable requires full coefficients in place of the symmetric default.

```
yalmip('clear')
Pfull = pdvar(2, 2, {[0 1]}, "full");
P = Pfull.coeffs([1]);
fullCoefficient = ishermitian(P{1})
variableClass = class(Pfull)
matrixSize = Pfull.MatrixSize
degree = Pfull.Degree
```

```
fullCoefficient = logical
   0

variableClass = 'pdvar'
matrixSize = 1×2 double
     2     2

degree = 1
```

Use `Degree=0` for a parameter-independent decision variable stored with grid metadata.

```
yalmip('clear')
P0 = pdvar(1, [0 1 2], Degree=0);
variableClass = class(P0)
degree = P0.Degree
cellCount = P0.ncell()
```

```
variableClass = 'pdvar'
degree = 0
cellCount = 2
```

Direction-wise degree permits variation in selected parameters and exact constancy in the others. The following clean-session transcript uses $\boldsymbol{m} = (2, 0)$, so the result has three independent coefficients along the first parameter and is constant along the second. The symbolic identifiers printed by YALMIP are session-local display labels.

```
yalmip('clear')
Pa = pdvar(1, {[0 1], [10 20]}, Degree=[2 0]);
T = bernTable(Pa, [1 1], "oneLine");
disp(T)
```

```
    CellSubscript                                     Expression
    -------------    ------------------------------------------------------------------------------------

       {[1 1]}       "(1-α1)^2 * 1*[internal(1)] + 2(1-α1)α1 * 1*[internal(2)] + α1^2 * 1*[internal(3)]"
```

Rate bounds are metadata for later `rhodiff(P)` calls. `pdlmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

```
yalmip('clear')
Pr = pdvar(1, {[0 1], [10 20]}, RateBounds=[−1 2; −3 4]);
variableClass = class(Pr)
rateRowCount = Pr.NumRateRows
rateBounds = Pr.RateBounds
```

```
variableClass = 'pdvar'
rateRowCount = 0

rateBounds = 2×2 double
     -1     2
     -3     4
```

Remarks

- Missing or misplaced dimensions and grids raise `pdvar:InvalidInput`. Invalid sizes and degrees raise `pdvar:InvalidMatrixSize` or `pdvar:InvalidDegree`. Explicit multidimensional scalar degree expansion is accepted with warning `pdvar:ScalarDegreeExpansion`.
- Option failures use `pdvar:InvalidOptions`, `pdvar:UnknownOption`, or `pdvar:UnsupportedOption`. Invalid modes raise `pdvar:InvalidValidationMode`.
- A symmetric nonsquare request raises `pdvar:InvalidStructure`. Grid failures use `pdvar:InvalidGrid` or `pdvar:InvalidGridVector`.
- `RateBounds` must be a finite ℓ-by-2 matrix with each lower bound at most its upper bound. Violations raise `pdbase:InvalidRateBounds`.

5.3 Public Properties

Every pdvar property is public and read-only with private set access. Dot access exposes the decision-expression representation, while constructors and public algebra return new values when stored information changes.

Property	Stored information
GridInfo	A scalar tensor-grid structure. `Vectors` is a 1-by-ℓ cell row of axis grids, `Points` is a $\prod_{s=1}^{\ell} k_s$-by-ℓ numeric matrix of physical grid nodes in combination order, `Bounds` is the ℓ-by-2 parameter box, and `NumNodes` is the 1-by-ℓ row $[k_1, \ldots, k_\ell]$.
MatrixSize	The two-element row $[n_r, n_c]$ giving the size of every symbolic coefficient payload and every evaluated matrix expression.
Degree	The 1-by-ℓ row $[m_1, \ldots, m_\ell]$ of local Bernstein degrees used in every physical cell. A zero component denotes a decision that is constant along that parameter direction.
LocalValues	The ℓ-level nested physical-cell tree with $k_s - 1$ cells along direction s, storing symbolic or affine matrix coefficients in `lbls(P)` order. Each ordinary leaf is a 1-by-$\prod_{s=1}^{\ell}(m_s+1)$ cell row. A derivative or explicit-rate leaf is NumRateRows-by-$\prod_{s=1}^{\ell}(m_s+1)$, with rows in the stored distinct-rate-vertex order.
IsContinuous	A logical scalar owned by the operation that constructs the expression. Constructor decisions are continuous because adjacent cells reuse shared YALMIP handles, while rhodiff returns a cell-wise rate expression with this flag false.
ContainsDecision	A logical scalar that is true when at least one coefficient payload contains a YALMIP decision variable. Affine algebra records the actual decision presence for every result.
NumRateRows	A nonnegative integer scalar giving the number of explicit rate-vertex rows in every leaf. Zero denotes an ordinary decision or affine expression, including one that stores RateBounds only as metadata.
RateBounds	Empty or an ℓ-by-2 matrix whose row s gives the lower and upper bounds for $\dot{\rho}_s$. Distinct rate rows follow the package-wide lower-then-upper combination order.
SourceSummary	Scalar origin text describing the current expression, such as `"decision"` for a constructor-created object or `"derivative"` after rhodiff.

An ordinary constructor-created pdvar has one coefficient row. It is continuous, contains decisions, has zero explicit rate rows, and records `"decision"` as its source summary. Algebra may produce a general affine state and updates the continuity and decision flags according to the result. A derivative has one row per distinct rate vertex, stores the associated rate box and row order, and records `"derivative"` as its source summary.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1 2], [10 20]}, "symmetric", ...
    Degree=[1 0], RateBounds=[-1 1; -2 2]);
gridBounds = P.GridInfo.Bounds
nodeCounts = P.GridInfo.NumNodes
matrixSize = P.MatrixSize
degree = P.Degree
firstLeafSize = size(P.LocalValues{1}{1})
isContinuous = P.IsContinuous
containsDecision = P.ContainsDecision
rateRowCount = P.NumRateRows
rateBounds = P.RateBounds
sourceSummary = P.SourceSummary
```

```
gridBounds = 2×2 double
     0     2
    10    20
nodeCounts = 1×2 double
     3     2
matrixSize = 1×2 double
     2     2
degree = 1×2 double
     1     0
firstLeafSize = 1×2 double
     1     2
isContinuous =
  logical
   1
containsDecision =
  logical
   1
rateRowCount = 0
rateBounds = 2×2 double
    -1     1
    -2     2
sourceSummary = "decision"
```

Direct assignments such as `P.Degree = 1` or `P.LocalValues = vals` raise MATLAB's private-set property error. Use the constructor or public algebra to create a modified value.

Related APIs. Use section 5.4 to traverse `LocalValues`, section 5.5 to reconstruct an affine matrix, and section 3.3 for the backend implementation contract.

5.4 Inspect Inherited Bernstein Storage

Syntax

```
cellSubs = cells(P)
cellSubs = P.cells()
labels = lbls(P)
labels = P.lbls()
```

```
coefficients = coeffs(P, cellSubs)
coefficients = P.coeffs(cellSubs)
n = ncell(P)
n = P.ncell()
n = ncoeff(P)
n = P.ncoeff()
n = npar(P)
n = P.npar()
```

Description

The inherited implementation follows the shared `pdbase` traversal order. `cells` returns an `ncell(P)`-by-ℓ numeric matrix with one physical-cell subscript per row and component $c_s \in \{1, \ldots, k_s - 1\}$, where $k_s =$ `P.GridInfo.NumNodes`(s). `lbls` returns an `ncoeff(P)`-by-ℓ numeric matrix of tensor labels with component $i_s \in \{0, \ldots, m_s\}$. Both use the package-wide combination order. `coeffs` returns the selected leaf in that label order: an ordinary decision or affine leaf is a 1-by-`ncoeff(P)` cell row of symbolic matrix payloads, while a derivative or explicit-rate leaf is `P.NumRateRows`-by-`ncoeff(P)` in stored rate-vertex order.

The counts are `ncell(P)`$= \prod_{s=1}^{\ell}($`P.GridInfo.NumNodes`$(s) - 1)$, `ncoeff(P)`$= \prod_{s=1}^{\ell}($`P.Degree`$(s) + 1)$, and `npar(P)`$= \ell =$ `numel(P.GridInfo.Vectors)`.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full", Degree=1);
physicalCells = P.cells()
labels = P.lbls()
coefficientTableSize = size(P.coeffs(1))
counts = [P.ncell(), P.ncoeff(), P.npar()]
```

```
physicalCells = 1
labels = 2×1 double
     0
     1
coefficientTableSize = 1×2 double
     1     2
counts = 1×3 double
     1     2     1
```

Remarks

Invalid physical-cell selectors raise `pdbase:InvalidCellSubs`.

Related APIs. Use section 5.3 for the stored tree and metadata, section 5.7 for formatted symbolic coefficients, and sections 3.4 and 3.4.3 for the backend implementation details.

5.5 Evaluate pdvar Expressions

Syntax

```
X = evaluate(P, point)
X = P.evaluate(point)
```

Description

`point` is a finite real scalar for a one-parameter object or a finite real vector with one coordinate per parameter, and every coordinate must lie in `P.GridInfo.Bounds`. For ordinary storage, `X` is one affine `sdpvar` matrix with size `P.MatrixSize`. For derivative or explicit-rate storage, `X` is a 1-by-N_v cell row of affine matrices in stored distinct-rate-vertex order, where $N_v =$ `P.NumRateRows`. Evaluation reconstructs the symbolic expression and does not require an assignment or solver call.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full", Degree=1);
X = P.evaluate(0.25);
sampleClass = class(X)
sampleSize = size(X)
```

```
sampleClass = 'sdpvar'
sampleSize = 1×2 double
     2     2
```

Remarks

Malformed points raise `pdvar:InvalidPoint`, and out-of-bounds points raise `pdvar:PointOutOfBounds`. Evaluation remains symbolic until the user assigns or solves the underlying YALMIP variables. Use `value` for the post-assignment numeric conversion documented in section 5.12.

Related APIs. Use section 5.4 to inspect the contributing coefficients, section 5.6 for rate-row expressions, and section 3.5 for the inherited reconstruction algorithm.

5.6 rhodiff

Syntax

```
D = rhodiff(P)
D = rhodiff(P, rb)
```

Arguments

- `P` is an ordinary `pdvar` whose `NumRateRows` equals zero.
- `rb` is a finite ℓ-by-2 numeric matrix of lower and upper parameter-rate bounds, with each lower bound at most its upper bound. It must equal `P.RateBounds` when the object already carries nonempty bounds.
- The one-input form `rhodiff(P)` requires nonempty `P.RateBounds`.
- `D` is a discontinuous `pdvar` with one coefficient row per rate vertex. In one parameter, $\boldsymbol{m} = (m_1)$ becomes $(m_1 - 1)$ when $m_1 > 0$, while (0) remains (0). In two or more parameters, every nonzero-axis partial is elevated exactly from $\boldsymbol{m} - \boldsymbol{e}_s$ to the common tensor degree $\boldsymbol{m} =$ `P.Degree` before summation. A zero-degree axis contributes zero, and an all-zero tensor degree returns a complete zero row for every rate vertex.

For a coefficient row of direction-wise degree $\boldsymbol{m}$ on physical cell $\mathcal{H}_{\boldsymbol{c}}$, define $h_s^{\boldsymbol{c}} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)} > 0$. Before rate substitution, differentiation along a nonzero-degree axis uses the forward difference

$$\left(\frac{\partial P^{(\boldsymbol{c})}}{\partial \rho_s}\right)[\boldsymbol{i}] = \frac{m_s}{h_s^{\boldsymbol{c}}} \left(P^{(\boldsymbol{c})}[\boldsymbol{i} + \boldsymbol{e}_s] - P^{(\boldsymbol{c})}[\boldsymbol{i}]\right), \qquad i_s = 0, \ldots, m_s - 1.$$

Thus the s-th partial has degree $\boldsymbol{m} - \boldsymbol{e}_s$. In one parameter this reduced degree is the public result degree. In two or more parameters, `rhodiff` elevates every nonzero partial back to the common degree $\boldsymbol{m}$ before adding the rate-weighted rows, so partials from different axes have compatible tensor labels. Section A.9 derives the reduction and exact re-elevation.

For a one-parameter degree-one coefficient pair $P^{(c)}[0], P^{(c)}[1]$, the rate-dependent derivative row reduces to

$$\dot{P}(\rho, \nu) = \nu \, \frac{P^{(c)}[1] - P^{(c)}[0]}{h_1^c}.$$

This is the one-dimensional special case: the two rows correspond to the lower and upper allowed scheduling rates.

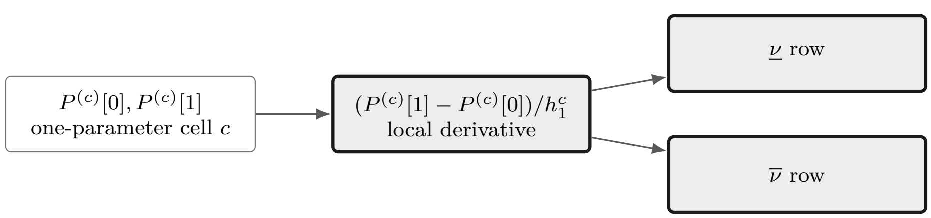


Figure 5.1: One-parameter rate-row construction. Differentiation replaces the local coefficient pair by its scaled forward difference, and the lower and upper rate values create two stored rows. Both operations preserve the physical-cell count.

For ℓ parameters, let $\boldsymbol{\nu} \in \mathcal{V}_{\mathcal{R}}$ denote one rate vertex. Nonuniform grids use a distinct step length in every direction and cell:

$$\dot{P}^{(\boldsymbol{c})}(\boldsymbol{\rho}, \boldsymbol{\nu}) = \sum_{s=1}^{\ell} \nu_s \, \frac{\partial P^{(\boldsymbol{c})}}{\partial \rho_s}(\boldsymbol{\rho}).$$

`RateBounds` fixes $\mathcal{R}$, so `rhodiff` evaluates this affine expression at the N_v distinct vertices in $\mathcal{V}_{\mathcal{R}}$ and stores one row per vertex. Here $N_v =$ `size(helper.rateVerts(P.RateBounds),1)` $\leq 2^{\ell}$, with one value from each fixed rate direction. Supply the bounds explicitly as `rhodiff(P, rb)`, or initialize `P` with `RateBounds=rb`. If both are present they must match. An empty bound set leaves the finite rate-vertex set undefined, so `rhodiff(P)` raises `pdvar:MissingRateBounds`. Section A.9.1 gives the exact hypercube-local row and coefficient layout.

Remarks

Unsupported call arity or differentiating an expression that already contains rate-vertex rows raises `pdvar:InvalidDiff`. Explicit malformed rate bounds raise `pdvar:InvalidRateBounds`. Calling the one-input form with empty stored bounds raises `pdvar:MissingRateBounds`. Explicit bounds that disagree with `P.RateBounds` raise `pdvar:RateBoundsMismatch`.

Examples and output

When rate bounds live on the object, `rhodiff(P)` uses them directly.

```
yalmip('clear')
P = pdvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
derivativeClass = class(D)
derivativeDegree = D.Degree
isContinuous = D.IsContinuous
coefficientTableSize = size(D.coeffs(1))
rateBounds = D.RateBounds
T = bernTable(D, 1, "oneLine");
disp("rateVertices =")
disp(T(:, ["RateVertexIndex", "RateVertex"]))
disp("expressions =")
disp(T.Expression)
```

```
derivativeClass = 'pdvar'
derivativeDegree = 0
isContinuous =
  logical
   0
coefficientTableSize = 1×2 double
     2     1
rateBounds = 1×2 double
    -1     1
rateVertices =
    RateVertexIndex    RateVertex
    _______________    __________

           1             {[-1]}

           2             {[1]}

expressions =
    "[internal(1)-internal(4), internal(2)-internal(5)]"
    "[internal(2)-internal(5), internal(3)-internal(6)]"
    "[-internal(1)+internal(4), -internal(2)+internal(5)]"
    "[-internal(2)+internal(5), -internal(3)+internal(6)]"
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

```
yalmip('clear')
Q = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 1; -2 2]);
```

```
E = rhodiff(Q);
% Rows 1:4 use [-1 -2], [-1 2], [1 -2], and [1 2], respectively.
derivativeClass = class(E)
derivativeDegree = E.Degree
coefficientTableSize = size(E.coeffs([1 1]))
rateVertexCount = coefficientTableSize(1)
coefficientsPerRateVertex = coefficientTableSize(2)
rateBounds = E.RateBounds
T = bernTable(E, [1 1], "oneLine");
disp("rateVertices =")
disp(T(:, ["RateVertexIndex", "RateVertex"]))
disp("expressions =")
disp(T.Expression)
```

```
derivativeClass = 'pdvar'
derivativeDegree = 1
coefficientTableSize = 1×2 double
     4     4
rateVertexCount = 4
coefficientsPerRateVertex = 4
rateBounds = 2×2 double
    -1     1
    -2     2
rateVertices =
    RateVertexIndex    RateVertex
    ---------------    ----------

           1           {[-1 -2]}
           2           {[ -1 2]}
           3           {[ 1 -2]}
           4           {[  1 2]}

expressions =
    "(1-α1) * (1-α2)*[1.2*internal(1)-0.2*internal(2)-internal(3)] + (1-α1) * α2*[0.2*internal(1)+0.8*internal(2)-internal(4)] + α1 * (1-α2)*[
      internal(1)-0.8*internal(3)-0.2*internal(4)] + α1 * α2*[internal(2)+0.2*internal(3)-1.2*internal(4)]"
    "(1-α1) * (1-α2)*[0.8*internal(1)+0.2*internal(2)-internal(3)] + (1-α1) * α2*[-0.2*internal(1)+1.2*internal(2)-internal(4)] + α1 * (1-α2)*[
      internal(1)-1.2*internal(3)+0.2*internal(4)] + α1 * α2*[internal(2)-0.2*internal(3)-0.8*internal(4)]"
    "(1-α1) * (1-α2)*[-0.8*internal(1)-0.2*internal(2)+internal(3)] + (1-α1) * α2*[0.2*internal(1)-1.2*internal(2)+internal(4)] + α1 * (1-α2)*[-
      internal(1)+1.2*internal(3)-0.2*internal(4)] + α1 * α2*[-internal(2)+0.2*internal(3)+0.8*internal(4)]"
    "(1-α1) * (1-α2)*[-1.2*internal(1)+0.2*internal(2)+internal(3)] + (1-α1) * α2*[-0.2*internal(1)-0.8*internal(2)+internal(4)] + α1 * (1-α2)
      *[-internal(1)+0.8*internal(3)+0.2*internal(4)] + α1 * α2*[-internal(2)-0.2*internal(3)+1.2*internal(4)]"
```

A zero-degree axis contributes exactly zero but remains present in the common multivariate degree and in the rate-vertex enumeration.

```
yalmip('clear')
P = pdvar(1, {[0 1], [10 20]}, Degree=[2 0]);
D = rhodiff(P, [-1 1; -2 2]);
derivativeDegree = D.Degree
coefficientTableSize = size(D.coeffs([1 1]))
rateVertexCount = coefficientTableSize(1)
coefficientsPerRateVertex = coefficientTableSize(2)
T = bernTable(D, [1 1], "oneLine");
disp("rateVertices =")
disp(T(:, ["RateVertexIndex", "RateVertex"]))
disp("expressions =")
disp(T.Expression)
```

```
derivativeDegree = 1×2 double
     2     0

coefficientTableSize = 1×2 double
     4     3

rateVertexCount = 4
coefficientsPerRateVertex = 3
rateVertices =
    RateVertexIndex    RateVertex
    ---------------    ----------
```

```
        1          {[-1 -2]}
        2          {[ -1 2]}
        3          {[ 1 -2]}
        4          {[  1 2]}

expressions =
    "(1-α1)^2 * 1*[2*internal(1)-2*internal(2)] + 2(1-α1)α1 * 1*[internal(1)-internal(3)] + α1^2 * 1*[2*internal(2)-2*internal(3)]"
    "(1-α1)^2 * 1*[2*internal(1)-2*internal(2)] + 2(1-α1)α1 * 1*[internal(1)-internal(3)] + α1^2 * 1*[2*internal(2)-2*internal(3)]"
    "(1-α1)^2 * 1*[-2*internal(1)+2*internal(2)] + 2(1-α1)α1 * 1*[-internal(1)+internal(3)] + α1^2 * 1*[-2*internal(2)+2*internal(3)]"
    "(1-α1)^2 * 1*[-2*internal(1)+2*internal(2)] + 2(1-α1)α1 * 1*[-internal(1)+internal(3)] + α1^2 * 1*[-2*internal(2)+2*internal(3)]"
```

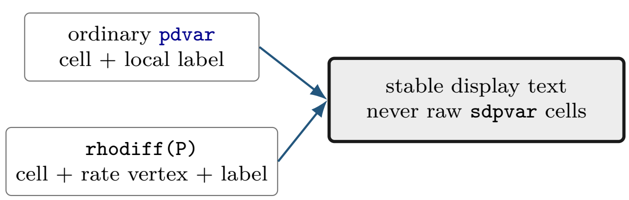
5.7 bernTable

Syntax

```
T = bernTable(P)
T = bernTable(P, cellSubs)
T = bernTable(P, "oneLine")
T = bernTable(P, cellSubs, "oneLine")
```

Arguments

- `P` is a `pdvar` object.
- `cellSubs` selects one physical cell. Omit it to list every cell.
- `"oneLine"` is the only supported text option. It returns one compact Bernstein expression per selected cell.
- `T` is a table. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes `RateVertexIndex` and `RateVertex`.



Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]});
T = bernTable(P, "oneLine");
tableHeight = height(T)
tableWidth = width(T)
hasExpression = ismember("Expression", string(T.Properties.VariableNames))
```

```
tableHeight = 1
tableWidth = 2
hasExpression = logical
   1
```

For `D = rhodiff(P)`, `bernTable(D, cellSubs, "oneLine")` emits one row for every selected physical cell and rate-box vertex in place of the local-coefficient row format. `RateVertexIndex` and `RateVertex` identify the lower or upper rate tuple, and their tensor order is defined in section A.9.1.

```
yalmip('clear')
P = pdvar(1, {[0 1], [0 1]}, RateBounds=[-1 1; -2 2]);
D = rhodiff(P);
T = bernTable(D, [1 1], "oneLine");
vertexIndices = T.RateVertexIndex
rateVertices = vertcat(T.RateVertex{:})
expressionCount = height(T)
```

```
vertexIndices = 4×1 double
     1
     2
     3
     4

rateVertices = 4×2 double
     -1    -2
     -1     2
     1    -2
     1     2

expressionCount = 4
```

Remarks

`bernTable` is an inspection method that remains separate from solver invocation. Text options other than `"oneLine"`, repeated selectors, invalid physical-cell selectors, or inconsistent rate-vertex rows raise `pdvar:InvalidBernsteinTableInput`.

5.8 Build Affine Algebra with Known Data

A `pdvar` result must remain affine in the underlying YALMIP decisions. The following blocks separate affine sums from products so the supported boundary is visible beside each example.

5.8.1 Signs, Addition, And Subtraction

Use this family to combine compatible decision expressions, coefficient-backed known data, finite real numeric data, and affine `sdpvar` matrices. For two parameter-dependent operands, alignment proceeds in a fixed order: verify matching outer bounds, form the sorted union of interior nodes on each axis, exactly subdivide/re-express both coefficient trees on that common grid, elevate both to the componentwise maximum degree, and finally add or subtract matching coefficients. Numeric and affine constant operands are promoted before the last two steps. The corresponding overload names are `plus`, `minus`, `uplus`, and `uminus`. See sections A.6 and 3.9.3.

Syntax

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P + X
R = X + P
R = P - Q
R = P - A
R = A - P
R = P - M
R = M - P
R = +P
R = -P
```

Here `A` is coefficient-backed `pdmat`, `M` is numeric, and `X` is affine `sdpvar`. A numeric scalar broadcasts to the payload size. An affine `sdpvar` is promoted as parameter-independent data.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
X = sdpvar(2, 2, 'full');
S = P + X;
D = eye(2) - P;
N = -P;
sumClass = class(S)
sumSize = size(S)
sumDegree = S.Degree
sumContainsDecision = S.ContainsDecision
differenceClass = class(D)
negativeClass = class(N)
```

```
sumClass = 'pdvar'
sumSize = 1×2 double
     2     2
sumDegree = 1
sumContainsDecision =
  logical
   1
differenceClass = 'pdvar'
negativeClass = 'pdvar'
```

A nonlinear symbolic operand such as `eta*eta` is rejected. Rate-vertex rows from `rhodiff` may be added to compatible ordinary expressions. The ordinary rows are broadcast over the active rate vertices.

The next example exercises both stages of parameter-dependent alignment: a coarse degree-one decision is refined to the known operand's two-cell grid and elevated to degree two before coefficientwise addition.

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=1);
A = pdmat({[0 0.5 1]}, @(rho) rho.^2, Degree=2);
S = P + A;
resultClass = class(S)
mergedGrid = S.GridInfo.Vectors{1}
resultDegree = S.Degree
T1 = bernTable(S, 1, "oneLine");
T2 = bernTable(S, 2, "oneLine");
disp(T1.Expression)
disp(T2.Expression)
```

```
resultClass = 'pdvar'

mergedGrid = 1×3 double
        0    0.5000    1.0000

resultDegree = 2
(1-α)^2*[internal(1)] + 2(1-α)α*[0.75*internal(1)+0.25*internal(2)] + α^2*[0.25+0.5*internal(1)+0.5*internal(2)]
(1-α)^2*[0.25+0.5*internal(1)+0.5*internal(2)] + 2(1-α)α*[0.5+0.25*internal(1)+0.75*internal(2)] + α^2*[1+internal(2)]
```

5.8.2 Multiplication By Known Or Numeric Data

The `mtimes` overload supports multiplication only when decision dependence occurs on at most one side. A coefficient-backed `pdmat` or finite real numeric matrix may multiply a `pdvar` from either side. A scalar `pdvar` may scale a known matrix. Two decision-bearing factors, including `P*Q` and `P*eta` for a decision `sdpvar eta`, are nonlinear and rejected.

Fig 5.2 summarizes the supported path from a known-data product to a finite comparison. The same affine-safety check applies before grid alignment or coefficient convolution, so an unsupported decision product cannot enter certificate assembly.

Syntax

```
R = A * P
R = P * A
R = M * P
R = P * M
```

The first example shows numeric left and right matrix products while keeping incidental YALMIP variable identifiers outside the transcript.

Examples and output

```
yalmip('clear')
P = pdvar(2, 1, {[0 1]}, "full");
L = [1 2] * P;
R = P * [4 5];
S = 3 * P;
leftSize = size(L)
rightSize = size(R)
scaledSize = size(S)
degrees = [L.Degree R.Degree S.Degree]
TL = bernTable(L, 1, "oneLine");
TR = bernTable(R, 1, "oneLine");
disp(TL.Expression)
disp(TR.Expression)
```

```
leftSize = 1×2 double
     1     1
rightSize = 1×2 double
     2     2
scaledSize = 1×2 double
     2     1
degrees = 1×3 double
     1     1     1
(1-α)*[internal(1)+2*internal(2)] + α*[internal(3)+2*internal(4)]
    "(1-α)*[4*internal(1), 5*internal(1)] + α*[4*internal(3), 5*internal(3)]"
    "(1-α)*[4*internal(2), 5*internal(2)] + α*[4*internal(4), 5*internal(4)]"
```

A known parameter-dependent factor adds its Bernstein degree componentwise and preserves the written matrix order. The three planned backend routes are derived in section A.8.

```matlab
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=2);
A = pdmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
L = A * P;
R = P * A;
knownLeftClass = class(L)
knownLeftDegree = L.Degree
knownRightClass = class(R)
knownRightDegree = R.Degree
TL = bernTable(L, 1, "oneLine");
TR = bernTable(R, 1, "oneLine");
disp(TL.Expression)
disp(TR.Expression)
```

```
knownLeftClass = 'pdvar'
knownLeftDegree = 3
knownRightClass = 'pdvar'
knownRightDegree = 3
    "(1-α)^3*[internal(1), internal(2)] + 3(1-α)^2α*[0.666666666667*internal(1)+0.666666666667*internal(4), 0.666666666667*internal(2)
      +0.666666666667*internal(5)] + 3(1-α)α^2*[1.33333333333*internal(4)+0.333333333333*internal(7), 1.33333333333*internal(5)+0.333333333333*
      internal(8)] + α^3*[2*internal(7), 2*internal(8)]"
    "(1-α)^3*[internal(2), internal(3)] + 3(1-α)^2α*[0.666666666667*internal(2)+0.666666666667*internal(5), 0.666666666667*internal(3)
      +0.666666666667*internal(6)] + 3(1-α)α^2*[1.33333333333*internal(5)+0.333333333333*internal(8), 1.33333333333*internal(6)+0.333333333333*
      internal(9)] + α^3*[2*internal(8), 2*internal(9)]"
    "(1-α)^3*[internal(1), internal(2)] + 3(1-α)^2α*[0.666666666667*internal(1)+0.666666666667*internal(4), 0.666666666667*internal(2)
      +0.666666666667*internal(5)] + 3(1-α)α^2*[1.33333333333*internal(4)+0.333333333333*internal(7), 1.33333333333*internal(5)+0.333333333333*
      internal(8)] + α^3*[2*internal(7), 2*internal(8)]"
    "(1-α)^3*[internal(2), internal(3)] + 3(1-α)^2α*[0.666666666667*internal(2)+0.666666666667*internal(5), 0.666666666667*internal(3)
      +0.666666666667*internal(6)] + 3(1-α)α^2*[1.33333333333*internal(5)+0.333333333333*internal(8), 1.33333333333*internal(6)+0.333333333333*
      internal(9)] + α^3*[2*internal(8), 2*internal(9)]"
```

Direction-wise degrees add independently. A known factor that varies only in the second parameter raises only the second component of the result degree.

```matlab
yalmip('clear')
P = pdvar(1, {[0 1], [0 1]}, Degree=[2 0]);
A = pdmat({[0 1], [0 1]}, ...
    @(rho1, rho2) 1 + rho2, Degree=[0 1]);
L = A * P;
R = P * A;
leftDegree = L.Degree
rightDegree = R.Degree
TL = bernTable(L, [1 1], "oneLine");
TR = bernTable(R, [1 1], "oneLine");
disp(TL.Expression)
disp(TR.Expression)
```

```
leftDegree = 1×2 double
     2     1

rightDegree = 1×2 double
     2     1
(1-α1)^2 * (1-α2)*[internal(1)] + (1-α1)^2 * α2*[2*internal(1)] + 2(1-α1)α1 * (1-α2)*[internal(2)] + 2(1-α1)α1 * α2*[2*internal(2)] + α1^2 *
      (1-α2)*[internal(3)] + α1^2 * α2*[2*internal(3)]
(1-α1)^2 * (1-α2)*[internal(1)] + (1-α1)^2 * α2*[2*internal(1)] + 2(1-α1)α1 * (1-α2)*[internal(2)] + 2(1-α1)α1 * α2*[2*internal(2)] + α1^2 *
      (1-α2)*[internal(3)] + α1^2 * α2*[2*internal(3)]
```

A derivative object may be multiplied by numeric data or a matching-grid known object while preserving every rate row. Products with rate dependence on both sides, products of a derivative with an ordinary decision expression, and products involving metadata-only rate dependence are rejected.

Two decision-bearing factors are also rejected: their product would be quadratic in YALMIP decision coefficients. The supported boundary is numeric or known pdmat data multiplying one affine pdvar expression on either side, with the written matrix order preserved.

Remarks

Incompatible affine operands raise `pdvar:InvalidAddition` or `pdvar:InvalidSubtraction`. Bad dimensions, two decision-bearing factors, nonlinear symbolic data, or unsupported rate combinations raise `pdvar:InvalidMultiplication`. Incompatible parameter bounds raise `pdvar:MixedGrid`. A function-only pdmat operand raises `pdvar:FunctionOnlyAlgebra`.

5.9 Reshape and Transform Decision Matrices

Every method below is inherited from the centralized pdbase implementation and transforms each local affine YALMIP payload while the pdvar reconstruction hook preserves the physical grid, degree, class, decision metadata, and compatible rate rows. The examples report stable classes, dimensions, and metadata in place of YALMIP's session-local variable numbers.

5.9.1 Shape, Equality, And MATLAB Protocols

Shape queries describe the matrix payload. isequal is an exact identity check: grid, degree, matrix size, metadata, and local symbolic payloads must already match. Each operand retains its original grid and degree during this check.

Syntax

```
sz = size(P)
d = size(P, dim)
n = height(P)
n = width(P)
n = length(P)
n = numel(P)
n = ndims(P)
tf = isequal(P, Q, ...)
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
matrixSize = size(P)
rowCount = height(P)
columnCount = width(P)
longestDimension = length(P)
elementCount = numel(P)
dimensionCount = ndims(P)
equalsUnaryPlus = isequal(P,+P)
```

```
matrixSize = 1×2 double
     2     3

rowCount = 2
columnCount = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
equalsUnaryPlus =
  logical
   1
```

`end`, `subsref`, `subsasgn`, and `numArgumentsFromSubscript` support the two-index and dot-access forms shown in section 5.11. Physical-cell storage remains encapsulated behind these matrix-level protocols.

5.9.2 Transpose And Conjugate Transpose

Both forms transpose every local affine payload and preserve the YALMIP coefficient graph.

Syntax

```
T = transpose(P)
H = ctranspose(P)
T = P.'
H = P'
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
T = P.';
H = P';
transposeSize = size(T)
conjugateTransposeSize = size(H)
transposeClass = class(T)
conjugateTransposeClass = class(H)
```

```
transposeSize = 1×2 double
     3     2
conjugateTransposeSize = 1×2 double
     3     2
transposeClass = 'pdvar'
conjugateTransposeClass = 'pdvar'
```

$$\begin{array}{|c|c|c|}\hline p_{11} & p_{12} & p_{13} \\ \hline p_{21} & p_{22} & p_{23} \\ \hline \end{array} \xrightarrow{\text{each coefficient}} \begin{array}{|c|c|}\hline p_{11} & p_{21} \\ \hline p_{12} & p_{22} \\ \hline p_{13} & p_{23} \\ \hline \end{array}$$

5.9.3 Vectorization, Reshape, And Squeeze

`vec` uses MATLAB column-major order. `reshape` accepts two positive dimensions with at most one inferred `[]` dimension and preserves the element count. `squeeze` retains the two-dimensional package convention.

Syntax

```
V = vec(P)
R = reshape(P, [m n])
R = reshape(P, m, n)
S = squeeze(P)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
V = vec(P);
R = reshape(P, [3 2]);
S = squeeze(P);
vectorSize = size(V)
reshapeSize = size(R)
squeezeSize = size(S)
```

```
vectorSize = 1×2 double
     6     1
reshapeSize = 1×2 double
     3     2
squeezeSize = 1×2 double
     2     3
```

Malformed or size-changing reshape requests raise `pdvar:InvalidReshape`.

5.9.4 Diagonals And Trace

`diag` extracts a diagonal or constructs a diagonal matrix from a vector. The selected offset requires a nonempty value. `trace` returns a scalar decision expression.

Syntax

```
d = diag(P)
d = diag(P, k)
D = diag(v)
D = diag(v, k)
t = trace(P)
```

Examples and output

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
d = diag(P);
D = diag(d);
t = trace(P);
diagonalSize = size(d)
rebuiltSize = size(D)
traceSize = size(t)
```

```
diagonalSize = 1×2 double
     3     1
rebuiltSize = 1×2 double
     3     3
traceSize = 1×2 double
     1     1
```

Invalid offsets or empty selections raise `pdvar:InvalidDiag`.

5.9.5 Triangular Parts And Reductions

Triangular methods zero the complementary part of every coefficient. Reductions act on matrix dimensions. `"all"` is available for `sum` and `mean`.

Syntax

```
L = tril(P)
L = tril(P, k)
U = triu(P)
U = triu(P, k)
S = sum(P)
S = sum(P, dim)
S = sum(P, vecdim)
S = sum(P, "all")
M = mean(P)
M = mean(P, dim)
M = mean(P, vecdim)
M = mean(P, "all")
C = cumsum(P)
C = cumsum(P, dim)
C = cumsum(P, direction)
C = cumsum(P, dim, direction)
```

First form the two triangular views.

Examples and output

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
L = tril(P);
U = triu(P);
lowerSize = size(L)
upperSize = size(U)
degrees = [L.Degree U.Degree]
```

```
lowerSize = 1×2 double
     3     3
upperSize = 1×2 double
     3     3
degrees = 1×2 double
     1     1
```

Then reduce a rectangular expression.

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
rowSum = sum(P, 2);
allMean = mean(P, "all");
running = cumsum(P, 2);
rowSumSize = size(rowSum)
allMeanSize = size(allMean)
runningSize = size(running)
```

```
rowSumSize = 1×2 double
     2     1
allMeanSize = 1×2 double
     1     1
runningSize = 1×2 double
     2     3
```

`vecdim` is a nonempty vector of unique positive integer dimensions. Dimensions above two are singleton identity operations. `direction` is `"forward"` or `"reverse"`, case-insensitively. A direction supplied by itself uses the first nonsingleton matrix dimension, and `cumsum` along a dimension above two returns the same symbolic payloads. Invalid triangular offsets raise `pdvar:InvalidTriangularPart`. Invalid reduction calls raise `pdvar:InvalidSum`, `pdvar:InvalidMean`, or `pdvar:InvalidCumsum`. Missing-value and native-output flags lie outside the supported call forms.

5.9.6 Reordering And Rotation

These methods rearrange matrix entries inside every symbolic coefficient. The parameter grid retains its original order.

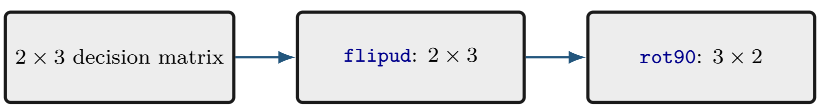
Syntax

```
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
F = flipud(P);
R = rot90(P);
flippedSize = size(F)
rotatedSize = size(R)
flippedClass = class(F)
rotatedClass = class(R)
```

```
flippedSize = 1×2 double
     2     3
rotatedSize = 1×2 double
     3     2
flippedClass = 'pdvar'
rotatedClass = 'pdvar'
```



Invalid calls raise `pdvar:InvalidFlip` or `pdvar:InvalidRot90`.

5.9.7 Concatenation And Block Assembly

Assembly accepts compatible `pdvar`, coefficient-backed `pdmat`, affine `sdpvar`, and finite real numeric blocks. It aligns grids and elevates all blocks exactly to the componentwise maximum degree while preserving an affine expression. `cat` supports only matrix dimensions 1 and 2.

Syntax

```
H = horzcat(P, Q, ...)
V = vertcat(P, Q, ...)
H = [P, Q]
V = [P; Q]
C = cat(dim, P, Q, ...)
D = blkdiag(P, Q, ...)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
H = [P, P];
V = [P; P];
D = blkdiag(P, eye(2));
horizontalSize = size(H)
verticalSize = size(V)
blockDiagonalSize = size(D)
```

```
horizontalSize = 1×2 double
     2     4
verticalSize = 1×2 double
     4     2
blockDiagonalSize = 1×2 double
     4     4
```

Incompatible blocks or rate metadata raise `pdvar:InvalidConcatenation`. Unsupported dimensions raise `pdvar:UnsupportedCatDimension`. Invalid block-diagonal inputs raise `pdvar:InvalidBlkdiag`. Incompatible bounds raise `pdvar:MixedGrid`. Function-only known blocks raise `pdvar:FunctionOnlyAlgebra`.

5.9.8 Repetition

`repmat` tiles the matrix payload while retaining the parameter metadata. It accepts a positive scalar, two positive counts, or a two-element count vector.

Syntax

```
Q = repmat(P, m)
Q = repmat(P, m, n)
Q = repmat(P, [m n])
```

Examples and output

```
yalmip('clear')
P = pdvar(1, 2, {[0 1]}, "full");
Q = repmat(P, [2 2]);
repeatedSize = size(Q)
repeatedDegree = Q.Degree
containsDecision = Q.ContainsDecision
```

```
repeatedSize = 1×2 double
     2     4
repeatedDegree = 1
containsDecision =
  logical
   1
```

Invalid repetition shapes raise `pdvar:InvalidRepmat`.

5.10 Elevate `pdvar` Bernstein Storage

Syntax

```
Q = elevate(P, degreeIncrement)
Q = P.elevate(degreeIncrement)
Q = elevate(P, degreeIncrement, validationMode)
Q = P.elevate(degreeIncrement, validationMode)
```

Description

`degreeIncrement` is a finite nonnegative integer scalar shorthand or an ℓ-element direction-wise vector. A scalar expands uniformly, while a vector adds componentwise to `P.Degree`. `elevate` returns a new `pdvar` of the same dynamic class with the same represented expression, physical cells, YALMIP variables, metadata, rate-row order, and decision freedom at the higher degree. The original `P` remains unchanged. The optional positional `validationMode` is the case-insensitive scalar text `"fast"` or `"strict"`. Its call-local checks follow the canonical contract in section 3.9.1.

The sparse change of Bernstein basis is exact. It raises the source degree $\boldsymbol{m}$ to $\boldsymbol{M} = \boldsymbol{m} + \boldsymbol{d}$ while preserving the existing decision variables and represented affine expression. Section A.6 derives the shared elevation operator.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full", Degree=1);
Q = P.elevate(2);
elevatedClass = class(Q)
degrees = [P.Degree, Q.Degree]
elevatedCoefficientTableSize = size(Q.coeffs(1))
sourceCoefficients = P.coeffs(1);
elevatedCoefficients = Q.coeffs(1);
sourceVariables = unique(getvariables([sourceCoefficients{:}]));
elevatedVariables = unique(getvariables([elevatedCoefficients{:}]));
variableCounts = [numel(sourceVariables), numel(elevatedVariables)]
reusesVariables = isequal(sourceVariables, elevatedVariables)
```

```
elevatedClass = 'pdvar'
degrees = 1×2 double
     1     3
elevatedCoefficientTableSize = 1×2 double
     1     4
variableCounts = 1×2 double
     8     8
reusesVariables =
  logical
   1
```

Vector increments may elevate different axes by different amounts, including an axis on which the original decision is constant.

```
yalmip('clear')
P = pdvar(1, {[0 1], [0 1]}, Degree=[1 0]);
Q = P.elevate([1 2]);
elevatedClass = class(Q)
sourceDegree = P.Degree
elevatedDegree = Q.Degree
elevatedCoefficientTableSize = size(Q.coeffs([1 1]))
sourceCoefficients = P.coeffs([1 1]);
elevatedCoefficients = Q.coeffs([1 1]);
sourceVariables = unique(getvariables([sourceCoefficients{:}]));
elevatedVariables = unique(getvariables([elevatedCoefficients{:}]));
variableCounts = [numel(sourceVariables), numel(elevatedVariables)]
reusesVariables = isequal(sourceVariables, elevatedVariables)
```

```
elevatedClass = 'pdvar'
sourceDegree = 1×2 double
     1     0

elevatedDegree = 1×2 double
     2     2
elevatedCoefficientTableSize = 1×2 double
     1     9
variableCounts = 1×2 double
     2     2
reusesVariables =
  logical
   1
```

Remarks

Invalid increments raise `pdbase:InvalidDegreeIncrement`, and invalid validation modes raise `pdbase:InvalidValidationMode`. Strict validation reports malformed leaf structure through `pdbase:InvalidCoefficientCell` and malformed or inconsistent payloads through `pdbase:InvalidCoefficientPayload`.

Related APIs. Use section 5.3 to inspect the preserved metadata, section 5.7 to view the elevated coefficients, and sections A.6 and 3.9.1 for the backend contract and operator derivation.

5.11 Index and Assign `pdvar` Entries

Parenthesis indexing selects a matrix block from every symbolic coefficient. Assignment writes a compatible numeric, `pdvar`, coefficient-backed `pdmat`, or affine `sdpvar` block into that position after the same common-grid refinement and componentwise maximum-degree elevation used by affine addition. The left object and parameter-dependent right-hand side are exactly re-expressed on the sorted union grid before the selected payload block is replaced at every aligned label. Physical-cell selection belongs to `cells` and `coeffs`. See sections A.6 and 3.9.3.

Syntax

```
Q = P(rows, cols)
Q = P(1:end, end)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
lastCol = P(:, end);
P(:, 1) = 0;
selectedSize = size(lastCol)
assignedSize = size(P)
assignedClass = class(P)
assignedDegree = P.Degree
containsDecision = P.ContainsDecision
```

```
selectedSize = 1×2 double
     2     1
assignedSize = 1×2 double
     2     3
assignedClass = 'pdvar'
assignedDegree = 1
containsDecision =
```

```
  logical
   1
```

Assignment may therefore increase both the grid resolution and the degree of the complete decision expression while retaining its affine decision content.

```
yalmip('clear')
P = pdvar(2, 2, {[0 1]}, "full", Degree=1);
A = pdmat({[0 0.5 1]}, ...
    @(rho) [rho.^2; 1+rho], Degree=2);
P(:, 2) = A;
assignedGrid = P.GridInfo.Vectors{1}
assignedDegree = P.Degree
assignedClass = class(P)
storageCounts = [P.ncell(), P.ncoeff()]
containsDecision = P.ContainsDecision
```

```
assignedGrid = 1×3 double
         0    0.5000    1.0000

assignedDegree = 2
assignedClass = 'pdvar'

storageCounts = 1×2 double
     2     3

containsDecision =
  logical
   1
```

Invalid selectors raise `pdvar:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdvar:UnsupportedAssignment`. Incompatible or nonlinear right-hand sides raise `pdvar:InvalidAssignment`.

5.12 value

Syntax

```
A = value(P)
rows = value(rhodiff(P))
```

`value` converts assigned `pdvar` coefficients into known, coefficient-backed `pdmat` data. An ordinary expression returns one `pdmat`, preserving its grid, matrix size, degree, local coefficient order, and continuity metadata. A rate-vertex expression created by `rhodiff` returns a 1-by-N_v cell array of `pdmat` objects, where $N_v =$ `P.NumRateRows`, in `helper.rateVerts(P.RateBounds)` order. Each returned `pdmat` has empty rate metadata. Every symbolic coefficient must have an assigned finite real YALMIP value. Otherwise the method raises `pdvar:UnassignedValue`.

Examples and output

```
yalmip('clear')
P = pdvar(1, [0 1], Degree=2);
c = P.coeffs(1);
assign([c{:}], [1 2 4]);
A = value(P);
assignedCoefficients = cell2mat(A.coeffs(1))
```

```
assignedCoefficients = 1×3 double
     1     2     4
```

For rate rows, `value` returns one known object per rate vertex and preserves the row separation.

```
yalmip('clear')
P = pdvar(1, [0 1], Degree=1);
c = P.coeffs(1);
assign([c{:}], [1 3]);
D = rhodiff(P, [-1 1]);
rows = value(D);
rowCount = numel(rows)
lowerRowClass = class(rows{1})
upperRowClass = class(rows{2})
lowerRateCoefficients = cell2mat(rows{1}.coeffs(1))
upperRateCoefficients = cell2mat(rows{2}.coeffs(1))
```

```
rowCount = 2
lowerRowClass = 'pdmat'
upperRowClass = 'pdmat'
lowerRateCoefficients = -2
upperRateCoefficients = 2
```

5.13 Comparisons To `pdlmi`

The `le`, `ge`, and `eq` overloads implement `<=`, `>=`, and `==`. Inequalities select either semidefinite or entry-wise meaning once for the complete original residual. Equality is always direct coefficient equality.

Syntax

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- `rhs` may be compatible numeric, coefficient-backed `pdmat`, affine `sdpvar`, or `pdvar` data. Ordinary operands use the same grid refinement, componentwise maximum-degree alignment, and promotion as subtraction.
- For `<=` and `>=`, every coefficient in every physical cell and rate row is scanned before assembly. The complete relation is semidefinite only if every original coefficient is square Hermitian. Numeric Hermitian testing uses the inclusive rule $\|A-A^\top\|_\infty \leq 10^{-10}$. Symbolic coefficients use YALMIP's `ishermitian`. If any coefficient fails, the complete inequality is entry-wise and warning `pdlmi:ElementwiseInequality` is issued once. Each complete relation uses one consistent mode across all coefficients and entries.
- `==` accepts rectangular and nonsymmetric residuals and equates corresponding coefficient entries directly. Ordinary rows compare with ordinary rows. Derivative rows compare only with compatible derivative rows.
- `C` is a `pdlmi` wrapper. A comparison assembles constraints and leaves optimization invocation to the caller.

Examples and output

Comparison creates a `pdlmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

```
yalmip('clear')
P = pdvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
negativeWrapperClass = class(Cneg)
negativeConstraintCount = numel(Cneg.Constraints)
Cpos = P >= 0;
positiveWrapperClass = class(Cpos)
positiveConstraintCount = numel(Cpos.Constraints)
```

```
negativeWrapperClass = 'pdlmi'
negativeConstraintCount = 4
positiveWrapperClass = 'pdlmi'
positiveConstraintCount = 4
```

A rectangular residual selects entry-wise inequality globally. Equality is entry-wise by definition and is warning-free.

```
yalmip('clear')
V = pdvar(3, 2, {[0 1]}, "full");
warnId = "pdlmi:ElementwiseInequality";
warnState = warning("off", warnId);
restoreWarning = onCleanup(@() warning(warnState));
Centry = V >= 0;
Ceq = V == sdpvar(3, 2, 'full');
entrywiseConstraintGroups = numel(Centry.Constraints)
equalityConstraintGroups = numel(Ceq.Constraints)
identity = V == V;
identityConstraints = identity.toYalmip()
clear restoreWarning
```

```
entrywiseConstraintGroups = 2
equalityConstraintGroups = 2

identityConstraints =

     []
```

Each group above corresponds to one local coefficient and contains six scalar entries in MATLAB column-major order. Under Pólya, Putinar, SparsePutinar, SparseFullBox, or FullBox, each entry of an entry-wise inequality receives an independent scalar certificate. Equality is direct-only. Constructor certificate options and `usePolya`, `usePutinar`, `useSpPut`, `useSpBox`, or `useFullBox` raise `pdlmi:UnsupportedEqualityCertificate` before validating an increment, clique size, width, or order. Mixed ordinary and derivative row kinds raise `pdvar:InvalidEqualityRows`. Use overloaded `==` to build a modeling equality, and use `isequal` only for MATLAB structural comparison.

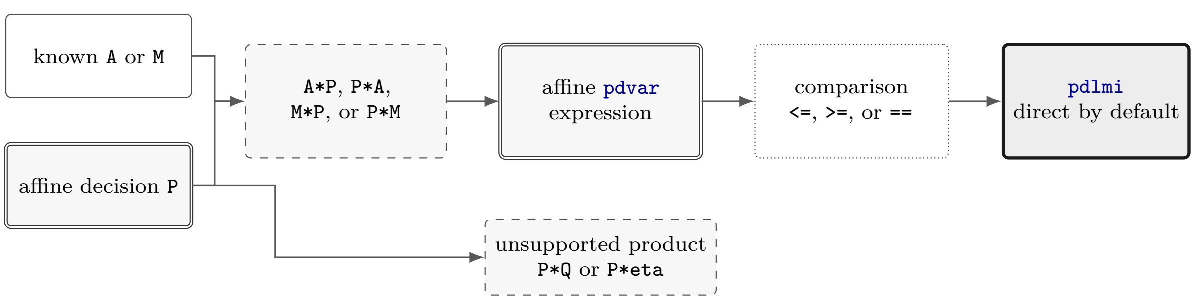


Figure 5.2: Supported affine path from known-data multiplication to a `pdlmi` comparison. Decision dependence occurs on one product side, so the result remains affine and can form a residual. A product with decisions on both sides is rejected before comparison.

5.14 Boundaries and Related APIs

1. Constructor-created pdvar objects support every finite nonnegative scalar or direction-wise integer degree. Each cell stores $\prod_{s=1}^{\ell}(m_s + 1)$ Bernstein control matrices. A zero-degree axis is constant in that direction.

2. Products may contain decision variables on at most one side. Nonlinear decision products such as `P * Q` are outside this slice.

3. Function-only pdmat operands require reconstruction with explicit Bernstein evidence before entering pdvar algebra.

4. The current public rhodiff workflow accepts an ordinary expression whose NumRateRows equals zero.

Related APIs. pdvar, rhodiff, bernTable, mtimes, pdlmi, pdmat.

Chapter 6

pdlmi: Finite Certificate Assembly

pdlmi stores finite certificates assembled from pdvar expressions or coefficient-backed pdmat inequalities. Square Hermitian inequality families use semidefinite constraints, other inequalities use entry-wise scalar constraints, and pdvar equality uses direct coefficient equations. Direct assembly is the default. The five explicit inequality alternatives are cell-wise Pólya elevation, dense Putinar, tensor-window SparsePutinar, tensor-window SparseFullBox, and the dense full-box preordering. This class is the package boundary between Bernstein objects and ordinary YALMIP solver calls.

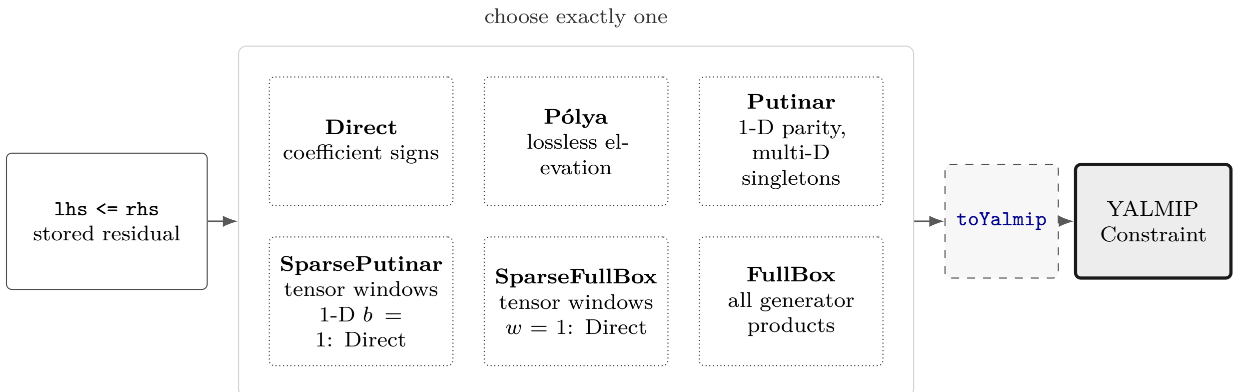


Figure 6.1: Certificate-family state and export path. The six paths are alternatives, and every selector rebuilds from the original comparison. SparsePutinar becomes dense Putinar when a nontrivial window covers every active Putinar basis. SparseFullBox becomes FullBox when its window covers every order axis.

6.1 API Map

Task	Entry
Construct wrapper	`pdlmi`, `<=`/`>=`/`==`
Count constraints	direct coefficient constraints
Select Pólya elevation	`UsePolya`, `PolyaDegree`, and `usePolya`
Select Putinar certificate	`UsePutinar`, `PutinarOrder`, and `usePutinar`
Select SparsePutinar	`UseSparsePutinar`, `SparsePutinarOrder`, `CliqueSize`, and `useSpPut`
Select SparseFullBox	`UseSparseFullBoxPreorder`, `SparseFullBoxOrder`, `BandWidth`, and `useSpBox`
Select FullBox	`UseFullBoxPreorder`, `FullBoxOrder`, and `useFullBox`
Export to YALMIP	`toYalmip`
Solve	solver-facing use, Examples

6.2 Inspect Certificate State

All `pdlmi` properties have private set access and are readable with dot notation. `Constraints` contains the assembled YALMIP entries. `Residual` and `Relation` retain the original comparison for rebuilding. The remaining properties record the selected certificate state.

Property	Meaning
`Constraints`	Stored finite YALMIP constraint entries.
`Residual`, `Relation`	Original `pdvar` residual and one of `"<="`, `">="`, or `"=="`.
`UsePolya`, `PolyaDegree`	Pólya selector and nonnegative degree increment.
`UsePutinar`, `PutinarOrder`	Putinar selector and absolute Gram order.
`UseSparsePutinar`, `SparsePutinarOrder`, `CliqueSize`	SparsePutinar selector, absolute Putinar Gram order, and tensor-window side length.
`UseSparseFullBoxPreorder`, `SparseFullBoxOrder`, `BandWidth`	SparseFullBox selector, absolute Gram order, and common sliding tensor-window side length.
`UseFullBoxPreorder`, `FullBoxOrder`	Full-box selector and absolute Gram order.

For example, `C.Relation`, `C.Residual.Degree`, and `C.UseSparsePutinar` inspect a wrapper. Selector methods return a rebuilt value, while assigning these properties is unsupported. Selected Pólya and Gram parameters are stored as 1-by-ℓ rows. An intermediate SparsePutinar result stores `UseSparsePutinar=true`, its order row, and its clique size. In one parameter, clique size one becomes actual Direct state. In several parameters, clique size one remains SparsePutinar. Any clique size greater than one that covers every active Gram basis becomes dense Putinar state. SparseFullBox separately uses `SparseFullBoxOrder` and the historically named `BandWidth` property to store its common tensor-window side length, with Direct and FullBox endpoints as described in section 6.8.

6.3 Construct pdlmi Comparisons

Syntax

```
C = pdlmi(expr, relation)
C = pdlmi(expr, "==")
C = pdlmi(expr, relation, "UsePolya")
C = pdlmi(expr, relation, UsePolya=true, PolyaDegree=d)
C = pdlmi(expr, relation, "UsePolya", "PolyaDegree", d)
C = pdlmi(expr, relation, "UsePutinar")
C = pdlmi(expr, relation, UsePutinar=true, PutinarOrder=r)
C = pdlmi(expr, relation, PutinarOrder=r)
C = pdlmi(expr, relation, "UseSparsePutinar")
C = pdlmi(expr, relation, UseSparsePutinar=true)
C = pdlmi(expr, relation, CliqueSize=b)
C = pdlmi(expr, relation, SparsePutinarOrder=r)
C = pdlmi(expr, relation, UseSparsePutinar=true, ...
    CliqueSize=b, SparsePutinarOrder=r)
C = pdlmi(expr, relation, "UseSparseFullBoxPreorder")
C = pdlmi(expr, relation, UseSparseFullBoxPreorder=true)
C = pdlmi(expr, relation, BandWidth=w)
C = pdlmi(expr, relation, SparseFullBoxOrder=r)
C = pdlmi(expr, relation, UseSparseFullBoxPreorder=true, ...
    BandWidth=w, SparseFullBoxOrder=r)
C = pdlmi(expr, relation, "UseFullBoxPreorder")
C = pdlmi(expr, relation, UseFullBoxPreorder=true)
C = pdlmi(expr, relation, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=true, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=false, FullBoxOrder=r)
C = pdlmi(expr, relation, ValidationMode=mode)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- `expr` is a `pdvar` residual, or a coefficient-backed `pdmat` residual for `"<="` or `">="`. `relation` is exactly `"<="`, `">="`, or `"=="`. Inequalities are classified globally from the original coefficient family. `pdvar` equality is entry-wise direct coefficient assembly and accepts rectangular or nonsymmetric residuals. `pdmat` equality is instead the scalar logical comparison `A==B` and never creates a `pdlmi`.
- `UsePolya` is a logical scalar option with default `false`. Its bare form, `"UsePolya"`, and `UsePolya=true` with the degree omitted select increment one.
- `PolyaDegree=d` accepts one finite nonnegative integer scalar shorthand or an ℓ-element vector and is stored as a 1-by-ℓ row. It elevates the residual componentwise by $\boldsymbol{d}$ before assembling constraints. Supplying the degree by itself enables Pólya and issues `pdlmi:ImplicitUsePolya`. `UsePolya=false` with any positive component raises `pdlmi:ConflictingPolyaOptions`.

- UsePutinar is a logical scalar option with default `false`. Its bare form and the explicit value `true` select the dimension-appropriate Putinar certificate at the smallest admissible order.

- `PutinarOrder=r` accepts one finite nonnegative integer scalar shorthand or an ℓ-element vector and is stored as the 1-by-ℓ absolute Bernstein Gram order $\boldsymbol{r}$, which is distinct from a degree increment. For one parameter, $\boldsymbol{M} = (M_1)$ and the default and minimum are $\boldsymbol{r} = (\lfloor M_1/2 \rfloor)$. For two or more parameters, the default and componentwise minimum are $\lceil \boldsymbol{M}/2 \rceil$. Supplying the order by itself enables Putinar assembly. Pairing `UsePutinar=false` with any explicit order, including zero, raises `pdlmi:ConflictingPutinarOptions`.

- UseSparsePutinar is a logical scalar option with default `false`. Its bare form and the explicit value `true` select SparsePutinar with default `CliqueSize=2`. Supplying CliqueSize or SparsePutinarOrder alone also enables this family. Explicit `UseSparsePutinar=false` conflicts with either parameter.

- `CliqueSize=b` is one finite positive integer tensor-window side length. `SparsePutinarOrder=r` accepts scalar shorthand or an ℓ-element vector and is stored as the 1-by-ℓ absolute Putinar Gram order $\boldsymbol{r}$. Its default and minimum are $(\lfloor M_1/2 \rfloor)$ when $\boldsymbol{M} = (M_1)$ and componentwise $\lceil \boldsymbol{M}/2 \rceil$ otherwise. Endpoint normalization occurs only after order validation. In one parameter, `b=1` returns actual Direct state. In several parameters, `b=1` retains SparsePutinar state. Any `b>1` satisfying $b \geq r_s + 1$ for every direction returns actual dense Putinar state, while all other valid sizes retain SparsePutinar state.

- `UseSparseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select SparseFullBox with default `BandWidth=2`. BandWidth or SparseFullBoxOrder alone also enables this family. Explicit `UseSparseFullBox Preorder=false` conflicts with either sparse parameter.

- `BandWidth=w` remains the selected-state property name, while w is implemented as one finite positive tensor-window side length. Flattened matrix bandwidths belong to a different construction. `SparseFullBoxOrder=r` accepts scalar shorthand or an ℓ-element vector and is stored as the 1-by-ℓ absolute Gram order $\boldsymbol{r}$. Its default and minimum are $(\lfloor M_1/2 \rfloor)$ when $\boldsymbol{M} = (M_1)$ and componentwise $\lceil \boldsymbol{M}/2 \rceil$ otherwise, where $\boldsymbol{M} =$ `C.Residual.Degree`. Width one canonicalizes to actual Direct state after order validation, a width that spans every order axis canonicalizes to actual FullBox state, and intermediate widths select SparseFullBox state.

- UseFullBoxPreorder is a logical scalar option with default `false`. Its bare form and the explicit value `true` select full-box assembly at the smallest admissible order.

- `FullBoxOrder=r` accepts scalar shorthand or an ℓ-element vector and is stored as the 1-by-ℓ absolute order $\boldsymbol{r}$, which is distinct from a degree increment. Supplying it by itself enables full-box assembly. When FullBox is enabled, the minimum is $\boldsymbol{r} = (\lfloor M_1/2 \rfloor)$ for a one-parameter residual with $\boldsymbol{M} = (M_1)$, and componentwise $\lceil \boldsymbol{M}/2 \rceil$ for two or more parameters. Explicitly pairing `UseFullBoxPreorder=false` with `FullBoxOrder=r` suppresses that automatic selection: `C` remains direct and records the normalized row only as inspectable metadata, while assembly remains direct.

- Pólya, Putinar, SparsePutinar, SparseFullBox, and FullBox selection are mutually exclusive and apply only to inequalities. Any recognized certificate option on an equality raises `pdlmi:UnsupportedEqualityCertificate` before clique, width, or order validation. Enabling more than one inequality family raises `pdlmi:ConflictingRelaxations`.

- `ValidationMode=mode` accepts scalar text `"fast"` or `"strict"`, case-insensitively, and defaults to `"fast"`. It is transient and discarded after validation. Every certificate selector accepts its own trailing `ValidationMode`, and each later call uses its locally supplied mode or the default.

- `C` is direct coefficient-wise assembly unless one opt-in inequality mode is selected. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and accept compatible numeric, `pdmat`, affine `sdpvar`, and `pdvar` right-hand sides.

The inspectable properties `Constraints`, `Residual`, and `Relation` expose the assembled finite constraints and the original comparison data used for rebuilding. Their set access is private. `UsePolya` and `PolyaDegree` record the Pólya state. `UsePutinar` and `PutinarOrder` record the dense Putinar state. `UseSparsePutinar`, `SparsePutinarOrder`, and `CliqueSize` record the SparsePutinar state. `UseSparseFullBoxPreorder`, `SparseFullBoxOrder`, and `BandWidth` record the SparseFullBox state. `UseFullBoxPreorder` and `FullBoxOrder` record the FullBox state. A comparison-created Direct object has every selector set to false and every numeric certificate property set to zero. The explicit-false FullBox form is the sole exception because it retains the supplied order in `FullBoxOrder`, keeps `UseFullBoxPreorder` false, and leaves the stored constraints direct. The Putinar, SparsePutinar, and SparseFullBox selectors reject their corresponding explicit-false and explicit-order combinations.

Examples and output

The direct constructor is equivalent to the comparison path when the relation and options are explicit.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = pdlmi(P, ">=", UsePolya=false, PolyaDegree=0);
wrapperClass = class(C)
constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
polyaDegree = C.PolyaDegree
usePutinar = C.UsePutinar
putinarOrder = C.PutinarOrder
useSparsePutinar = C.UseSparsePutinar
sparsePutinarOrder = C.SparsePutinarOrder
cliqueSize = C.CliqueSize
useFullBox = C.UseFullBoxPreorder
fullBoxOrder = C.FullBoxOrder
useSparseFullBox = C.UseSparseFullBoxPreorder
sparseFullBoxOrder = C.SparseFullBoxOrder
bandWidth = C.BandWidth
```

```
wrapperClass = 'pdlmi'
constraintCount = 2
usePolya =
  logical
   0
polyaDegree = 0
usePutinar =
  logical
   0
putinarOrder = 0
useSparsePutinar =
  logical
```

```
    0
sparsePutinarOrder = 0
cliqueSize = 0
useFullBox =
  logical
    0
fullBoxOrder = 0
useSparseFullBox =
  logical
    0
sparseFullBoxOrder = 0
bandWidth = 0
```

Remarks

1. A residual outside the supported `pdvar`/`pdmat` forms raises `pdlmi:InvalidExpression`. A function-only `pdmat` raises `pdlmi:MissingCoefficientEvidence`, `pdlmi(A,"==")` for `pdmat` raises `pdlmi:UnsupportedPdmatEquality`, and an unsupported relation raises `pdlmi:InvalidRelation`.

2. Malformed, unknown, or duplicate options raise `pdlmi:InvalidOptions`, `pdlmi:UnknownOption`, or `pdlmi:DuplicateOption`. Invalid validation modes raise `pdlmi:InvalidValidationMode`, and nonlogical selectors use their corresponding `InvalidUse...` identifier.

3. Malformed Pólya increments, Putinar orders, SparsePutinar orders, clique sizes, SparseFullBox orders, widths, and full-box orders raise their respective `Invalid...` identifiers. These include `pdlmi:InvalidSparsePutinarOrder` and `pdlmi:InvalidCliqueSize`. An integer order below an active Gram-family minimum raises the corresponding `...OrderTooLow` identifier, including `pdlmi:SparsePutinarOrderTooLow`.

4. Explicit false plus a Putinar order raises `pdlmi:ConflictingPutinarOptions`. Explicit false plus a SparsePutinar parameter raises `pdlmi:ConflictingSparsePutinarOptions`, while explicit false plus a SparseFullBox parameter raises `pdlmi:ConflictingSparseFullBoxOptions`. Explicit false plus a positive Pólya increment raises `pdlmi:ConflictingPolyaOptions`. The full-box explicit-false form remains direct and retains the supplied order as metadata.

5. An inequality with any nonsquare or non-Hermitian original coefficient is assembled entry-wise and emits `pdlmi:ElementwiseInequality`. Numeric Hermitian testing accepts equality at the inclusive 10^{-10} infinity-norm tolerance.

6. Equality is direct-only. Every certificate selector raises `pdlmi:UnsupportedEqualityCertificate` before validating its supplied increment, clique size, width, or order.

Every certificate selector rebuilds the wrapper from its original residual and relation, so selecting a family replaces the previous family. Direct and Pólya terminate in coefficient constraints. The four Gram families first create a certificate specification, expand its Gram contributions, and then store the PSD constraints before the exact coefficient identities. The detailed coefficient maps are given in sections B.3, B.5, B.6 and B.8 to B.11.

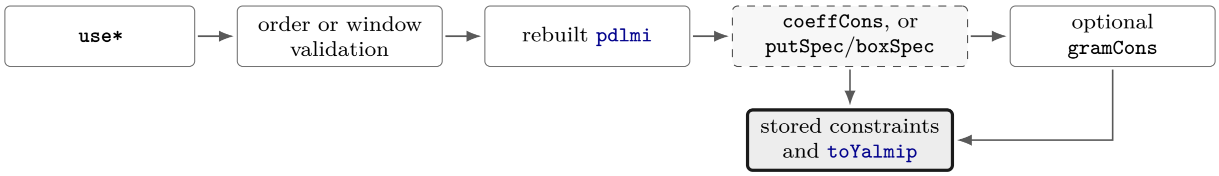


Figure 6.2: Certificate assembly call path. The direct branch stores coefficient constraints immediately, while Gram families store PSD blocks and exact coefficient identities from a validated specification.

6.4 Assemble Direct Coefficient Constraints

For a `pdvar` residual, including equality behavior, each physical cell, local Bernstein coefficient, and rate-vertex row if present creates one stored YALMIP constraint entry. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of entries is $N_c N_b$. With N_v rate vertices it is $N_c N_b N_v$. A semidefinite-mode entry is one matrix constraint. An entry-wise-mode entry is the column-major vector of scalar matrix entries compared independently. An equality entry is the corresponding direct coefficient equation and may be rectangular. For a coefficient-backed known `pdmat` inequality, Direct and Pólya assembly instead scan the complete numeric coefficient family and store exactly one logical cell containing the global certificate. `toYalmip` exports that state as one scalar logical.

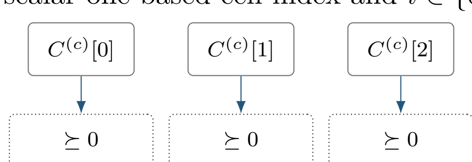
The idea is the Bernstein convex-hull property. After orienting the cell residual $\mathcal{F}^{(\boldsymbol{c})}$ as the sign-normalized positive target $S^{(\boldsymbol{c})}$, suppress the already-stacked rate-row index and write

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i} \in \prod_{s=1}^{\ell} \{0,\ldots,M_s\}} B_{\boldsymbol{i}}^{\boldsymbol{M}}(\boldsymbol{\alpha}) C^{(\boldsymbol{c})}[\boldsymbol{i}],$$

where $\boldsymbol{M}$ is the assembled residual tensor degree and $\boldsymbol{i} \in \prod_{s=1}^{\ell}\{0,\ldots,M_s\}$. The semidefinite direct certificate requires $C^{(\boldsymbol{c})}[\boldsymbol{i}] \succeq 0$ for every physical cell $\boldsymbol{c}$ and local label $\boldsymbol{i}$, repeated independently for every stored rate row. This condition is sufficient, and feasible residuals may lie outside its certificate set.

For an entry-wise inequality, replace each matrix coefficient by `C(:)` and impose the oriented scalar sign on every component. The global classification is made before Pólya elevation or Gram assembly, so every coefficient and every certificate in one wrapper uses the same mode. Putinar, SparsePutinar, SparseFullBox, and FullBox create one independent scalar Gram certificate for each column-major entry. Direct equality simply imposes `C(:) == 0` with an exact zero target and direct equations in place of a positivity certificate. Section B.5 proves the convex-hull implication and separates this sufficient coefficient test from point sampling.

The diagram specializes to one parameter, degree two, and one ordinary (non-rate) row. Therefore c is a scalar one-based cell index and $i \in \{0, 1, 2\}$.



repeat independently for each cell and rate row

Syntax

```
C = pdlmi(expr, relation)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

The constructor and comparison forms above create direct coefficient constraints when every inequality-certificate selector is false. For a `pdvar` residual, each stored coefficient is constrained in the oriented positive-semidefinite or entry-wise mode. Equality constrains each coefficient entry directly to zero. Its stored-constraint count is therefore the number of physical cells times the number of local Bernstein coefficients, with an additional factor for active rate vertices. As stated above, a coefficient-backed known `pdmat` inequality instead stores exactly one logical cell containing the global numeric certificate.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
C = P >= 0;

constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
usePutinar = C.UsePutinar
useSparsePutinar = C.UseSparsePutinar
useSparseFullBox = C.UseSparseFullBoxPreorder
useFullBox = C.UseFullBoxPreorder
```

```
constraintCount = 3
usePolya =
  logical
   0
usePutinar =
  logical
   0
useSparsePutinar =
  logical
   0
useSparseFullBox =
  logical
   0
useFullBox =
  logical
   0
```

Here the degree-two, one-parameter residual has three local Bernstein coefficients, so direct assembly stores three coefficient constraints. The wrapper remains direct because every certificate selector retains its false default. A call such as `C.usePolya(1)` returns a new wrapper with an elevated coefficient family and leaves `C` unchanged.

6.5 Select Pólya Assembly with usePolya

With `UsePolya=true`, `pdlmi` first elevates the stored residual componentwise by $\boldsymbol{d} = $ `PolyaDegree`. For a `pdvar` residual it then constrains every elevated local coefficient and stored rate row: if the assembled residual degree is $\boldsymbol{M}$, the count is $N_c \prod_{s=1}^{\ell}(M_s + d_s + 1)$ for ordinary expressions and $N_c N_v \prod_{s=1}^{\ell}(M_s + d_s + 1)$ for rate-row expressions. A coefficient-backed known `pdmat` residual instead stores exactly one logical cell containing the global numeric certificate, including after Pólya elevation. An all-zero increment is valid and has the same coefficient count as Direct. For an entry-wise `pdvar` inequality, elevation preserves the global mode and the oriented sign is applied independently to every column-major scalar entry of each elevated coefficient. Equality wrappers reject this method.

Syntax

```
Cpolya = C.usePolya()
Cpolya = C.usePolya(d)
Cpolya = C.usePolya(..., ValidationMode=mode)
```

`usePolya` is a value-class method that returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from `C.Residual`. Selecting a new increment therefore replaces a previous selection. The form with the increment omitted uses one in every direction. An invalid scalar or vector increment raises `pdlmi:InvalidPolyaDegree`.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=1);
direct = P >= 0;
polya = direct.usePolya(2);
directDegree = direct.PolyaDegree
usePolya = polya.UsePolya
polyaDegree = polya.PolyaDegree
constraintCount = numel(polya.Constraints)
```

```
directDegree = 0
usePolya =
  logical
   1
polyaDegree = 2
constraintCount = 4
```

`PolyaDegree` is a direction-wise representation increment. Decision degree, grid refinement, and Gram order are separate controls. Section B.6 gives the mathematical interpretation and fixed-increment limitations.

Anisotropic assembly output. For an independent two-parameter target with $\boldsymbol{M} = (2, 1)$, choose $\boldsymbol{d} = (1, 2)$. The selector elevates the unchanged target to $\boldsymbol{H} = \boldsymbol{M} + \boldsymbol{d} = (3, 3)$ and sends all $16 = (3 + 1)(3 + 1)$ elevated labels to Direct coefficient constraints.

$\boldsymbol{M} = (2,1)$ → $+\boldsymbol{d} = (1,2)$ exact elevation → $\boldsymbol{H} = (3,3)$ 16 coefficient LMIs

6.6 Select Putinar Assembly with `usePutinar`

Putinar is dimension-aware. One parameter uses the exact parity-specific interval form, while two or more parameters use the fixed-order box-specific singleton-generator module. In one parameter, $\boldsymbol{M} = (M_1)$ and the default and minimum order vector is $\boldsymbol{r} = (\lfloor M_1/2 \rfloor)$. In several parameters, the default and minimum are componentwise $\lceil \boldsymbol{M}/2 \rceil$. Equality wrappers reject Putinar selection.

Syntax

```
Cputinar = C.usePutinar()
Cputinar = C.usePutinar(r)
Cputinar = C.usePutinar(..., ValidationMode=mode)
```

Here `r` is scalar shorthand or a direction-wise absolute Bernstein Gram order. The form with the order omitted uses the applicable minimum, and an explicit order must satisfy the same bound. Each physical cell, active rate row, and column-major entry in entry-wise mode receives fresh Gram variables. All PSD blocks precede exact coefficient identities in the stored constraint order. Sections B.7 and B.8 gives the derivations.

Examples and output

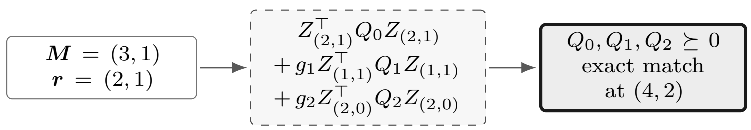
```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinar = direct.usePutinar();
directUsesPutinar = direct.UsePutinar
directOrder = direct.PutinarOrder
directConstraintCount = numel(direct.Constraints)
selectedUsesPutinar = putinar.UsePutinar
selectedOrder = putinar.PutinarOrder
selectedConstraintCount = numel(putinar.Constraints)
```

```
directUsesPutinar =
  logical
   0
directOrder = 0
directConstraintCount = 3
selectedUsesPutinar =
  logical
   1
selectedOrder = 1
selectedConstraintCount = 5
```

For this one-parameter scalar degree-two cell, order one uses the exact even Markov–Lukács form:

an unweighted Gram block of dimension two and one generator-weighted block of dimension one, followed by three exact coefficient identities. The result is identical to `direct.useFullBox()`. Each physical cell and each active rate-vertex row receives independent copies of these PSD blocks and identities.

Anisotropic assembly output. Consider a separate two-parameter target with $\boldsymbol{M} = (3, 1)$ and absolute order $\boldsymbol{r} = (2, 1)$. The Putinar specification uses the empty mask and the two singleton generators $g_s = \alpha_s(1 - \alpha_s)$, includes every nonnegative basis degree, and matches the elevated target at $2\boldsymbol{r} = (4, 2)$.



`usePutinar` is value-semantic. It rebuilds from the original `Residual` and `Relation`, replaces any previous family, uses a zero implicit margin, and leaves solver invocation to the caller.

Remarks

---

1. Malformed scalar or vector orders raise `pdlmi:InvalidPutinarOrder`. An order below $(\lfloor M_1/2 \rfloor)$ for $\boldsymbol{M} = (M_1)$, or below componentwise $\lceil \boldsymbol{M}/2 \rceil$ in two or more parameters, raises `pdlmi:PutinarOrderTooLow`.

2. A malformed selector raises `pdlmi:InvalidUsePutinar`. Explicit false plus any supplied order raises `pdlmi:ConflictingPutinarOptions`.

3. Simultaneous relaxation families raise `pdlmi:ConflictingRelaxations`. Expression, relation, and option validation in section 6.3 still applies. Global inequality classification is unchanged: the family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar Putinar certificates in MATLAB column-major order.

6.7 Select SparsePutinar with `useSpPut`

SparsePutinar applies sliding tensor windows to the active Putinar terms. Each window owns an independent PSD block, and the exact coefficient identities retain their common target degree. A smaller window can reduce the largest PSD block while increasing the number of blocks, so clique size alone provides zero universal ordering of solve time.

Syntax

```
Csp = C.useSpPut()
Csp = C.useSpPut(b)
Csp = C.useSpPut(b, r)
Csp = C.useSpPut(..., ValidationMode=mode)
```

Arguments

- `b` is one finite positive integer clique size and defaults to two. It is the common requested side length of every tensor window. Graph cliques inferred from matrix sparsity belong to a separate construction.
- `r` is scalar shorthand or an ℓ-element direction-wise absolute Putinar Gram order $\boldsymbol{r}$. It defaults to $(\lfloor M_1/2 \rfloor)$ when $\boldsymbol{M} = (M_1)$ and componentwise $\lceil \boldsymbol{M}/2 \rceil$ in two or more parameters. An explicit order must satisfy the same minimum before endpoint normalization.
- `mode` is the optional transient `"fast"` or `"strict"` validation mode. It is case-insensitive and is discarded after validation.
- `Csp` is a new `pdlmi`. An intermediate result sets `UseSparsePutinar` to true, stores r in `SparsePutinarOrder`, and stores b in `CliqueSize`. The source wrapper remains unchanged.

Description

In one parameter, `b=1` returns actual Direct state after order validation. In two or more parameters, `b=1` remains SparsePutinar. A size greater than one that spans every active basis returns actual dense Putinar state, while other valid sizes retain SparsePutinar state. Every cell, rate row, and entry-wise matrix entry receives fresh window variables, with PSD blocks stored before coefficient identities. Section B.9 gives the window construction and accumulated identities.

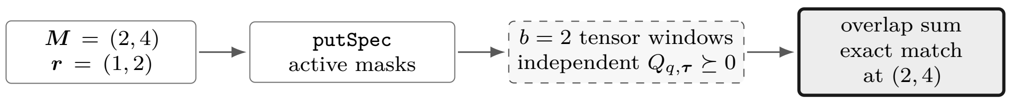
Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=4);
direct = P >= 0;
sparse = direct.useSpPut(2, 2);
directEndpoint = sparse.useSpPut(1, 2);
denseEndpoint = sparse.useSpPut(3, 2);

sparseState = [sparse.UseSparsePutinar, ...
    sparse.SparsePutinarOrder, sparse.CliqueSize, ...
    sparse.UsePutinar, numel(sparse.Constraints)]
directEndpointState = [directEndpoint.UseSparsePutinar, ...
    directEndpoint.UsePutinar, directEndpoint.CliqueSize, ...
    numel(directEndpoint.Constraints)]
denseEndpointState = [denseEndpoint.UseSparsePutinar, ...
    denseEndpoint.UsePutinar, denseEndpoint.PutinarOrder, ...
    numel(denseEndpoint.Constraints)]
```

```
sparseState = 1×5 double
     1     2     2     0     8
directEndpointState = 1×4 double
     0     0     0     5
denseEndpointState = 1×4 double
     0     1     2     7
```

The intermediate result has three independent clique-size-two PSD blocks of dimension two followed by five exact coefficient identities. The size-one request returns actual Direct state with five coefficient constraints. The size-three request covers every active Gram basis and returns dense Putinar state with PSD-block dimensions three and two followed by the same five identities.

Anisotropic assembly output. For an independent target with $\boldsymbol{M} = (2,4)$, order $\boldsymbol{r} = (1,2)$, and clique size $b = 2$, each active empty or singleton Putinar term is split into its own overlapping tensor windows. Every window receives an independent PSD block, and the accumulated contributions match the same anisotropic target degree $2\boldsymbol{r} = (2,4)$.



Remarks

---

1. `useSpPut` rebuilds from the stored `Residual` and `Relation`. It replaces any earlier Pólya, Putinar, SparsePutinar, SparseFullBox, or FullBox selection, uses a zero implicit margin, and leaves solver invocation to the caller.

2. For `>=`, coefficient identities match the residual. For `<=`, they match its negative. Every physical cell and active rate row receives independent window Gram variables and identities. Entry-wise mode adds an independent scalar certificate for each matrix entry in MATLAB column-major order.

3. Within each local certificate, all PSD window blocks precede all exact target-coefficient identities. Windows may overlap, and a basis-label pair contributes once through each independent window that contains it.

4. Invalid clique sizes raise `pdlmi:InvalidCliqueSize`. Malformed orders raise `pdlmi:InvalidSparsePutinarOrder`, while orders below the dimension-dependent minimum raise `pdlmi:SparsePutinarOrderTooLow`. More than two positional inputs raise `pdlmi:InvalidApplyOptions`.

5. A malformed selector raises `pdlmi:InvalidUseSparsePutinar`. Explicit false with `CliqueSize` or `SparsePutinarOrder` raises `pdlmi:ConflictingSparsePutinarOptions`. Co-selection with another family raises `pdlmi:ConflictingRelaxations`. Equality raises `pdlmi:UnsupportedEqualityCertificate` before clique-size or order validation.

6. The tensor windows group Bernstein Gram-basis labels. Sparsity-graph inference, chordal completion, and the structural-matrix chordal decomposition of Zheng and Fantuzzi [17] are separate constructions.

See also. `usePutinar`, `useSpBox`, `useFullBox`, `toYalmip`, and SparsePutinar mathematics.

6.8 Select SparseFullBox with `useSpBox`

SparseFullBox applies sliding tensor windows to the FullBox parity or generator-mask terms. The represented residual, absolute order, relation orientation, and target coefficient degree remain unchanged.

Syntax

```
Csp = C.useSpBox()
Csp = C.useSpBox(w)
Csp = C.useSpBox(w, r)
Csp = C.useSpBox(..., ValidationMode=mode)
```

Arguments

`w` is one finite positive integer tensor-window side length and defaults to two. The selected state retains the property name `BandWidth`, while w selects tensor windows directly and leaves flattened matrix indices irrelevant. `r` is scalar shorthand for the optional direction-wise absolute SparseFullBox Gram order $\boldsymbol{r}$. It defaults to $(\lfloor M_1/2 \rfloor)$ when $\boldsymbol{M} = (M_1)$, or componentwise $\lceil \boldsymbol{M}/2 \rceil$ for two or more parameters. Every explicit order is validated against that same componentwise minimum before endpoint normalization.

Description

Width one canonicalizes to actual Direct state. A width that spans every order axis canonicalizes to actual FullBox state, while intermediate widths retain SparseFullBox state. Every cell, rate row, and entry-wise matrix entry receives fresh window variables, with PSD blocks stored before coefficient identities. Section B.11 derives the window construction, block dimensions, accumulated identities, and endpoints.

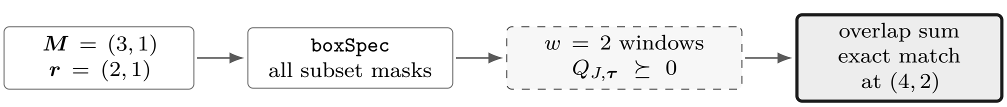
Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=4);
direct = P >= 0;
sparse = direct.useSpBox(2, 2);
directEndpoint = direct.useSpBox(1, 2);
denseEndpoint = direct.useSpBox(3, 2);
sparseState = [sparse.UseSparseFullBoxPreorder, ...
    sparse.SparseFullBoxOrder, sparse.BandWidth, ...
    sparse.UseFullBoxPreorder, numel(sparse.Constraints)]
directEndpointState = [directEndpoint.UseSparseFullBoxPreorder, ...
    directEndpoint.UseFullBoxPreorder, directEndpoint.BandWidth, ...
    numel(directEndpoint.Constraints)]
denseEndpointState = [denseEndpoint.UseSparseFullBoxPreorder, ...
    denseEndpoint.UseFullBoxPreorder, denseEndpoint.FullBoxOrder, ...
    numel(denseEndpoint.Constraints)]
```

```
sparseState = 1×5 double
     1     2     2     0     8
directEndpointState = 1×4 double
     0     0     0     5
denseEndpointState = 1×4 double
     0     1     2     7
```

The intermediate result retains SparseFullBox state with $r = 2$ and $w = 2$. Its three overlapping size-two windows own independent PSD blocks, and all five target coefficients are matched exactly. Width one returns actual Direct state with empty sparse metadata and zero Gram variables. Since $w = 3 = r + 1$, the dense endpoint returns actual FullBox state with two dense PSD blocks followed by the same five identities. These canonical public states expose the selector that owns each constraint list.

Anisotropic assembly output. For a separate target with $\boldsymbol{M} = (3, 1)$, choose $\boldsymbol{r} = (2, 1)$ and window side length $w = 2$. The FullBox subset terms use different direction-wise Gram degrees, each term is split into its own tensor windows, and the accumulated window contributions match the elevated target at $(4, 2)$.



Remarks

1. `useSpBox` is a value-class method: it returns a new wrapper, leaves the source unchanged, and rebuilds from `Residual` and `Relation`, replacing any earlier Pólya, Putinar, SparsePutinar, SparseFullBox, or FullBox selection. It uses a zero implicit positivity margin and leaves solver invocation to the caller.

2. Every physical cell and active rate row owns independent window Gram matrices and coefficient identities. Semidefinite mode builds matrix Gram terms, while entry-wise mode builds an

independent scalar certificate for every matrix entry in MATLAB column-major order.

3. Width one canonicalizes to actual Direct state. A width satisfying `w >= max(r + 1)` spans every order axis and canonicalizes to actual FullBox state. For $1 < w < \max_s(r_s + 1)$, the result records `UseSparseFullBoxPreorder=true`, `SparseFullBoxOrder=r`, and `BandWidth=w`.

4. Invalid widths raise `pdlmi:InvalidBandWidth`. Malformed orders raise `pdlmi:InvalidSparseFullBoxOrder`. Orders below the dimension-dependent minimum raise `pdlmi:SparseFullBoxOrderTooLow`. More than two positional inputs raise `pdlmi:InvalidApplyOptions`.

5. Constructor selector conflicts use `pdlmi:ConflictingSparseFullBoxOptions` or `pdlmi:ConflictingRelaxations`. Equality raises `pdlmi:UnsupportedEqualityCertificate` before width or order validation.

6.9 Select FullBox with `useFullBox`

FullBox selects the box-specific dense Bernstein–Gram preordering. General-domain SOS parsing belongs to a separate construction. FullBox uses a zero implicit margin and rejects equality wrappers. Every column-major scalar entry in entry-wise mode receives an independent certificate.

Syntax

```
Cbox = C.useFullBox()
Cbox = C.useFullBox(r)
Cbox = C.useFullBox(..., ValidationMode=mode)
```

The form with the order omitted selects the smallest admissible absolute order. The explicit form accepts scalar shorthand or a direction-wise `r`, which remains separate from the residual degree. Both forms are value-class selectors that return a new `pdlmi`, leave `C` unchanged, and rebuild from `C.Residual`. Reapplication therefore replaces an earlier FullBox order or a Pólya, Putinar, SparsePutinar, or SparseFullBox selection.

In one parameter, $\boldsymbol{M} = (M_1)$ and the minimum absolute order vector is $\boldsymbol{r} = (\lfloor M_1/2 \rfloor)$. In two or more parameters, the componentwise minimum is $\lceil \boldsymbol{M}/2 \rceil$. The selected order fixes dense Gram bases and remains separate from the residual degree. Sections B.7 and B.10 gives the parity and multivariate generator constructions.

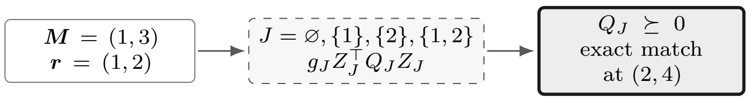
Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
box = direct.useFullBox();
directConstraintCount = numel(direct.Constraints)
useFullBox = box.UseFullBoxPreorder
fullBoxOrder = box.FullBoxOrder
fullBoxConstraintCount = numel(box.Constraints)
```

```
directConstraintCount = 3
useFullBox =
  logical
   1
fullBoxOrder = 1
fullBoxConstraintCount = 5
```

At this order the scalar degree-two cell has two PSD Gram constraints followed by three exact Bernstein coefficient identities.

Anisotropic assembly output. For an independent target with $\boldsymbol{M} = (1, 3)$ and absolute order $\boldsymbol{r} = (1, 2)$, FullBox uses the four masks $\varnothing$, $\{1\}$, $\{2\}$, and $\{1, 2\}$. Each active mask has its own PSD Gram block, and their weighted coefficient maps match the target after exact elevation to $2\boldsymbol{r} = (2, 4)$.



Remarks

FullBox retains all box-generator products, which distinguishes it from the multivariate singleton-generator Putinar family. Assembly is independent for every cell, active rate row, and entry-wise matrix entry. For `>=`, it represents the residual, while for `<=`, it represents the negated target. All PSD blocks precede exact coefficient identities in stored order.

1. Malformed orders raise `pdlmi:InvalidFullBoxOrder`. An admissible integer below the minimum raises `pdlmi:FullBoxOrderTooLow`, and the constructor rejects malformed selectors with `pdlmi:InvalidUseFullBoxPreorder`.
2. Simultaneous Pólya, Putinar, SparsePutinar, SparseFullBox, or FullBox selection raises `pdlmi:ConflictingRelaxations`. Expression, relation, and option validation in section 6.3 still applies.
3. Global inequality classification is unchanged. The family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar FullBox certificates in MATLAB column-major order.

6.10 Export with `toYalmip`

Syntax

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments

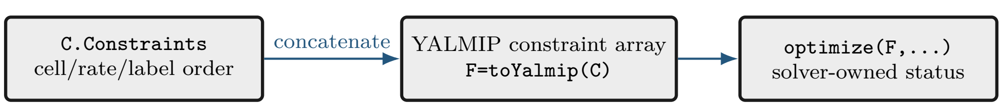
For a `pdvar` residual, `toYalmip` concatenates the stored constraint cells into a YALMIP constraint array for Direct, Pólya, Putinar, SparsePutinar, SparseFullBox, and FullBox objects. For a coefficient-backed known `pdmat` residual, Direct and Pólya instead return the one stored scalar logical certificate. Its numeric check uses the wrapper's global inequality classification and inclusive absolute tolerance 10^{-10}. Semidefinite mode symmetrizes each coefficient matrix as $(C + C^{\top})/2$ and tests its eigenvalues, while element-wise mode tests individual scalar entries. A false result emits warning `pdlmi:InconclusiveCertificate`. Failure means this sufficient coefficient certificate is inconclusive about violation or indefiniteness of the continuous function. Known-data Putinar, SparsePutinar, SparseFullBox, and FullBox objects still export YALMIP constraints and may contain auxiliary Gram variables. The method accepts zero package-specific options and returns the assembled constraint object. Solver wrappers, objectives, margins, and diagnostic layers belong to the caller's YALMIP workflow.

Examples and output

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
wrapperClass = class(C)
storedConstraintCount = numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
firstIsConstraint = isa(F1, "lmi") || isa(F1, "constraint")
firstConstraintCount = length(F1)
secondIsConstraint = isa(F2, "lmi") || isa(F2, "constraint")
secondConstraintCount = length(F2)
```

```
wrapperClass = 'pdlmi'
storedConstraintCount = 2
firstIsConstraint =
  logical
   1
firstConstraintCount = 2
secondIsConstraint =
  logical
   1
secondConstraintCount = 2
```



After this boundary, Bernstein assembly is complete. YALMIP owns solver choice, diagnostics, and optimization status.

Examples and output

Known coefficient evidence can be certified through a direct numeric check on the Direct and Pólya paths.

```
yalmip('clear')
A = pdmat([0 1], {{1, 2; 3, 4}}, ...
    Degree=1, RateBounds=[-1 2]);
direct = A >= 0;
directCertificate = toYalmip(direct)
```

```
polya = direct.usePolya(1);
polyaCertificate = toYalmip(polya)
```

```
directCertificate = logical
   1

polyaCertificate = logical
   1
```

6.11 Boundaries and Related APIs

`pdvar`, `rhodiff`, `usePolya`, `usePutinar`, `useSpPut`, `useSpBox`, `useFullBox`, `toYalmip`, and `optimize`.

6.11.1 Solve with YALMIP

Solver settings and the call to `optimize` belong to the caller's YALMIP workflow. After `toYalmip`, validate the returned status before retaining an objective or recovered coefficient data:

```
opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
objectiveValue = value(objective);
```

An optimized certificate value requires `problem == 0`. Numerical, unknown, aborted, memory, and space-exhaustion statuses are inconclusive about both the original continuous problem and the value of γ.

Chapter 7

Shared Helper Utilities

The `+helper` namespace contains thirteen publicly callable, developer-facing utilities whose ownership crosses class boundaries. This chapter covers `bernConvRatios`, `bernConvWeights`, `cellGet`, `chk`, `chkCont`, `combRows`, `fitVals`, `isZero`, `mkGrid`, `mkNest`, `normDeg`, `normMode`, and `rateVerts`. Ordinary modeling code should enter through `pdbase`, `pdmat`, `pdvar`, and `pdlmi`, while local subfunctions and protected class kernels remain implementation details.

7.1 Bernstein Convolution Utilities

`helper.bernConvWeights` returns normalized tensor binomial weights, and `helper.bernConvRatios` returns the row-aligned ratios used by Bernstein products, elevation, and Gram maps.

Syntax

```
weights = helper.bernConvWeights(labels, degree)
ratios = helper.bernConvRatios(lhsLabels, lhsDegree, ...
    rhsLabels, rhsDegree)
ratios = helper.bernConvRatios(lhsLabels, lhsDegree, ...
    rhsLabels, rhsDegree, outputLabels, outputDegree)
```

Arguments

`labels` and each label argument are finite nonnegative integer tables with one row per requested factor and one column per parameter. Each `degree` argument is a finite nonnegative integer row with the same parameter count, and every label must lie inside its degree box. The four-input ratio form sets `outputLabels=lhsLabels+rhsLabels` and `outputDegree=lhsDegree+rhsDegree`. The six-input form accepts explicit shifted outputs, as required by generator-weighted Gram terms. `weights` and `ratios` are numeric column vectors aligned with the input rows. Invalid weight inputs raise `helper:InvalidBernConvWeights`, while invalid ratio arity, shape, labels, or degrees raise `helper:InvalidBernConvRatios`.

Examples and output

```
weights = helper.bernConvWeights((0:2).', 2)
ratios = helper.bernConvRatios([0; 1], 1, [0; 0], 1)
```

```
weights = 3×1 double
    0.2500
    0.5000
    0.2500

ratios = 2×1 double
    1.0000
    0.5000
```

See also. Sections A.6, A.8 and B.3 derive the operations that use these ratios.

7.2 helper.cellGet

Purpose. `helper.cellGet` reads one flat coefficient leaf from a nested physical-cell tree.

Syntax

```
leaf = helper.cellGet(vals, subs)
```

Description

`vals` is addressed as `vals{i1}{i2}...{i_ell}` and `subs` supplies one index per level. The returned `leaf` is the selected flat coefficient cell or rate-row table. Selection preserves the stored payload, shape, and index interpretation.

Examples and output

```
vals = {{{10, 20}}, {{30, 40}}};
leaf = cell2mat(helper.cellGet(vals, [2 1]))
```

```
leaf = 1×2 double
    30    40
```

Remarks

The helper deliberately relies on ordinary MATLAB cell indexing. Callers validate the subscript length and bounds before traversal.

7.3 `helper.chk`

Purpose. Apply shared validation predicates while retaining a caller-owned error identifier and message.

Syntax

```
val = helper.chk(val, errId, msg, tags...)
val = helper.chk(val, errId, msg, tags..., Name, Value)
```

Description

Predicate tags are `numeric`, `real`, `cell`, `struct`, `nonempty`, `scalar`, `vector`, `matrix`, `finite`, `integer`, `positive`, `nonnegative`, `increasing`, and `rowbounds`. Named tests are `Size`, `Numel`, `MinNumel`, `Min`, and `Max`. Successful validation returns `val` unchanged.

Examples and output

```
validated = helper.chk([1 2], "demo:BadValue", "bad value", ...
    "numeric", "real", "finite", "Size", [1 2])
```

```
validated = 1×2 double
     1     2
```

Remarks

A failed data predicate raises `errId`. An unknown predicate or a named test that omits its value raises `helper:InvalidValidatorCall`. These errors identify a validator-programming defect, while `errId` identifies rejected caller data.

7.4 `helper.combRows`

Purpose. Enumerate Cartesian combinations in the ordering shared by physical cells, local labels, and rate vertices.

Syntax

```
rows = helper.combRows(vecs)
```

Description

`vecs` is a cell array containing one vector per tensor axis. `rows` contains every Cartesian combination, with earlier dimensions varying more slowly.

Examples and output

```
rows = helper.combRows({0:1, 10:11})
```

```
rows = 4×2 double
     0    10
     0    11
     1    10
     1    11
```

Remarks

This ordering is part of the storage contract. Reordering the rows would misalign tensor labels, physical-cell traversal, and derivative rate vertices.

7.5 `helper.isZero`

Purpose. Prove the distinct zero notions used by known-data and affine algebra fast paths.

Syntax

```
tf = helper.isZero(val, "num")
tf = helper.isZero(val, "add", matrixSize)
tf = helper.isZero(vals, "vals")
tf = helper.isZero(obj, "obj")
```

Description

`"num"` requires a nonempty finite real numeric zero. `"add"` additionally enforces scalar expansion or the supplied matrix shape. `"vals"` recursively checks nested, symbolic, and rate-structured payloads. And `"obj"` accepts coefficient-backed `pdmat` or `pdvar` evidence.

Examples and output

```
numericZero = helper.isZero(zeros(2), "add", [2 2])
A = pdmat({[0 1]}, {0, 0}, Degree=1);
objectZero = helper.isZero(A, "obj")
```

```
numericZero =
  logical
   1
objectZero =
  logical
   1
```

Remarks

Function-only `pdmat` placeholders contain handle evaluation exclusively, so their zero status remains inconclusive. Invalid modes raise `helper:InvalidZeroMode`. The wrong mode-specific arity raises `helper:InvalidZeroCall`.

7.6 `helper.mkGrid`

Purpose. Validate tensor-grid vectors and construct the shared `GridInfo` record.

Syntax

```
info = helper.mkGrid(grid)
info = helper.mkGrid(grid, owner)
```

Description

`grid` is a nonempty cell array of finite, real, strictly increasing vectors with at least two nodes each. `owner` defaults to `"pdbase"` and prefixes validation identifiers. The result contains `Vectors`, Cartesian-product `Points`, per-axis `Bounds`, and `NumNodes`.

Examples and output

```
info = helper.mkGrid({[0 1], [10 20]}, "demo");
bounds = info.Bounds
points = info.Points
```

```
bounds = 2×2 double
     0     1
    10    20

points = 4×2 double
     0    10
     0    20
     1    10
     1    20
```

Remarks

Malformed containers use `owner + ":InvalidGrid"`. Malformed individual axes use `owner + ":InvalidGridVector"`. Consumers derive cell counts directly from `GridInfo.NumNodes`.

7.7 `helper.mkNest`

Purpose. Allocate recursive physical-cell storage and construct each leaf from its complete cell subscript.

Syntax

```
vals = helper.mkNest(nCell, mkLeaf)
vals = helper.mkNest(nCell, mkLeaf, prefix)
```

Description

`nCell` gives the positive cell count per parameter direction and `mkLeaf` receives one completed one-based subscript row. `prefix` is recursion state used internally. Ordinary callers omit it. The result is addressed as `vals{i1}{i2}...{i_ell}`.

Examples and output

```
vals = helper.mkNest([2 1], @(subs) {sum(subs)});
secondLeaf = cell2mat(helper.cellGet(vals, [2 1]))
```

```
secondLeaf = 3
```

Remarks

`mkLeaf` owns the leaf payload. The helper supplies deterministic physical-cell subscripts, while the caller supplies coefficient count, matrix size, and rate semantics.

7.8 `helper.normDeg`

Purpose. Validate scalar shorthand or direction-wise Bernstein degrees while preserving the caller's error namespace.

Syntax

```
degree = helper.normDeg(value, nPar, errId, label)
```

Description

`value` must be one finite nonnegative integer scalar or an `nPar`-element vector. A scalar expands uniformly, a column vector is reshaped, and every accepted result is a `1`-by-`nPar` double row. `errId` is raised for empty, nonnumeric, complex, nonfinite, noninteger, negative, or incorrectly sized input. `label` supplies the public option name in the error message and defaults to `"Degree"`.

Examples and output

```
degree = helper.normDeg([0; 2; 1], 3, ...
    "demo:InvalidDegree", "Degree")
```

```
degree = 1×3 double
     0     2     1
```

Remarks

The helper performs normalization only. Constructors and certificate selectors own user-facing warnings, minimum-order checks, and their class-specific error identifiers.

7.9 Continuity Classification and Coefficient Fitting

7.9.1 `helper.chkCont`

Purpose. Classify whether stored coefficients agree on every shared physical-cell face.

Syntax

```
tf = helper.chkCont(vals, nCell, degree)
```

Description

`vals` is a normalized nested coefficient tree, while `nCell` and `degree` are direction-wise row vectors. The logical scalar `tf` is true only when every row of each neighboring leaf has matching boundary coefficients on every shared face. Numeric faces use a scale-aware tolerance, while affine YALMIP faces require identical coefficient bases. The helper classifies existing storage and preserves every coefficient. It expects the normalized internal storage contract, so malformed trees, row counts, or payloads use the underlying indexing or matrix-operation error namespace.

Examples and output

```
vals = {{0, 1}, {1, 2}};
isContinuous = helper.chkCont(vals, 2, 1)
```

```
isContinuous =
  logical
   1
```

See also. `pdbase.IsContinuous`, the `pdmat` constructor in section 4.2, and the shared-face convention in section A.7.

7.9.2 helper.fitVals

Purpose. Fit cell-wise Bernstein coefficients from values sampled at the local tensor Bernstein nodes.

Syntax

```
vals = helper.fitVals(info, degree, matrixSize, evalFcn, owner)
[vals, labels] = helper.fitVals(info, degree, matrixSize, evalFcn, owner)
```

Description

`info` is normalized grid metadata from `helper.mkGrid`, `degree` is scalar shorthand or one value per parameter, and `matrixSize` is the common sampled matrix size. The function `evalFcn` receives one physical parameter point as a row vector, and `owner` supplies the error-identifier stem for degree validation. The result `vals` is a nested physical-cell tree in `helper.combRows` label order, while `labels` is the common local label table. Invalid degree input raises `owner + ":InvalidDegree"`. Errors from `evalFcn` propagate to the caller, and this developer utility assumes that `info`, `matrixSize`, and sampled payload sizes were already normalized by the owning constructor.

Examples and output

```
info = helper.mkGrid({[0 1]}, "demo");
[vals, labels] = helper.fitVals(info, 2, [1 1], ...
    @(rho) 1 + rho(1)^2, "demo");
localLabels = labels
localCoefficients = cell2mat(vals{1})
```

```
localLabels = 3×1 double
     0
     1
     2

localCoefficients = 1×3 double
     1     1     2
```

See also. The `pdmat` constructor in section 4.2 and the power-to-Bernstein conversion in section A.4.

7.10 Validation Modes and Rate Vertices

7.10.1 helper.normMode

Purpose. Normalize a transient validation mode while preserving the caller's error namespace.

Syntax

```
mode = helper.normMode(value, owner)
```

Description

`value` must be scalar text equal to `"fast"` or `"strict"`, case-insensitively, and `owner` is the class or package stem used in diagnostics. The returned `mode` is the lowercase string scalar `"fast"` or `"strict"`. Invalid or missing text raises `owner + ":InvalidValidationMode"`. The result controls operation-local consistency checks and remains transient call state.

Examples and output

```
mode = helper.normMode("Strict", "pdlmi")
```

```
mode = "strict"
```

See also. `ValidationMode` in sections 3.2, 4.2 and 5.2.

7.10.2 `helper.rateVerts`

Purpose. Enumerate the distinct vertices of a normalized parameter-rate box.

Syntax

```
vertices = helper.rateVerts(rateBounds)
```

Description

`rateBounds` is an ℓ-by-2 matrix that has already passed the package rate-bound checks. Each varying direction contributes its lower value and then its upper value, while a direction with equal bounds contributes one value. The rows of `vertices` follow `helper.combRows` order, and empty bounds return a zero-row matrix. This helper performs enumeration, while constructors and `rhodiff` own public input validation and malformed-bound errors.

Examples and output

```
vertices = helper.rateVerts([-1 2; 3 3])
```

```
vertices = 2×2 double
    -1     3
     2     3
```

See also. `helper.combRows`, `pdbase.NumRateRows`, and the rate-row storage derivation in section A.9.1.

Chapter 8

Worked Solver Examples

8.1 Deterministic Putinar, SparsePutinar, SparseFullBox, And FullBox Selection

This example selects default Putinar and FullBox certificates, an explicit FullBox order, one intermediate SparsePutinar certificate, and the three canonical SparseFullBox outcomes before any optimization solve. The reported values follow the public assembly contract and remain independent of incidental YALMIP decision-variable identifiers.

```
yalmip('clear')
Pa = pdvar(1, {[0 1], [10 20]}, Degree=[2 0]);
directA = Pa >= 0;
polyaA = directA.usePolya([0 1]);
decisionDegree = Pa.Degree
polyaIncrement = polyaA.PolyaDegree
elevatedDegree = polyaA.Residual.Degree + polyaA.PolyaDegree
constraintCount = numel(polyaA.Constraints)
```

```
decisionDegree = 1×2 double
     2     0

polyaIncrement = 1×2 double
     0     1

elevatedDegree = 1×2 double
     2     1

constraintCount = 6
```

This anisotropic example has three original labels and six labels after direction-wise elevation: $(2+1)(0+1) = 3$ and $(2+1)(1+1) = 6$. The zero second component of `Pa.Degree` makes the decision constant in the second parameter. `PolyaDegree=[0 1]` changes only its certificate representation.

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinarDefault = direct.usePutinar();
boxDefault = direct.useFullBox();
boxHigher = boxDefault.useFullBox(2);
polya = direct.usePolya(1);
boxFromPolya = polya.useFullBox();
putinarFromBox = boxHigher.usePutinar();
Fbox = toYalmip(boxDefault);

directState = [direct.UseFullBoxPreorder, direct.FullBoxOrder, ...
    numel(direct.Constraints)]
```

```
defaultState = [boxDefault.UseFullBoxPreorder, boxDefault.FullBoxOrder, ...
    numel(boxDefault.Constraints)]
putinarState = [putinarDefault.UsePutinar, putinarDefault.PutinarOrder, ...
    numel(putinarDefault.Constraints)]
higherState = [boxHigher.FullBoxOrder, numel(boxHigher.Constraints)]
replacementState = [boxFromPolya.UsePolya, ...
    boxFromPolya.UseFullBoxPreorder, boxFromPolya.FullBoxOrder]
putinarReplacementState = [putinarFromBox.UsePutinar, ...
    putinarFromBox.UseFullBoxPreorder, putinarFromBox.PutinarOrder]
exportedConstraintCount = length(Fbox)
```

```
directState = 1×3 double
     0     0     3
defaultState = 1×3 double
     1     1     5
putinarState = 1×3 double
     1     1     5
higherState = 1×2 double
     2     7
replacementState = 1×3 double
     0     1     1
putinarReplacementState = 1×3 double
     1     0     1
exportedConstraintCount = 5
```

For this one-dimensional degree-two scalar residual, order one uses two PSD Gram blocks followed by three exact coefficient identities. Order two matches a degree-four target and stores two PSD blocks followed by five identities. The original `direct` object remains unchanged, and the selection made from `polya` replaces Pólya by rebuilding the FullBox constraints from the original residual. Likewise, selecting Putinar from `boxHigher` rebuilds from the original residual and clears the full-box selection.

The next degree-four residual makes clique size two an intermediate SparsePutinar certificate. It also makes width two an intermediate SparseFullBox certificate, while SparseFullBox widths one and three expose its canonical endpoints.

```
yalmip('clear')
P4 = pdvar(1, {[0 1]}, Degree=4);
direct4 = P4 >= 0;
sparsePutinar4 = direct4.useSpPut(2, 2);
sparse4 = direct4.useSpBox(2, 2);
direct4Endpoint = sparse4.useSpBox(1, 2);
full4Endpoint = sparse4.useSpBox(3, 2);
sparsePutinar4State = [sparsePutinar4.UseSparsePutinar, ...
    sparsePutinar4.SparsePutinarOrder, sparsePutinar4.CliqueSize, ...
    numel(sparsePutinar4.Constraints)]
sparse4State = [sparse4.UseSparseFullBoxPreorder, ...
    sparse4.SparseFullBoxOrder, sparse4.BandWidth, ...
    numel(sparse4.Constraints)]
direct4State = [direct4Endpoint.UseSparseFullBoxPreorder, ...
    direct4Endpoint.UseFullBoxPreorder, ...
    numel(direct4Endpoint.Constraints)]
full4State = [full4Endpoint.UseSparseFullBoxPreorder, ...
    full4Endpoint.UseFullBoxPreorder, full4Endpoint.FullBoxOrder, ...
    numel(full4Endpoint.Constraints)]
```

```
sparsePutinar4State = 1×4 double
     1     2     2     8
sparse4State = 1×4 double
     1     2     2     8
direct4State = 1×3 double
     0     0     5
full4State = 1×4 double
     0     1     2     7
```

The intermediate SparsePutinar wrapper stores three independent clique-size-two PSD blocks followed by five identities. The intermediate SparseFullBox wrapper also stores three PSD constraints and five identities, but both sparse families use their own sliding tensor-window basis construction. SparseFullBox width one rebuilds actual Direct state with five coefficient constraints. Width three spans the order-two tensor basis and rebuilds actual FullBox state with two dense PSD blocks followed by the five identities. Every selector call is value-semantic and starts from the stored original residual, so the source wrapper and any earlier selection remain unchanged.

8.2 Solve for a Parameter-Dependent Lyapunov Matrix

The first solver smoke test in `+tests/+pdlmi/test_solver_smoke.m` contrasts a constant Lyapunov matrix with a degree-one, parameter-dependent one. It solves the same decay and positivity constraints in both cases. The degree-one solution is then materialized as known `pdmat` data and plotted. `pdvar` owns symbolic decision expressions, while the valued `pdmat` owns `plot`. Applying `value` to the local YALMIP payloads after a successful solve supplies the numeric Bernstein coefficients for `pdmat`.

```
yalmip('clear')
grid = {[0 1]};
A = pdmat(grid, @(rho) (1 - rho) * [-1 -1; 1 -1] + ...
    rho * [-1 -10; 0.1 -1], Degree=1);
solver = tests.select_sdp_solver();
opts = sdpsettings('solver', solver, 'verbose', 0);

Pc = pdvar(2, grid, Degree=0);
CconstantDecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
CconstantPositive = Pc >= eye(2);
solConstant = optimize([CconstantDecay.toYalmip, ...
    CconstantPositive.toYalmip], [], opts);
assert(solConstant.problem == 1, solConstant.info)

Pd = pdvar(2, grid);
CdependentDecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
CdependentPositive = Pd >= eye(2);
solDependent = optimize([CdependentDecay.toYalmip, ...
    CdependentPositive.toYalmip], [], opts);
assert(solDependent.problem == 0, solDependent.info)

Pplot = value(Pd);
figure(Name="Parameter-dependent Lyapunov matrix")
plot(Pplot, SamplesPerCell=40, LineWidth=1.5)
title("Solved parameter-dependent Lyapunov matrix")
```

The figure contains one curve for each entry of the solved 2-by-2 matrix $P(\rho)$, sampled on the parameter interval. It is a diagnostic visualization of the feasible solution, while the coefficient-wise constraints supplied to the solver remain the certificate. The constant-case result is retained to exercise the comparison in the smoke test. A MOSEK solve certifies its intended infeasibility distinction.

8.3 Solve a Rate-Dependent Block LMI

This example is adapted from `+tests/+pdlmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \qquad P(\rho) \succeq 0.$$

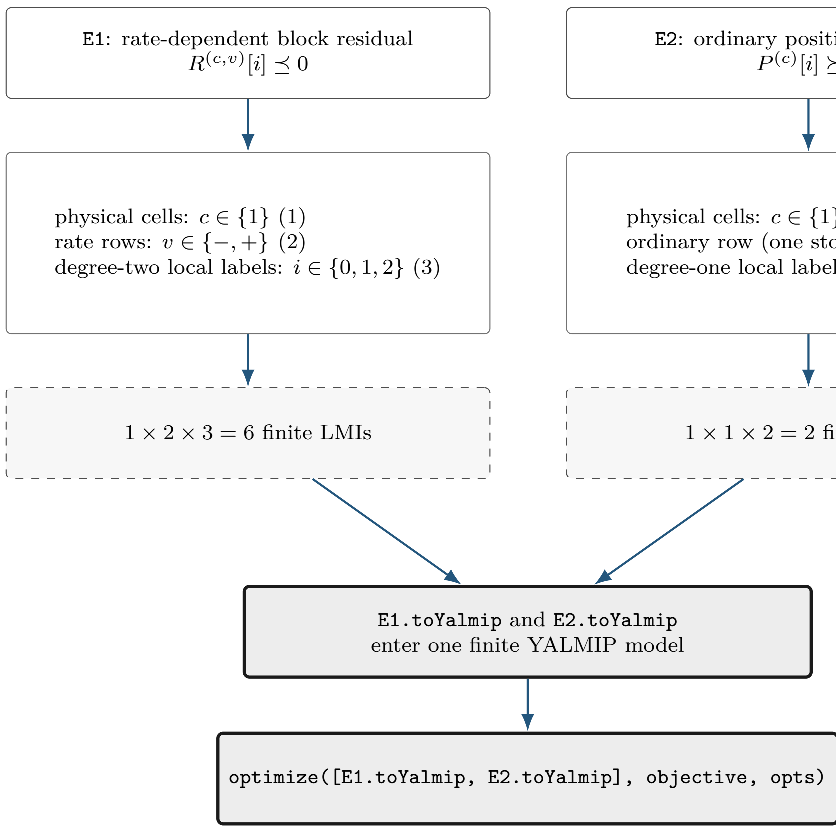
The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `pdlmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP. The second listing assigns the solver- and tolerance-dependent numeric value of γ to `gammaValue`. For this model, `numel(E1.Constraints)` returns 6, and `numel(E2.Constraints)` returns 2.

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = pdmat(grid, @(rho) [-1, 0.5; -1, -2] + ...
    rho * [-1.3, -20; 2, -10], Degree=1);
B = pdmat(grid, @(rho) [1, -4; -1, -1] + ...
    rho * [2.2, 0.5; -6, -5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = pdvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = pdvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
      B' * P, -gamma * eye(2), Dmat';
      Cmat, Dmat, -gamma * eye(2)] <= 0;
E2 = P >= 0;
decayConstraintCount = numel(E1.Constraints)
positivityConstraintCount = numel(E2.Constraints)
```

```
decayConstraintCount = 6
positivityConstraintCount = 2
```

```
objective = gamma.LocalValues{1}{1};
solver = tests.select_sdp_solver();
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
gammaValue = value(objective);
assert(isfinite(gammaValue))
gammaValue
```

The constraint counts above follow directly from the three independent assembly axes: physical-cell identity, rate-row identity, and local Bernstein label. In this tested example the two wrappers use the same physical grid but different rate-row and degree structures.



The numbers (6) and (2) are certificate-assembly counts that state how many finite semidefinite constraints represent `E1` and `E2`. Solver success requires the separate `sol.problem` check, and the finite objective is checked after the shared `optimize` call.

Appendix A

Bernstein Representation and Exact Operations

This appendix develops the representation used by the package from the one-dimensional degree-one case to cell-wise tensor polynomials. The reader needs only scalar polynomials, matrices, and elementary calculus. Every conversion, elevation, product, derivative, and subdivision below is exact. Sampling approximation is a separate operation. Farouki gives a broader historical and numerical account [3].

Each main construction follows the same reading path: *Basic idea*, *Formula*, *Worked example*, and *Visual conclusion*. The extra staging is intentional: first identify what the operation means, then read its indices, and only afterward connect it to package storage.

A.1 Convex Combinations Before Polynomials

Basic idea. A convex combination of numbers or matrices has the form

$$X = \sum_{i=0}^{q} w_i X_i, \qquad w_i \geq 0, \qquad \sum_{i=0}^{q} w_i = 1.$$

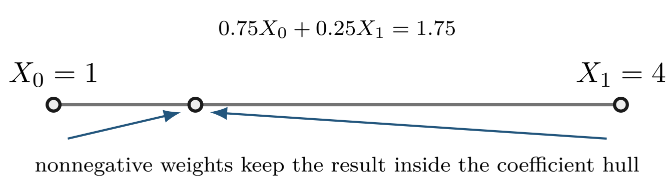
For scalars, X lies between the smallest and largest X_i. For symmetric matrices, if every $X_i \succeq 0$, then $X \succeq 0$, because

$$u^\top X u = \sum_{i=0}^{q} w_i \, u^\top X_i u \geq 0 \quad \text{for every } u.$$

Formula. Bernstein bases are useful because their basis values are precisely such weights at every point of a cell.

Worked example. Take $X_0 = 1$, $X_1 = 4$, and weights 0.75 and 0.25. Their combination is 1.75, which remains between the two inputs.

Visual conclusion.



The implication is one-way. A polynomial may be positive everywhere while one Bernstein coefficient is negative. Appendix B uses that fact to motivate stronger certificates.

A.2 Degree-One Interpolation On One Physical Cell

Basic idea. Let $\mathcal{H}_c = [\rho_1^{(c)}, \rho_1^{(c+1)}]$, with width $h_1^c = \rho_1^{(c+1)} - \rho_1^{(c)} > 0$. Its forward map and inverse coordinate are

$$\phi_c(\alpha) = \rho_1^{(c)} + h_1^c \alpha, \qquad \alpha = \phi_c^{-1}(\rho_1) = \frac{\rho_1 - \rho_1^{(c)}}{h_1^c} \in [0,1].$$

Formula. The degree-one Bernstein basis is

$$B_0^1(\alpha) = 1 - \alpha, \qquad B_1^1(\alpha) = \alpha.$$

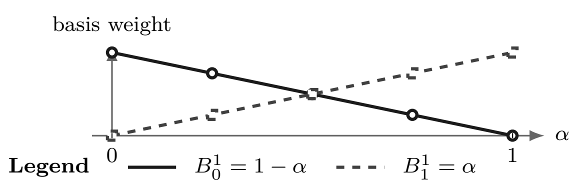
Therefore

$$C^{(c)}(\alpha) = (1-\alpha)C^{(c)}[0] + \alpha C^{(c)}[1].$$

At the lower physical face, $\alpha = 0$ and the value is $C^{(c)}[0]$. At the upper face, $\alpha = 1$ and the value is $C^{(c)}[1]$. Between them the matrix follows the straight segment joining those two coefficient matrices.

Worked example. For example, $C^{(c)}[0] = 2$ and $C^{(c)}[1] = 4$ give $C^{(c)}(0.25) = 0.75(2) + 0.25(4) = 2.5$, exactly as in section 4.5.

Visual conclusion.



A.3 Higher Degree And The Partition Of Unity

Basic idea. Higher degree adds interior shape controls while keeping the same physical cell and the same physical grid nodes.

Formula. In one parameter, write the degree vector as $\boldsymbol{m} = (m_1)$ and define

$$B_i^{m_1}(\alpha) = \binom{m_1}{i}(1-\alpha)^{m_1 - i}\alpha^i, \qquad i = 0, \dots, m_1.$$

Each basis function is nonnegative on $[0,1]$. The binomial theorem gives

$$\sum_{i=0}^{m_1} B_i^{m_1}(\alpha) = ((1-\alpha)+\alpha)^{m_1} = 1.$$

Thus

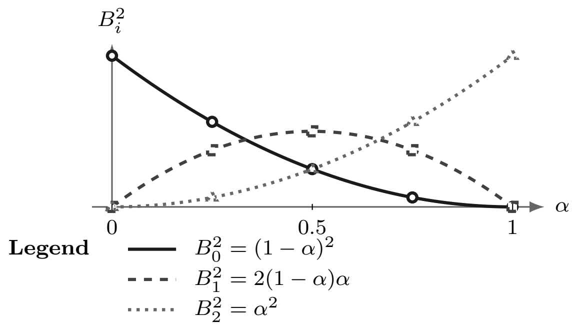
$$C^{(c)}(\alpha) = \sum_{i=0}^{m_1} B_i^{m_1}(\alpha) C^{(c)}[i]$$

is a convex combination of its local coefficient matrices.

Worked example. For degree two,

$$B_0^2 = (1-\alpha)^2, \qquad B_1^2 = 2(1-\alpha)\alpha, \qquad B_2^2 = \alpha^2.$$

Visual conclusion.



At every α, the three plotted heights are nonnegative and sum to one.

The label i identifies a basis position inside the selected physical cell and is distinct from a physical grid-node subscript.

A.4 Worked Conversion: $1 + 2\alpha + 3\alpha^2$

Consider

$$p(\alpha) = 1 + 2\alpha + 3\alpha^2.$$

Seek degree-two Bernstein coefficients b_0, b_1, b_2:

$$p = b_0(1-\alpha)^2 + 2b_1(1-\alpha)\alpha + b_2\alpha^2.$$

Matching powers of α gives

$$b_0 = 1, \qquad b_1 = 2, \qquad b_2 = 6.$$

Indeed,

$$1B_0^2 + 2B_1^2 + 6B_2^2 = (1 - 2\alpha + \alpha^2) + (4\alpha - 4\alpha^2) + 6\alpha^2 = 1 + 2\alpha + 3\alpha^2.$$

The three values $1, 2, 6$ are basis-function coefficients and are distinct from samples at three equally spaced physical points. Only the two endpoint coefficients are endpoint values. At $\alpha = 1/2$,

$$p(1/2) = \tfrac{1}{4}(1) + \tfrac{1}{2}(2) + \tfrac{1}{4}(6) = 2.75.$$

```
p = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
valueAtHalf = p.evaluate(0.5)
T = bernTable(p, "oneLine");
disp(T)
```

```
valueAtHalf = 2.7500
    CellSubscript                      Expression
    -------------    ----------------------------------------------

        {[1]}        "(1-alpha)^2*1 + 2(1-alpha)alpha*2 + alpha^2*6"
```

A.5 Exact Power–Bernstein Conversion

More generally, use brackets for algebraic coefficient labels as well:

$$p(\alpha) = \sum_{j=0}^{n} a[j]\alpha^j = \sum_{k=0}^{n} b[k]B_k^n(\alpha).$$

Power coefficients convert to Bernstein coefficients by

$$b[k] = \sum_{j=0}^{k} \frac{\binom{k}{j}}{\binom{n}{j}} a[j], \qquad k = 0, \ldots, n.$$

The inverse map is

$$a[j] = \binom{n}{j} \sum_{i=0}^{j} (-1)^{j-i} \binom{j}{i} b[i], \qquad j = 0, \ldots, n.$$

These are changes of basis inside the same degree-n polynomial space. For matrix polynomials they act entrywise. For tensor polynomials they act along one coordinate axis at a time.

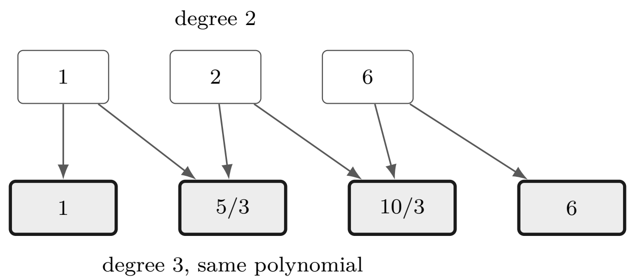
A.6 Lossless Degree Elevation

In one parameter, a polynomial on physical cell c with degree vector $\boldsymbol{m} = (m_1)$ has a Bernstein representation of every higher degree. One elevation step gives

$$\widehat{C}^{(c)}[0] = C^{(c)}[0], \qquad \widehat{C}^{(c)}[m_1+1] = C^{(c)}[m_1],$$
$$\widehat{C}^{(c)}[k] = \frac{k}{m_1+1} C^{(c)}[k-1] + \left(1 - \frac{k}{m_1+1}\right) C^{(c)}[k], \qquad k = 1, \ldots, m_1.$$

For the worked coefficients $[1, 2, 6]$, elevation from degree two to degree three produces

$$\left[1, \frac{5}{3}, \frac{10}{3}, 6\right].$$



Direct elevation from $\boldsymbol{m} = (m_1)$ to $\boldsymbol{M} = (M_1) \geq \boldsymbol{m}$ is

$$\widehat{C}^{(c)}[k] = \sum_{i=\max(0,k-M_1+m_1)}^{\min(m_1,k)} C^{(c)}[i] \frac{\binom{m_1}{i}\binom{M_1-m_1}{k-i}}{\binom{M_1}{k}}.$$

This formula is a binomially normalized discrete convolution. Define

$$U^{(c)}_{m_1}[i] = \binom{m_1}{i} C^{(c)}[i], \qquad E_{M_1-m_1}[j] = \binom{M_1 - m_1}{j}, \qquad \widehat{U}^{(c)}_{M_1}[k] = \binom{M_1}{k} \widehat{C}^{(c)}[k].$$

Then

$$\widehat{U}^{(c)}_{M_1} = U^{(c)}_{m_1} * E_{M_1-m_1}, \qquad \widehat{C}^{(c)}[k] = \frac{(U^{(c)}_{m_1} * E_{M_1-m_1})[k]}{\binom{M_1}{k}},$$

where $*$ is ordinary one-dimensional discrete convolution and out-of-range entries are zero. Thus elevation first binomially scales the coefficient row, convolves it with the fixed binomial kernel of component gap $M_1 - m_1$, and divides by the degree-component M_1 binomial weight. The new coefficients are fixed convex combinations of the old ones. Elevation therefore changes representation with zero added decision freedom. Constructing a new higher-degree `pdvar` enlarges the decision parameterization.

For a tensor coefficient array of degree $\boldsymbol{m} = (m_1, \ldots, m_\ell)$, elevate independently to any component-wise target $\boldsymbol{M} \geq \boldsymbol{m}$. With

$$U^{(\boldsymbol{c})}_{\boldsymbol{m}}[\boldsymbol{i}] = \left(\prod_{s=1}^{\ell} \binom{m_s}{i_s}\right) C^{(\boldsymbol{c})}[\boldsymbol{i}], \qquad E_{\boldsymbol{M}-\boldsymbol{m}}[\boldsymbol{j}] = \prod_{s=1}^{\ell} \binom{M_s - m_s}{j_s},$$

the elevated array satisfies

$$\left(\prod_{s=1}^{\ell} \binom{M_s}{k_s}\right) \widehat{C}^{(\boldsymbol{c})}[\boldsymbol{k}] = (U^{(\boldsymbol{c})}_{\boldsymbol{m}} *_\ell E_{\boldsymbol{M}-\boldsymbol{m}})[\boldsymbol{k}],$$

where $*_\ell$ denotes ℓ-dimensional discrete convolution. The same identity defines a sparse linear operator with zeros represented implicitly. Enumerate source labels $\boldsymbol{i}$ and target labels $\boldsymbol{k}$ in repository order, and define $E_{\boldsymbol{M}\leftarrow\boldsymbol{m}} \in \mathbb{R}^{N_{\boldsymbol{M}} \times N_{\boldsymbol{m}}}$ by

$$[E_{\boldsymbol{M}\leftarrow\boldsymbol{m}}]_{\boldsymbol{k},\boldsymbol{i}} = \begin{cases} \displaystyle\prod_{s=1}^{\ell} \frac{\binom{m_s}{i_s}\binom{M_s-m_s}{k_s-i_s}}{\binom{M_s}{k_s}}, & 0 \leq k_s - i_s \leq M_s - m_s \text{ for every } s, \\ 0, & \text{otherwise.} \end{cases}$$

Here $N_{\boldsymbol{d}} = \prod_{s=1}^{\ell}(d_s+1)$. Each source label contributes to the target labels in the translated gap box $\boldsymbol{i} + \prod_{s=1}^{\ell}\{0,\ldots,M_s - m_s\}$, so the operator has exactly $N_{\boldsymbol{m}} \prod_{s=1}^{\ell}(M_s - m_s + 1)$ positive entries. All remaining entries are zero.

If each coefficient is an a-by-b matrix, pack one cell and rate row horizontally as $C_{\text{pack}} = [C[\boldsymbol{i}_1] \ \cdots \ C[\boldsymbol{i}_{N_{\boldsymbol{m}}}]]$. The elevated payload is then

$$\widehat{C}_{\text{pack}} = C_{\text{pack}} \left(E_{\boldsymbol{M}\leftarrow\boldsymbol{m}}^{\top} \otimes I_b\right).$$

This right multiplication preserves the a rows and separates the b columns of each target coefficient. It also applies to affine `sdpvar` payloads because the operator and Kronecker factor are numeric sparse matrices. Public `elevate` delegates complete tree alignment to the protected `elevData` kernel and packed row application to `elevRow`. These kernels group temporary operands by source degree, build one operator for each pair $(\boldsymbol{m}, \boldsymbol{M})$, and reuse it across every physical cell and stored rate row with that shape. The returned object keeps the same represented function, existing YALMIP variables, metadata, physical-cell order, and rate-row order.

For example, take the one-component vectors $\boldsymbol{m} = (m_1) = (1)$ and $\boldsymbol{M} = (M_1) = (3)$. The corresponding operator is

$$E_{\boldsymbol{M}\leftarrow\boldsymbol{m}} = E_{(3)\leftarrow(1)} = \begin{bmatrix} 1 & 0 \\ 2/3 & 1/3 \\ 1/3 & 2/3 \\ 0 & 1 \end{bmatrix}, \qquad E_{(3)\leftarrow(1)} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix} = \begin{bmatrix} c_0\$2c_0 + c_1)/3\$c_0 + 2c_1)/3 \\ c_1 \end{bmatrix}.$$

The sparse operator is a different application form of the same Bernstein identity, so it preserves both the represented polynomial and the decision freedom.

A.7 Physical Tensor Grids And Local Tensor Labels

Suppose direction s has the axis grid

$$\mathcal{G}_s = \{\rho_s^{(1)}, \ldots, \rho_s^{(k_s)}\}, \qquad \rho_s^{(1)} < \rho_s^{(2)} < \cdots < \rho_s^{(k_s)}.$$

The counts k_s may differ. Thus $\mathcal{G} = \mathcal{G}_1 \times \cdots \times \mathcal{G}_\ell$ has $\prod_{s=1}^{\ell} k_s$ physical nodes. A physical hypercube is identified separately by the one-based multi-index $\boldsymbol{c} = (c_1, \ldots, c_\ell)$, with $\mathcal{H}_{\boldsymbol{c}} = \prod_{s=1}^{\ell}[\rho_s^{(c_s)}, \rho_s^{(c_s+1)}]$. Define its directional width and forward affine map by

$$\begin{aligned} h_s^{\boldsymbol{c}} &= \rho_s^{(c_s+1)} - \rho_s^{(c_s)}, \\ \phi_{\boldsymbol{c}} &: [0,1]^{\ell} \longrightarrow \mathcal{H}_{\boldsymbol{c}}, \\ \phi_{\boldsymbol{c},s}(\alpha_s) &= \rho_s^{(c_s)} + h_s^{\boldsymbol{c}} \alpha_s, \\ \boldsymbol{\alpha} &= \phi_{\boldsymbol{c}}^{-1}(\boldsymbol{\rho}), \qquad s = 1, \ldots, \ell. \end{aligned}$$

For any cell-wise quantity C, write $C^{(\boldsymbol{c})} = C \circ \phi_{\boldsymbol{c}}$ for its pullback to the unit box. For direction-wise degree $\boldsymbol{m} = (m_1, \ldots, m_\ell)$,

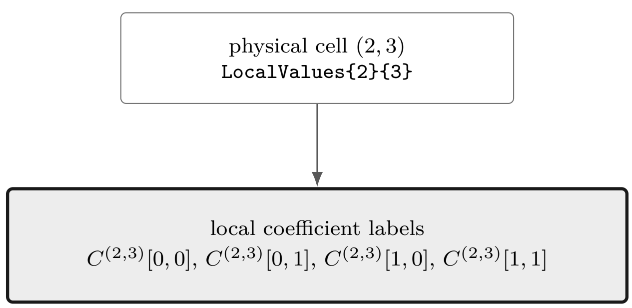
$$C^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i} \in \prod_{s=1}^{\ell}\{0,\ldots,m_s\}} B_{\boldsymbol{i}}^{\boldsymbol{m}}(\boldsymbol{\alpha}) C^{(\boldsymbol{c})}[\boldsymbol{i}], \qquad B_{\boldsymbol{i}}^{\boldsymbol{m}} = \prod_{s=1}^{\ell} B_{i_s}^{m_s}(\alpha_s).$$

There are $\prod_{s=1}^{\ell}(k_s - 1)$ physical hypercubes and $\prod_{s=1}^{\ell}(m_s + 1)$ zero-based local Bernstein labels per hypercube. A zero component $m_s = 0$ leaves only label $i_s = 0$, so the polynomial is constant in direction s. Physical node and hypercube indices remain one-based. If every component is the common scalar $\bar{m}$, then $\boldsymbol{m} = \bar{m}\mathbf{1}$ and the label count specializes to $(\bar{m}+1)^{\ell}$.

```
obj = pdbase({[0 1 2], [10 20 30 40]}, [1 1], 1);
physicalCellCount = obj.ncell()
coefficientsPerCell = obj.ncoeff()
localLabels = obj.lbls()
```

```
physicalCellCount = 6
coefficientsPerCell = 4
localLabels = 4×2 double
     0     0
     0     1
     1     0
     1     1
```

The nested tree `LocalValues{c1}{c2}...{cell}` follows physical-cell coordinates. Only after reaching a leaf does the flat coefficient cell follow the `lbls` order. Earlier dimensions vary more slowly in both `cells` and `lbls`.



Select the physical-cell leaf first. Only then read the flat local coefficient labels in `lbls` order.

A.8 Products As Ordered Weighted Convolution

Basic idea. Multiplying two Bernstein polynomials creates every ordered pair of input coefficients. Pairs with the same label sum contribute to the same output. For matrix payloads, "ordered" matters: the left factor stays on the left.

Formula. First consider one parameter, write the two degree vectors as $\boldsymbol{m} = (m_1)$ and $\boldsymbol{n} = (n_1)$, and fix the one-based physical cell c. Let

$$P = \sum_{i=0}^{m_1} B_i^{m_1} P^{(c)}[i], \qquad Q = \sum_{j=0}^{n_1} B_j^{n_1} Q^{(c)}[j].$$

The basis-product identity

$$B_i^{m_1} B_j^{n_1} = \frac{\binom{m_1}{i}\binom{n_1}{j}}{\binom{m_1+n_1}{i+j}} B_{i+j}^{m_1+n_1}$$

gives

$$(PQ)^{(c)}[k] = \sum_{i+j=k} P^{(c)}[i]Q^{(c)}[j]\frac{\binom{m_1}{i}\binom{n_1}{j}}{\binom{m_1+n_1}{k}}.$$

This is an ordered matrix convolution. In general $P^{(c)}[i]Q^{(c)}[j] \neq Q^{(c)}[j]P^{(c)}[i]$.
Equivalently, define the binomially scaled rows

$$\widetilde{P}^{(c)}[i] = \binom{m_1}{i}P^{(c)}[i], \qquad \widetilde{Q}^{(c)}[j] = \binom{n_1}{j}Q^{(c)}[j], \qquad \widetilde{R}^{(c)}[k] = \binom{m_1+n_1}{k}(PQ)^{(c)}[k].$$

Then

$$\widetilde{R}^{(c)} = \widetilde{P}^{(c)} * \widetilde{Q}^{(c)}.$$

The convolution remains ordered for matrix payloads, with the written factor order $\widetilde{P}^{(c)}[i]\widetilde{Q}^{(c)}[j]$ in every summand.

Worked example. Take quadratic coefficients $P = [1, 2, 6]$ and affine coefficients $Q = [2, 4]$. The anti-diagonals give

$$\begin{aligned} R[0] &= 1(2) = 2, \\ R[1] &= (1(4) + 2\,2(2))/3 = 4, \\ R[2] &= (2\,2(4) + 6(2))/3 = 28/3, \\ R[3] &= 6(4) = 24. \end{aligned}$$

```
P = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
Q = pdmat({[0 1]}, {2, 4}, Degree=1);
R = P * Q;
productDegree = R.Degree
productCoefficients = cell2mat(R.coeffs(1))
```

```
productDegree = 3
productCoefficients = 1×4 double
    2.0000    4.0000    9.3333   24.0000
```

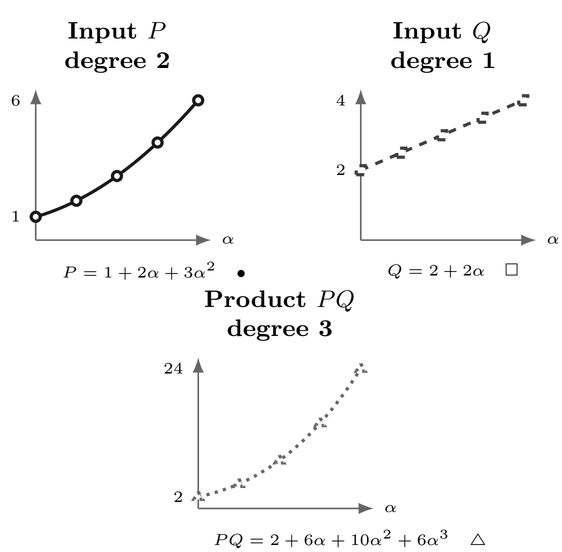
The product degree in the transcript is three: degrees add, so $2 + 1 = 3$. The four outputs are coefficients in that cubic Bernstein basis and are distinct from values sampled at four points.

The same calculation can be checked as a function identity. On $\alpha \in [0, 1]$,

$$P(\alpha) = 1 + 2\alpha + 3\alpha^2, \qquad Q(\alpha) = 2 + 2\alpha,$$

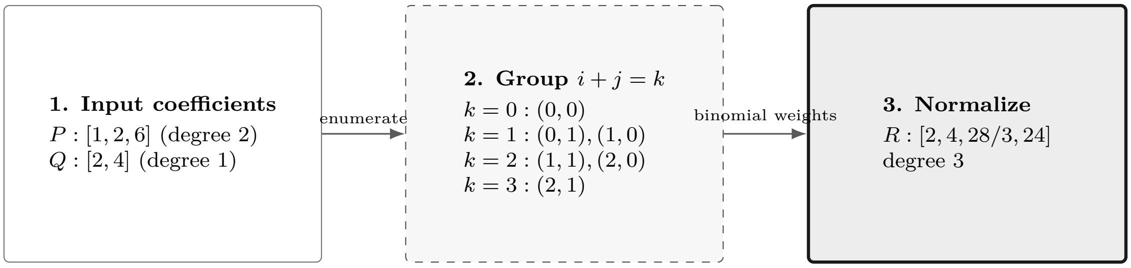
and

$$P(\alpha)Q(\alpha) = 2 + 6\alpha + 10\alpha^2 + 6\alpha^3.$$



Direct formula labels sit outside the plot regions. Solid/circle, dashed/square, and dotted/triangle treatments distinguish the three functions and remain legible in monochrome.

Visual conclusion.



Only the panel-to-panel arrows encode flow. Contributions are listed inside their anti-diagonal group, so every edge stays clear of labels and equations.

For tensor degrees $\boldsymbol{m}$ and $\boldsymbol{n}$, labels add componentwise and the product has degree $\boldsymbol{m} + \boldsymbol{n}$:

$$(PQ)^{(\boldsymbol{c})}[\boldsymbol{k}] = \sum_{\boldsymbol{i}+\boldsymbol{j}=\boldsymbol{k}} P^{(\boldsymbol{c})}[\boldsymbol{i}] Q^{(\boldsymbol{c})}[\boldsymbol{j}] \prod_{s=1}^{\ell} \frac{\binom{m_s}{i_s}\binom{n_s}{j_s}}{\binom{m_s+n_s}{k_s}}.$$

In binomially scaled form, the tensor rule is the ℓ-dimensional discrete convolution

$$\left(\prod_{s=1}^{\ell} \binom{m_s + n_s}{k_s}\right) (PQ)^{(\boldsymbol{c})}[\boldsymbol{k}] = \left(\left\{ \left(\prod_{s=1}^{\ell} \binom{m_s}{i_s}\right) P^{(\boldsymbol{c})}[\boldsymbol{i}] \right\}_{\boldsymbol{i}} *_{\ell} \left\{ \left(\prod_{s=1}^{\ell} \binom{n_s}{j_s}\right) Q^{(\boldsymbol{c})}[\boldsymbol{j}] \right\}_{\boldsymbol{j}} \right) [\boldsymbol{k}].$$

Labels add componentwise, physical-cell identity remains fixed, and every matrix product retains the written left-to-right factor order. The implementation uses this identity through three payload-specific kernels that share the same planned degrees, output labels, and coefficient weights.

A.8.1 Numeric Tensor Convolution

When both operands are numeric, each matrix entry is packed into a tensor whose coefficient at label $\boldsymbol{i}$ is multiplied by the normalized weight $w_{\boldsymbol{m}}(\boldsymbol{i}) = 2^{-\sum_{s=1}^{\ell} m_s} \prod_{s=1}^{\ell} \binom{m_s}{i_s}$. For output entry (a, b) and compatible inner dimension n_{in}, the matrix inner index q gives

$$\widetilde{R}_{ab} = \sum_{q=1}^{n_{\text{in}}} \widetilde{P}_{aq} *_{\ell} \widetilde{Q}_{qb}.$$

The planned tensor shapes map repository label order to MATLAB tensor indices, and division by $w_{\boldsymbol{m}+\boldsymbol{n}}(\boldsymbol{k})$ returns the Bernstein coefficients because the powers of two cancel. The normalized tables are generated directly from normalized weights, which keeps large raw binomial coefficients outside the computation. MATLAB `conv` handles one parameter and `convn` handles two or more parameters. This route requires numeric payloads. Affine `sdpvar` payloads use the generic or block-contraction route because `convn` implements numeric tensor semantics, while the other routes preserve YALMIP's affine expression structure.

A.8.2 Known–Affine Block Contraction

Suppose the left coefficients $A[\boldsymbol{i}]$ are numeric matrices and the right coefficients $X[\boldsymbol{j}]$ are affine `sdpvar` matrices. For each output label $\boldsymbol{k}$, precompute the pair table

$$\mathcal{P}_{\boldsymbol{k}} = \{(\boldsymbol{i}, \boldsymbol{j}) : \boldsymbol{i} + \boldsymbol{j} = \boldsymbol{k}\}$$

and its numeric weights $\omega_{\boldsymbol{i},\boldsymbol{j}}$. Stack the affine blocks vertically in repository order. The output can then be evaluated as one ordinary matrix product

$$R[\boldsymbol{k}] = \begin{bmatrix} \omega_1 A[\boldsymbol{i}_1] & \cdots & \omega_t A[\boldsymbol{i}_t] \end{bmatrix} \begin{bmatrix} X[\boldsymbol{j}_1] \\ \vdots \\ X[\boldsymbol{j}_t] \end{bmatrix}, \qquad t = |\mathcal{P}_{\boldsymbol{k}}|.$$

The symmetric implementation for an affine left operand and known right operand stacks the affine blocks horizontally and the weighted known blocks vertically, which preserves the written order $X[\boldsymbol{i}]A[\boldsymbol{j}]$. The pair labels and weights are numeric plan data and are reused across physical cells and rate rows.

For a one-parameter degree-one known factor A and degree-one affine factor X, the middle coefficient is

$$(AX)[1] = \frac{1}{2}\left(A[0]X[1] + A[1]X[0]\right) = \frac{1}{2}\begin{bmatrix} A[1] & A[0] \end{bmatrix} \begin{bmatrix} X[0] \\ X[1] \end{bmatrix}.$$

For example, with

$$A[0] = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad A[1] = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix},$$

the contraction multiplies one stacked affine vector of matrix blocks by the fixed numeric block row $\frac{1}{2}[A[1]\ A[0]]$. It creates the same affine YALMIP expression as the two-term sum and introduces zero new decision variables.

A.8.3 Planned Generic Fallback

Scalar broadcasting and other supported payload shapes use the same precomputed pair table through direct term accumulation. For every output label, the generic route computes

$$R[\boldsymbol{k}] = \sum_{r=1}^{|\mathcal{P}_{\boldsymbol{k}}|} \omega_r P[\boldsymbol{i}_r] Q[\boldsymbol{j}_r]$$

in deterministic pair order. This fallback keeps MATLAB scalar and matrix multiplication rules visible, while avoiding repeated label enumeration and repeated binomial-ratio construction. Ordinary coefficient rows may broadcast over one rate-row operand, but two actual rate-row operands are rejected because their product would be quadratic in the scheduling rates.

The protected `prodVals` kernel owns the three routes and reuses their numeric plans across the complete cell-wise trees. All routes preserve the same product polynomial and output degree. Numeric tensor shapes, pair tables, and weights may be reused because they are purely numeric. The input coefficient payloads and their existing decision variables remain attached to their own physical cells and rate rows.

A.9 Differentiation And Rate Vertices

Basic idea. Differentiate inside each physical cell. Adjacent control differences produce the derivative coefficients. Multiplying by a bounded rate then creates one stored row per rate-box vertex. Value continuity of P permits distinct cell-wise derivatives across a shared face.

Formula. For one parameter, write $\boldsymbol{m} = (m_1)$ and consider physical cell c of width h_1^c:

$$\frac{\partial P}{\partial \rho} = \frac{m_1}{h_1^c} \sum_{i=0}^{m_1-1} (C^{(c)}[i+1] - C^{(c)}[i]) B_i^{m_1-1}(\alpha).$$

Worked example. For the worked polynomial $[1, 2, 6]$ on $[0, 2]$, $\boldsymbol{m} = (2)$ and $h_1^c = 2$, so the derivative coefficients are

$$\left[\frac{2}{2}(2-1), \frac{2}{2}(6-2)\right] = [1, 4].$$

This represents $dp/d\rho = 1 + 3\alpha$.

In ℓ dimensions with tensor degree $\boldsymbol{m}$, differentiation with respect to ρ_s replaces each coefficient by

$$\frac{m_s}{h_s^{\boldsymbol{c}}} \left(C^{(\boldsymbol{c})}[\boldsymbol{i} + \boldsymbol{e}_s] - C^{(\boldsymbol{c})}[\boldsymbol{i}] \right),$$

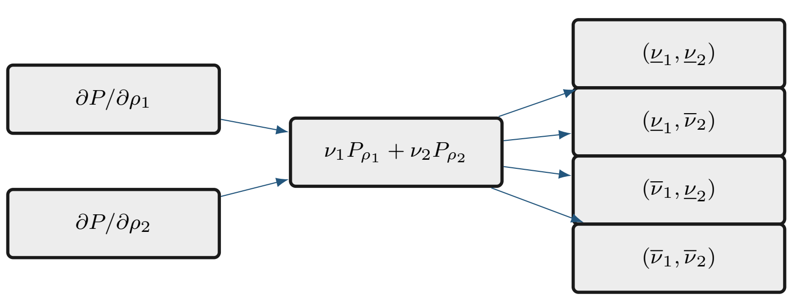
when $m_s > 0$, reduces only that component to $m_s - 1$, and leaves the other label components unchanged. A zero-degree axis has identically zero partial derivative. For two or more parameters, the implementation elevates every nonzero partial exactly from $\boldsymbol{m} - \boldsymbol{e}_s$ back to the common tensor degree $\boldsymbol{m}$ before adding them. In one parameter it retains the established reduced-degree result. If every component is zero, `rhodiff` stores a complete zero coefficient row for every rate vertex.

Finally, let $\boldsymbol{\nu} \in \mathcal{V}_{\mathcal{R}}$ denote one rate vertex. At this vertex,

$$\dot{P}(\boldsymbol{\rho}, \boldsymbol{\nu}) = \sum_{s=1}^{\ell} \nu_s \frac{\partial P}{\partial \rho_s}(\boldsymbol{\rho}).$$

The corresponding matrix expression is affine in the scheduling rate $\dot{\boldsymbol{\rho}}$. Every value inside the rate box $\mathcal{R}$ is therefore a convex combination of the distinct vertex matrices in $\mathcal{V}_{\mathcal{R}}$. Because the positive- and negative-semidefinite cones are convex, the requested semidefinite sign holds on all of $\mathcal{R}$ whenever it holds at every distinct vertex. `rhodiff` stores one row for each of these vertices.

Visual conclusion.



A.9.1 Rate-Vertex Row Storage

Let RateBounds$(s,:) = [\underline{\nu}_s\ \overline{\nu}_s]$. The rate box $\mathcal{R} = \prod_{s=1}^{\ell}[\underline{\nu}_s, \overline{\nu}_s]$ has the finite vertex set

$$\mathcal{V}_{\mathcal{R}} = \prod_{s=1}^{\ell}\{\underline{\nu}_s, \overline{\nu}_s\}, \qquad N_v := |\mathcal{V}_{\mathcal{R}}| = \prod_{s=1}^{\ell}\left(1 + \mathbf{1}_{\{\underline{\nu}_s < \overline{\nu}_s\}}\right) \le 2^{\ell}.$$

Here a varying direction contributes its lower and upper values, while a fixed direction contributes its single value. For a derivative object $D =$ rhodiff$(P,$ RateBounds$)$ and one physical hypercube $\boldsymbol{c} = (c_1, \ldots, c_\ell)$, the leaf `D.LocalValues`$\{c_1\}\cdots\{c_\ell\}$ is an N_v-by-$\prod_{s=1}^{\ell}($`D.Degree`$(s)+1)$ cell array. Its rows enumerate vertices $\boldsymbol{\nu}^{(q)} \in \mathcal{V}_{\mathcal{R}}$. Its columns enumerate local Bernstein labels in combRows order. Each entry already contains $\sum_{s=1}^{\ell} \nu_s^{(q)}\, \partial P^{(\boldsymbol{c})}/\partial \rho_s$. The rows add rate alternatives inside one existing hypercube while preserving the physical-cell count.

For $\ell = 2$ when both rate directions vary, the lower/upper tensor order is

row	ν_1	ν_2
1	$\underline{\nu}_1$	$\underline{\nu}_2$
2	$\underline{\nu}_1$	$\overline{\nu}_2$
3	$\overline{\nu}_1$	$\underline{\nu}_2$
4	$\overline{\nu}_1$	$\overline{\nu}_2$

The last parameter direction changes fastest. This is the same tensor ordering used by the package's coefficient and cell traversal, so pdlmi can constrain every rate row directly from the stored rate-box representation.

A.10 de Casteljau Evaluation And Exact Subdivision

de Casteljau recursion evaluates a Bernstein polynomial using only affine combinations. For one parameter, write the degree as $\boldsymbol{m} = (m_1)$. On a fixed physical cell c, begin with $C^{(c,0)}[i] = C^{(c)}[i]$ and define

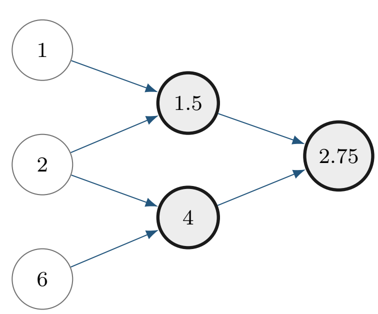
$$C^{(c,r)}[i] = (1-\tau)C^{(c,r-1)}[i] + \tau C^{(c,r-1)}[i+1], \qquad r = 1, \ldots, m_1.$$

The value at $\alpha = \tau$ is $C^{(c,m_1)}[0]$.

For $[1,2,6]$ and $\tau = 1/2$, the recursion is

$$[1,2,6] \;\longrightarrow\; [1.5,4] \;\longrightarrow\; [2.75].$$

The left edge of this triangle gives exact coefficients $[1, 1.5, 2.75]$ on the left subinterval. The right edge gives $[2.75, 4, 6]$ on the right.



One recursion triangle supplies both the evaluation and the two child-cell coefficient lists.

Tensor evaluation or subdivision applies the same recursion along one parameter direction at a time. Subdivision changes the physical-cell partition and can tighten coefficient hulls while preserving the polynomial. The package uses exact coefficient transformations internally where documented. Its current public transformation API exposes evaluation, degree elevation, and differentiation, while de Casteljau subdivision and adaptive refinement remain implementation concepts.

A.11 Changed and Preserved Quantities

The following distinctions prevent several common modeling mistakes.

Operation	Changes	Preserves
Basis conversion	coefficient coordinates	polynomial, degree, physical cell
Degree elevation	Bernstein degree and coefficient list	polynomial, physical grid, decision freedom
Grid subdivision	physical-cell partition and local coefficients	polynomial on the original domain
Higher-degree `pdvar`	number of decision coefficients	grid and matrix size
Multiplication	polynomial degree and coefficients	physical-cell identity
Differentiation	degree and coefficient values	physical-cell partition before rate rows

These representation facts support the finite certificates in the next appendix. Solver selection belongs to the caller, and each implemented certificate supplies a sufficient condition.

A.12 Further Reading

For a concise orientation before the primary paper by Farouki [3], see the Bernstein polynomial overview and the de Casteljau overview. They are background references for the basis and recursion; the package's cell-wise storage and certificate behavior are defined by this manual and the source implementation.

Appendix B

Polynomial-Matrix Certificates on Hypercubes

This appendix starts with positive semidefinite matrices, builds scalar and matrix sum-of-squares representations, and then explains the six certificate paths implemented by `pdlmi`: Direct, Pólya, Putinar, SparsePutinar, SparseFullBox, and FullBox. Background constructions place these APIs in their mathematical context, while the implementation boundary is stated explicitly near the end.

As in the preceding appendix, each major certificate follows the order *Basic idea* → *Formula* → *Worked example* → *Visual conclusion*. Keep two questions separate while reading. The first asks which polynomial matrix is represented, and the second asks which finite cone certifies its sign.

Fix one physical cell $\mathcal{H}_{\boldsymbol{c}}$ and one stored rate row. In the non-strict package formulation of (1.4), write the cell residual as $\mathcal{F}^{(\boldsymbol{c})} \preceq 0$ and define the sign-normalized positive target $S^{(\boldsymbol{c})} = -\mathcal{F}^{(\boldsymbol{c})}$. The package target is

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) \succeq 0 \quad \text{for every } \boldsymbol{\alpha} \in [0,1]^{\ell}.$$

The Direct and Gram constraints prove this non-strict inequality. A strict analysis condition such as the induced-$\mathcal{L}_2$-gain condition in Chapter 1 must state its own margin, for example $S^{(\boldsymbol{c})} - \varepsilon I \succeq 0$. The package uses a zero implicit margin.

B.1 Positive Semidefinite Matrices And LMIs

Basic idea. A real symmetric matrix Q belongs to the Positive Semidefinite (PSD) cone when

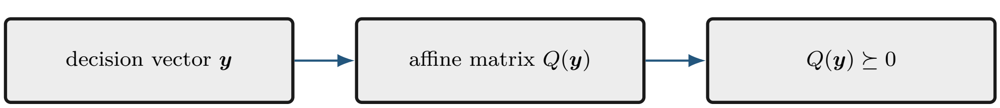
$$u^{\top} Q u \geq 0 \qquad \text{for every vector } u.$$

Formula. Let $\boldsymbol{y} = (y_1, \ldots, y_N)$ collect the scalar decision coordinates. An LMI is an affine matrix expression constrained to the PSD cone:

$$Q(\boldsymbol{y}) = Q_0 + \sum_{k=1}^{N} y_k Q_k \succeq 0.$$

The unknowns enter affinely, so this is a Semidefinite Program (SDP). A polynomial matrix sign condition appears infinite because it asks for one PSD matrix at every point. A certificate replaces those pointwise conditions by finitely many PSD matrices and affine coefficient identities.

Visual conclusion.



B.2 SOS And Gram Matrices

Basic idea. A polynomial in $\boldsymbol{\xi} = (\xi_1, \dots, \xi_\ell)$ is a finite sum

$$p(\boldsymbol{\xi}) = \sum_{\boldsymbol{\eta} \in \mathbb{N}^\ell} p_{\boldsymbol{\eta}} \boldsymbol{\xi}^{\boldsymbol{\eta}}, \qquad \boldsymbol{\xi}^{\boldsymbol{\eta}} = \prod_{s=1}^{\ell} \xi_s^{\eta_s},$$

with only finitely many nonzero coefficients. Its total degree is $\deg p = \max\{|\boldsymbol{\eta}| : p_{\boldsymbol{\eta}} \neq 0\}$, where $|\boldsymbol{\eta}| = \sum_{s=1}^{\ell} \eta_s$. A scalar polynomial is Sum-Of-Squares (SOS) if $p = q_1^2 + \cdots + q_t^2$, which immediately implies $p \geq 0$.

Formula. Suppose $\deg p \leq 2d$. Collect every monomial of total degree at most d into

$$v_d(\boldsymbol{\xi}) = \left(\boldsymbol{\xi}^{\boldsymbol{\beta}}\right)_{|\boldsymbol{\beta}| \leq d} \in \mathbb{R}^{q_d}, \qquad q_d = \binom{\ell + d}{d}.$$

The scalar Gram form is

$$p(\boldsymbol{\xi}) = v_d(\boldsymbol{\xi})^\top Q v_d(\boldsymbol{\xi}), \qquad Q \in \mathbb{S}^{q_d}, \qquad Q \succeq 0.$$

A factorization $Q = L^\top L$ yields $p = \|L v_d\|_2^2$, so the Gram form is SOS. Conversely, the coefficient vectors of any finite SOS decomposition can be stacked into such a factor L, which gives a PSD Gram matrix. Expanding the Gram form gives the coefficient identities

$$p_{\boldsymbol{\eta}} = \sum_{\substack{|\boldsymbol{\beta}| \leq d,\ |\boldsymbol{\gamma}| \leq d \\ \boldsymbol{\beta} + \boldsymbol{\gamma} = \boldsymbol{\eta}}} Q_{\boldsymbol{\beta}, \boldsymbol{\gamma}} \qquad \text{for every } \boldsymbol{\eta}.$$

These identities are affine in Q. Several Gram matrices may represent the same polynomial because the same monomial $\boldsymbol{\xi}^{\boldsymbol{\eta}}$ can arise from several pairs $(\boldsymbol{\beta}, \boldsymbol{\gamma})$. The certificate searches for any PSD Q satisfying all coefficient identities.

A symmetric n-by-n polynomial matrix S is matrix SOS if there is a polynomial matrix H such that $S = H^\top H$. To express this condition through one PSD Gram matrix, lift the monomial basis to

$$Z_d(\boldsymbol{\xi}) = v_d(\boldsymbol{\xi}) \otimes I_n \in \mathbb{R}^{q_d n \times n}, \qquad S^{(\boldsymbol{c})}(\boldsymbol{\xi}) = Z_d(\boldsymbol{\xi})^\top Q Z_d(\boldsymbol{\xi}), \qquad Q \in \mathbb{S}^{q_d n}, \quad Q \succeq 0.$$

Then $u^\top S^{(\boldsymbol{c})}(\boldsymbol{\xi}) u = (Z_d(\boldsymbol{\xi}) u)^\top Q (Z_d(\boldsymbol{\xi}) u) \geq 0$ for every u, so $S^{(\boldsymbol{c})}(\boldsymbol{\xi}) \succeq 0$. A factorization of Q also gives the polynomial factor H. Thus a feasible matrix Gram identity is a finite sufficient certificate for pointwise positive semidefiniteness. Matrix-SOS relaxations are developed in [11].

The package performs the analogous matching in tensor-product Bernstein coordinates. For a tensor degree $\boldsymbol{a} = (a_1, \dots, a_\ell) \in \mathbb{N}^\ell$, define

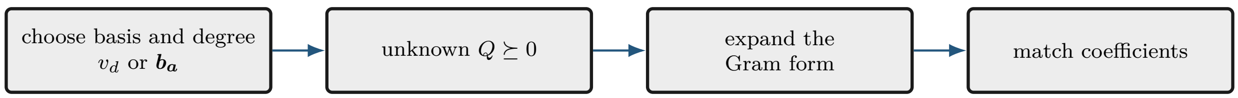
$$\boldsymbol{b}_{\boldsymbol{a}}(\boldsymbol{\alpha}) = \bigotimes_{s=1}^{\ell} \left[B_0^{a_s}(\alpha_s) \quad \cdots \quad B_{a_s}^{a_s}(\alpha_s) \right]^\top, \qquad Z_{\boldsymbol{a}}(\boldsymbol{\alpha}) = \boldsymbol{b}_{\boldsymbol{a}}(\boldsymbol{\alpha}) \otimes I_n.$$

The vector $\boldsymbol{b}_{\boldsymbol{a}}$ has $\prod_{s=1}^{\ell}(a_s + 1)$ entries. Therefore $Z_{\boldsymbol{a}}$ has $\prod_{s=1}^{\ell}(a_s + 1)$ block rows of size n-by-n, equivalently $n \prod_{s=1}^{\ell}(a_s + 1)$ scalar rows, and Q has scalar dimension $n \prod_{s=1}^{\ell}(a_s + 1)$. Each n-by-n block $Q[\boldsymbol{\eta}, \boldsymbol{\zeta}]$ multiplies one ordered pair of tensor Bernstein basis functions. When the Gram form is expanded, all ordered pairs that reach the same target label are added before that target coefficient is equated to the corresponding residual coefficient. The unweighted form $Z_{\boldsymbol{a}}^\top Q Z_{\boldsymbol{a}}$ has coordinatewise degree $2\boldsymbol{a}$. Multiplication by

$$g_J(\boldsymbol{\alpha}) = \prod_{s \in J} \alpha_s(1-\alpha_s)$$

adds $2\boldsymbol{\chi}_J$ to that coordinatewise degree. Consequently, a box-weighted term matched at directionwise degree $2\boldsymbol{r}$ uses $\boldsymbol{a} = \boldsymbol{r} - \boldsymbol{\chi}_J$. The uniform choice $\boldsymbol{r} = r\mathbf{1}$ is a specialization. In the odd one-dimensional interval form, the endpoint weights α and $1-\alpha$ instead add degree one. Expanding each weighted Bernstein Gram form and equating it to the target Bernstein array produces affine coefficient identities, just as in the monomial example.

Visual conclusion.



B.3 Reusable Bernstein–Gram Coefficient Maps

Fix one weighted Gram term with tensor basis degree $\boldsymbol{d}$, alpha power $\boldsymbol{p}$, and one-minus-alpha power $\boldsymbol{q}$. Let $\mathcal{B}$ be either the complete tensor label set $\prod_{s=1}^{\ell}\{0,\dots,d_s\}$ or one sliding tensor window, and partition the symmetric Gram matrix Q into n-by-n blocks $Q[\boldsymbol{\eta},\boldsymbol{\zeta}]$ indexed by labels in $\mathcal{B}$. The common target degree is $\boldsymbol{h} = 2\boldsymbol{d} + \boldsymbol{p} + \boldsymbol{q}$. For $\boldsymbol{k} \in \prod_{s=1}^{\ell}\{0,\dots,h_s\}$, define

$$\omega_{\boldsymbol{\eta},\boldsymbol{\zeta}}^{\boldsymbol{k}} = \prod_{s=1}^{\ell} \frac{\binom{d_s}{\eta_s}\binom{d_s}{\zeta_s}}{\binom{h_s}{k_s}}, \qquad \boldsymbol{k} = \boldsymbol{\eta} + \boldsymbol{\zeta} + \boldsymbol{p}.$$

The coefficient map separates diagonal and paired off-diagonal contributions:

$$\mathcal{G}_{\boldsymbol{k}}(Q) = \sum_{\substack{\boldsymbol{\eta}\in\mathcal{B} \\ 2\boldsymbol{\eta}+\boldsymbol{p}=\boldsymbol{k}}} \omega_{\boldsymbol{\eta},\boldsymbol{\eta}}^{\boldsymbol{k}} Q[\boldsymbol{\eta},\boldsymbol{\eta}] + \sum_{\substack{\boldsymbol{\eta},\boldsymbol{\zeta}\in\mathcal{B} \\ \boldsymbol{\eta}<\boldsymbol{\zeta} \\ \boldsymbol{\eta}+\boldsymbol{\zeta}+\boldsymbol{p}=\boldsymbol{k}}} \omega_{\boldsymbol{\eta},\boldsymbol{\zeta}}^{\boldsymbol{k}} \left(Q[\boldsymbol{\eta},\boldsymbol{\zeta}] + Q[\boldsymbol{\zeta},\boldsymbol{\eta}]\right).$$

The order $\boldsymbol{\eta} < \boldsymbol{\zeta}$ is the repository basis order. Grouping an off-diagonal pair once avoids enumerating both ordered pairs in the numeric plan, while the block sum preserves the complete Gram expansion. The powers $\boldsymbol{q}$ affect the target degree and its binomial denominator, whereas $\boldsymbol{p}$ also shifts the target label.

For the small scalar basis $z = [B_0^1, B_1^1]^\top$ with unit generator weight,

$$z^\top Q z = Q_{00}B_0^2 + \frac{Q_{01}+Q_{10}}{2}B_1^2 + Q_{11}B_2^2.$$

Thus the exact degree-two coefficient identities are $C[0] = Q_{00}$, $C[1] = (Q_{01}+Q_{10})/2$, and $C[2] = Q_{11}$. For a scalar symmetric Gram matrix, the middle coefficient reduces to Q_{01}. For matrix blocks, it is the symmetric part $(Q_{01} + Q_{01}^\top)/2$.

The label pairs, output labels, binomial ratios, diagonal groups, and off-diagonal groups are numeric. The consolidated protected `gramCons` kernel builds these maps once for every distinct term and window shape, then reuses them while assembling all compatible certificates. However, it allocates a fresh symmetric Gram variable for every matrix entry in entry-wise mode or every semidefinite block in matrix mode, for every physical cell, stored rate row, active term, and tensor window. Reusing a Gram variable across any of those axes would couple independent certificates and change the cone partition, so only the numeric maps are reusable. For each local certificate, all PSD cone constraints are stored first in term and window order, followed by the exact coefficient identities in target-label order. Physical cells, rate rows, and matrix entries retain their established outer traversal order.

B.4 Full-Space SOS

An unweighted monomial Gram form proves $S^{(\boldsymbol{c})}(\boldsymbol{\xi}) \succeq 0$ on all of $\mathbb{R}^{\ell}$. That can be much stronger than positivity only on a box. For example, $\alpha(1-\alpha)$ is nonnegative on $[0,1]$ but negative outside it. The package exposes weighted box-certificate selectors, while a general full-space SOS parser lies outside its public scope. This construction is background for understanding weighted certificates.

B.5 Direct Bernstein Coefficients

Basic idea. If every local coefficient matrix has the requested sign, every convex combination of those coefficients has the same sign. This yields a small, transparent sufficient certificate.

Formula. After any rate rows have been stacked, write the residual and target coefficients on physical cell $\boldsymbol{c}$ as $\mathcal{F}^{(\boldsymbol{c})} = \sum_{\boldsymbol{i}\in\prod_{s=1}^{\ell}\{0,\ldots,M_s\}} B_{\boldsymbol{i}}^{\boldsymbol{M}}\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}]$ and $C^{(\boldsymbol{c})}[\boldsymbol{i}] = -\mathcal{C}^{(\boldsymbol{c})}[\boldsymbol{i}]$. Then

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i}\in\prod_{s=1}^{\ell}\{0,\ldots,M_s\}} B_{\boldsymbol{i}}^{\boldsymbol{M}}(\boldsymbol{\alpha})C^{(\boldsymbol{c})}[\boldsymbol{i}].$$

Because the basis weights are nonnegative and sum to one,

$$C^{(\boldsymbol{c})}[\boldsymbol{i}] \succeq 0 \ \forall \boldsymbol{i} \quad \Longrightarrow \quad S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) \succeq 0.$$

This is the package default. It introduces zero Gram variables and creates one constraint per physical cell, rate row, and local label. Feasible residuals may lie outside this sufficient certificate set.

Worked example. Consider

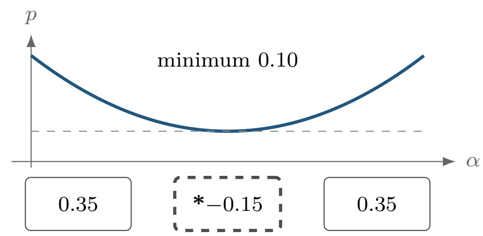
$$p(\alpha) = \alpha^2 - \alpha + 0.35 = (\alpha - 0.5)^2 + 0.10.$$

It is strictly positive, with minimum 0.10, but its degree-two Bernstein coefficients are

$$[\,0.35,\ -0.15,\ 0.35\,].$$

The polynomial remains positive despite its negative middle coefficient, so the direct certificate fails on this representation.

Visual conclusion.



A positive curve can have a negative Bernstein coefficient.

The same polynomial has the PSD Bernstein Gram representation

$$\boldsymbol{b}_1(\alpha) = \begin{bmatrix} 1-\alpha \\ \alpha \end{bmatrix}, \qquad Q = \begin{bmatrix} 0.35 & -0.15 \\ -0.15 & 0.35 \end{bmatrix} \succeq 0, \qquad p = \boldsymbol{b}_1^\top Q \boldsymbol{b}_1.$$

The eigenvalues of Q are 0.20 and 0.50. A complete Gram certificate can therefore succeed when coefficient-wise signs fail.

B.6 Pólya As Lossless Elevation

Basic idea. Pólya selection preserves the residual polynomial. It rewrites that same polynomial at a higher Bernstein degree and then retries the direct coefficient test.

Formula. Classical Pólya theory concerns homogeneous polynomials strictly positive on a simplex [14]. For one box coordinate,

$$\lambda_0 = 1-\alpha, \quad \lambda_1 = \alpha, \quad \lambda_0 + \lambda_1 = 1.$$

For ℓ coordinates, the implemented multiplier is

$$\prod_{s=1}^{\ell} (\lambda_{s,0} + \lambda_{s,1})^d = 1.$$

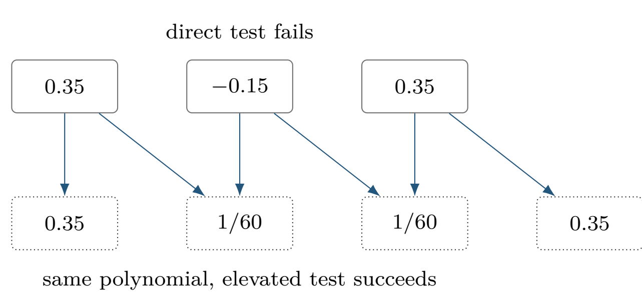
Regrouping is exactly Bernstein degree elevation: the represented residual remains unchanged.

Worked example. For the worked coefficients,

$$[\,0.35, -0.15, 0.35\,] \longrightarrow \left[0.35, \frac{1}{60}, \frac{1}{60}, 0.35\right]$$

after one elevation. Every new coefficient is positive, so the elevated coefficient test succeeds.

Visual conclusion.



`usePolya(d)` accepts scalar uniform shorthand or a direction-wise increment $\boldsymbol{d}$, elevates from $\boldsymbol{M}$ to $\boldsymbol{M}+\boldsymbol{d}$, and then applies coefficient constraints to every cell and rate row. It adds zero Gram variables and zero decision freedom. A fixed-level failure is inconclusive, and the implemented scope consists of fixed-level sufficient certificates on tensor boxes.

B.6.1 Preallocation and Ordered Coefficient Assembly

Direct and Pólya use the consolidated protected `coeffCons` assembly kernel after the optional elevation step. For a `pdvar` residual, let N_c be the physical-cell count, let N_r be one for ordinary storage or the stored rate-row count, and let $N_{\boldsymbol{H}} = \prod_{s=1}^{\ell}(H_s+1)$ be the coefficient count at the assembled degree $\boldsymbol{H}$. Before creating constraints, the implementation allocates $N_c N_r N_{\boldsymbol{H}}$ cells. It then traverses physical cells in `cells()` order, rate rows in stored vertex order, and local labels in `lbls()` order. Semidefinite mode adds one matrix inequality per coefficient, while entry-wise mode adds one vectorized scalar group in MATLAB column-major order. Equality uses the same traversal but omits coefficients that are proven numeric zero and trims the unused preallocated tail.

For Pólya, the sparse elevation operator first produces degree $\boldsymbol{H} = \boldsymbol{M}+\boldsymbol{d}$, after which the same loop is applied. A coefficient-backed known `pdmat` uses the same deterministic traversal to evaluate one global logical sufficient certificate instead of constructing YALMIP constraints. This preallocation and traversal preserve the previous constraint order while avoiding repeated growth of the constraint container.

B.7 Exact Interval Markov–Lukács Forms

Basic idea. On one interval, endpoint factors can encode the domain exactly. Even and odd target degrees require different weighted Gram patterns, so parity is a structural choice with distinct coefficient maps.

Formula. In one variable, write the Gram order as $\boldsymbol{r} = (r_1)$. Interval nonnegativity has exact parity-specific descriptions [8, 9]. If $\deg p \le 2r_1$,

$$p \ge 0 \text{ on } [0,1] \iff p = s_0 + \alpha(1-\alpha)s_1,$$

where s_0, s_1 are SOS of degrees at most $2r_1$ and $2r_1 - 2$. If $\deg p \le 2r_1 + 1$,

$$p \ge 0 \text{ on } [0,1] \iff p = (1-\alpha)s_L + \alpha s_U,$$

where s_L, s_U are SOS of degree at most $2r_1$.

For polynomial matrices, with

$$Z_d(\alpha) = \begin{bmatrix} B_0^d I_n & \cdots & B_d^d I_n \end{bmatrix}^\top,$$

the even and odd forms are

$$S^{(c)} = Z_{r_1}^\top Q_0 Z_{r_1} + \alpha(1-\alpha) Z_{r_1-1}^\top Q_1 Z_{r_1-1}, \qquad Q_0, Q_1 \succeq 0,$$

and

$$S^{(c)} = (1-\alpha) Z_{r_1}^\top Q_L Z_{r_1} + \alpha Z_{r_1}^\top Q_U Z_{r_1}, \qquad Q_L, Q_U \succeq 0.$$

The second even block is absent at $r_1 = 0$. See [10, 11] for polynomial-matrix context.

Worked example. A cubic residual has component degree $2r_1+1$ with $\boldsymbol{r}=(1)$. It therefore uses two degree-one Gram bases, one multiplied by $1-\alpha$ and one by α. This is the generic one-dimensional odd-degree full-box pattern.

Visual conclusion.

even $2r_1$	odd $2r_1+1$
unweighted Gram + $\alpha(1-\alpha)$-weighted Gram	$(1-\alpha)$ lower-face Gram + α upper-face Gram

Partition Q into n-by-n blocks Q_{ij}. The Bernstein coefficient at label k of $\alpha^a(1-\alpha)^b Z_d^\top Q Z_d$, expressed at total degree $2d+a+b$, is

$$\mathcal{G}_k^{d,a,b}(Q)=\frac{1}{\binom{2d+a+b}{k}}\sum_{\substack{0\le i,j\le d\\ i+j=k-a}}\binom{d}{i}\binom{d}{j}Q_{ij}.$$

Equating this affine map to each target coefficient gives exact linear identities, while $Q\succeq 0$ supplies the LMIs. For one parameter, `usePutinar` and `useFullBox` implement these exact parity forms. `useSpPut` and `useSpBox` apply their respective sliding tensor-window constructions to the active Putinar or FullBox terms. With assembled degree $\boldsymbol{M}=(M_1)$, their minimum absolute order vector is $\boldsymbol{r}=(\lfloor M_1/2\rfloor)$.

B.8 Putinar Modules And Schmüdgen Preorderings

For

$$K=\{\xi : g_1(\xi)\ge 0,\dots,g_q(\xi)\ge 0\},$$

a Putinar quadratic module has the form

$$\sigma_0+\sum_{j=1}^{q}\sigma_j g_j,\qquad \sigma_j \text{ SOS}.$$

Under the Archimedean hypothesis, every polynomial strictly positive on K has such a representation at some degree [12]. Compactness is a distinct, weaker property. The package implements one box-specific, fixed-order selector. General-domain and general-generator parsers lie outside its public scope.

For one parameter, `usePutinar(r)` dispatches to the same exact even/odd Markov–Lukács forms as `useFullBox(r)`. Write the assembled degree and absolute order as $\boldsymbol{M}=(M_1)$ and $\boldsymbol{r}=(r_1)$. The default and minimum satisfy $r_1=\lfloor M_1/2\rfloor$. Even targets use the unweighted and $\alpha(1-\alpha)$-weighted Gram blocks and match at component degree $2r_1$, while odd targets use the two endpoint-weighted Gram blocks and match at component degree $2r_1+1$.

For $\ell\ge 2$, use the local box generators

$$g_s(\boldsymbol{\alpha})=\alpha_s(1-\alpha_s),$$

and `usePutinar(r)` uses

$$S^{(\boldsymbol{c})}=S_0+\sum_{s=1}^{\ell}g_sS_s,\qquad S_0=Z_{\boldsymbol{r}}^\top Q_0 Z_{\boldsymbol{r}},\quad S_s=Z_{\boldsymbol{r}-\boldsymbol{e}_s}^\top Q_s Z_{\boldsymbol{r}-\boldsymbol{e}_s},$$

where every present $Q_s \succeq 0$. In this multivariate branch the direction-wise order $\boldsymbol{r}$ is absolute and satisfies $\boldsymbol{r} \geq \lceil \boldsymbol{M}/2 \rceil$ componentwise. Exact identities match the sign-normalized target, elevated while preserving it, at tensor degree $2\boldsymbol{r}$. A multiplier block whose basis degree has a negative component is omitted. Each physical cell and stored rate row owns independent Gram and equality blocks. The construction uses singleton generators, a zero implicit margin, and caller-owned solver invocation.

A Schmüdgen full preordering includes every square-free generator product:

$$p = \sum_{\beta \in \{0,1\}^q} \sigma_\beta \prod_{j=1}^{q} g_j^{\beta_j}, \qquad \sigma_\beta \text{ SOS}.$$

Strict positivity on a compact basic closed semialgebraic set has such a representation [13]. A finite truncation is still a sufficient certificate, and up to 2^q multiplier blocks may be required.

B.9 The Implemented SparsePutinar Tensor-Window State

Basic idea. Dense Putinar assigns one PSD Gram matrix to each active parity term or generator mask. SparsePutinar covers that term's tensor Bernstein Gram basis by overlapping axis-aligned windows and assigns an independent PSD matrix to every window. The coefficient identity then adds all contributions from all windows and all active terms. This windowed construction can reduce the maximum basis cardinality in one PSD block while preserving the target polynomial and the exact coefficient-matching degree.

Window construction. Fix one active weighted Putinar term with Gram degree $\boldsymbol{d}$, weight $\boldsymbol{\alpha}^{\boldsymbol{p}}(\boldsymbol{1}-\boldsymbol{\alpha})^{\boldsymbol{q}}$, and common target degree

$$\boldsymbol{h} = 2\boldsymbol{d} + \boldsymbol{p} + \boldsymbol{q}.$$

For clique size b, define the per-axis window size $w_s = \min\{b, d_s + 1\}$. A window start satisfies $0 \leq u_s \leq d_s - w_s + 1$, and its label set is

$$\mathcal{W}_{\boldsymbol{u}} = \{\boldsymbol{\eta} : u_s \leq \eta_s \leq u_s + w_s - 1 \text{ for } s = 1, \dots, \ell\}.$$

The window basis has $\prod_{s=1}^{\ell} w_s$ labels, so an n-by-n polynomial-matrix term uses an independent PSD block $Q_{\boldsymbol{u}}$ of scalar dimension

$$n \prod_{s=1}^{\ell} w_s = n \prod_{s=1}^{\ell} \min\{b, d_s + 1\}.$$

The number of windows for this term is

$$\prod_{s=1}^{\ell} (d_s - w_s + 2).$$

Exact accumulated identity. For multi-indices, write $\binom{\boldsymbol{a}}{\boldsymbol{k}} = \prod_{s=1}^{\ell} \binom{a_s}{k_s}$. The contribution of one window to target label $\boldsymbol{k}$ is

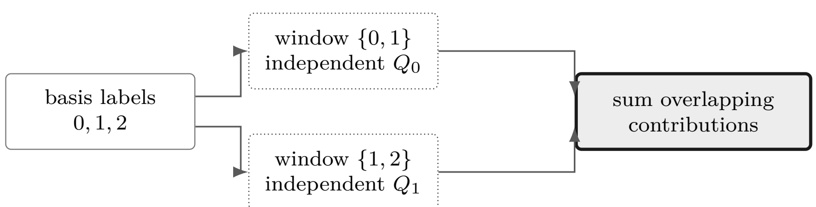
$$\mathcal{A}_{\boldsymbol{k},\boldsymbol{u}}(Q_{\boldsymbol{u}}) = \sum_{\substack{\boldsymbol{\eta},\boldsymbol{\zeta}\in\mathcal{W}_{\boldsymbol{u}} \\ \boldsymbol{\eta}+\boldsymbol{\zeta}+\boldsymbol{p}=\boldsymbol{k}}} \frac{\binom{\boldsymbol{d}}{\boldsymbol{\eta}}\binom{\boldsymbol{d}}{\boldsymbol{\zeta}}}{\binom{\boldsymbol{h}}{\boldsymbol{k}}} Q_{\boldsymbol{u}}[\boldsymbol{\eta},\boldsymbol{\zeta}].$$

Let $\mathcal{T}$ be the finite set of active Putinar terms, and let $\widetilde{C}^{(\boldsymbol{c})}[\boldsymbol{k}]$ be the sign-normalized target coefficient after exact elevation to their common degree $\boldsymbol{h}$. SparsePutinar imposes

$$\widetilde{C}^{(\boldsymbol{c})}[\boldsymbol{k}] = \sum_{t\in\mathcal{T}} \sum_{\boldsymbol{u}\in\mathcal{U}_t} \mathcal{A}^{t}_{\boldsymbol{k},\boldsymbol{u}}(Q_{t,\boldsymbol{u}}) \qquad \text{for every } \boldsymbol{k} \in \prod_{s=1}^{\ell}\{0,\ldots,h_s\}.$$

The sum is literal. If two windows contain the same ordered pair $(\boldsymbol{\eta},\boldsymbol{\zeta})$, their independent Gram blocks both contribute to the same target coefficient. For a symmetric scalar Gram matrix, an off-diagonal entry represents both ordered pairs. For a polynomial matrix, the two ordered block terms $Q[\boldsymbol{\eta},\boldsymbol{\zeta}]$ and $Q[\boldsymbol{\zeta},\boldsymbol{\eta}]$ provide the corresponding symmetry accounting. The implementation assembles every local PSD block first and then creates exactly $\prod_{s=1}^{\ell}(h_s+1)$ coefficient identities.

Worked example. Consider one scalar parameter, target degree $\boldsymbol{M} = (4)$, absolute order $\boldsymbol{r} = (2)$, and clique size $b = 2$. The unweighted Putinar term has component Gram degree $d_1 = 2$, window size two, and the two windows $\{0,1\}$ and $\{1,2\}$. The weighted term has component Gram degree $d_1 = 1$ and one window $\{0,1\}$. SparsePutinar therefore has three PSD blocks of dimension two and five exact coefficient identities. The label pair $(1,1)$ belongs to both unweighted windows, so both independent blocks contribute to target label two.



Overlapping windows add independent Gram contributions before exact coefficient matching.

SparsePutinar and dense Putinar use the same active terms. Dense Putinar uses one full Gram block per term, while SparsePutinar trades each full block for several smaller window blocks. SparseFullBox uses the same sliding-window rule but starts from the FullBox parity or all-subset generator terms. The two sparse parameters therefore control window side length over distinct certificate families and require family-specific interpretation.

Smaller maximum PSD blocks can accompany a larger optimization problem. Decreasing b can increase the number of cones and repeated Gram contributions, while the target coefficient-equality count remains unchanged. The resulting memory use and solve time depend on the complete block pattern, equality map, matrix size, number of cells, number of rate rows, and solver. Clique size alone provides zero universal computational ordering.

The word *clique* refers only to a tensor window of Bernstein Gram-basis labels. Sparsity-graph inference, chordal completion, and structural decomposition of a polynomial matrix are separate constructions. This implementation is therefore distinct from the structural-matrix chordal SOS decomposition studied by Zheng and Fantuzzi [17].

B.10 The Implemented Multivariate FullBox State

Basic idea. The mathematical construction is a full-box preordering: for several parameters, use every square-free product of the box generators. Each subset selects one weighted Gram block, and exact coefficient identities make the sum equal the sign-normalized target $S^{(\boldsymbol{c})}$ on one physical cell and stored rate row. The software state is `FullBox`.

Formula. For the hypercube, use

$$g_s(\boldsymbol{\alpha}) = \alpha_s(1-\alpha_s), \qquad s = 1, \dots, \ell.$$

Worked example. For two parameters, the four subset products are $1, g_1, g_2, g_1 g_2$.

Visual conclusion.

$J = \varnothing$ 1	$J = \{1\}$ $\alpha_1(1-\alpha_1)$
$J = \{2\}$ $\alpha_2(1-\alpha_2)$	$J = \{1,2\}$ $g_1 g_2$

For $\ell \geq 2$ and direction-wise absolute order $\boldsymbol{r}$, use the already defined tensor-product Bernstein lift $Z_{\boldsymbol{a}} = \boldsymbol{b}_{\boldsymbol{a}} \otimes I_n$. The implemented form is

$$S^{(\boldsymbol{c})} = \sum_{J \subseteq \{1,\dots,\ell\}} \left(\prod_{s \in J} g_s \right) Z_{\boldsymbol{r}-\chi_J}^{\top} Q_J Z_{\boldsymbol{r}-\chi_J}, \qquad Q_J \succeq 0.$$

Terms with negative tensor degree are omitted. A present block has dimension

$$n \prod_{s=1}^{\ell} \left(r_s + 1 - \mathbf{1}_{s \in J} \right).$$

Every physical cell and stored rate row gets independent dense Gram blocks and exact coefficient identities. The minimum order is componentwise $\lceil \boldsymbol{M}/2 \rceil$ for $\ell \geq 2$. This is a specific dense Bernstein realization of the full-box preordering. Full-space SOS, Putinar-only, sparse, and general-domain parsers are distinct constructions. A fixed multivariate order supplies a sufficient certificate and may remain inconclusive.

Finite SDP example: $\ell = 2$, $\boldsymbol{M} = (3,3)$. Let $(\alpha_1, \alpha_2) \in [0,1]^2$ be the local coordinates of one cell, and suppose the sign-normalized target is

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{i_1=0}^{3} \sum_{i_2=0}^{3} B_{i_1}^3(\alpha_1) B_{i_2}^3(\alpha_2) C^{(\boldsymbol{c})}[i_1, i_2] \succeq 0.$$

This is the uniform specialization $\boldsymbol{r} = (2,2) = \lceil \boldsymbol{M}/2 \rceil$. After exact elevation to degree $(4,4)$, the implementation introduces four independent PSD Gram matrices and enforces the identity

$$\begin{aligned} S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = {} & Z_{(2,2)}^{\top} Q_{\varnothing} Z_{(2,2)} + \alpha_1(1-\alpha_1) Z_{(1,2)}^{\top} Q_1 Z_{(1,2)} \\ & + \alpha_2(1-\alpha_2) Z_{(2,1)}^{\top} Q_2 Z_{(2,1)} \\ & + \alpha_1(1-\alpha_1)\alpha_2(1-\alpha_2) Z_{(1,1)}^{\top} Q_{12} Z_{(1,1)}, \\ Q_{\varnothing} \succeq 0, \qquad & Q_1 \succeq 0, \qquad Q_2 \succeq 0, \qquad Q_{12} \succeq 0. \end{aligned}$$

For an n-by-n residual, these blocks have dimensions $9n$, $6n$, $6n$, and $4n$. If

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{i_1=0}^{4}\sum_{i_2=0}^{4} B_{i_1}^4(\alpha_1)B_{i_2}^4(\alpha_2)\widetilde{C}^{(\boldsymbol{c})}[i_1, i_2],$$

then $\widetilde{C}^{(\boldsymbol{c})}[0,0] = C^{(\boldsymbol{c})}[0,0]$ and $\widetilde{C}^{(\boldsymbol{c})}[1,0] = C^{(\boldsymbol{c})}[0,0]/4 + 3C^{(\boldsymbol{c})}[1,0]/4$. The finite SDP has the four PSD block constraints above and one exact matrix equality for each of the $5 \times 5 = 25$ elevated Bernstein labels. With $[Q]_{\boldsymbol{i},\boldsymbol{j}}$ denoting the n-by-n Gram block indexed by labels $\boldsymbol{i}, \boldsymbol{j}$, two illustrative equalities are

$$\begin{aligned} C^{(\boldsymbol{c})}[0,0] &= [Q_\varnothing]_{(0,0),(0,0)}, \\ \frac{1}{4}C^{(\boldsymbol{c})}[0,0] + \frac{3}{4}C^{(\boldsymbol{c})}[1,0] &= \frac{1}{2}[Q_\varnothing]_{(0,0),(1,0)} + \frac{1}{2}[Q_\varnothing]_{(1,0),(0,0)} + \frac{1}{4}[Q_1]_{(0,0),(0,0)}. \end{aligned}$$

The remaining 23 are generated by the same weighted Bernstein coefficient map. Thus these 25 conditions are affine equalities and remain distinct from additional LMIs. A $\preceq 0$ comparison instead matches the negatives of the elevated target coefficients.

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
polya = direct.usePolya(1);
putinar = direct.usePutinar();
box = direct.useFullBox();
directConstraintCount = numel(direct.Constraints)
polyaState = [polya.PolyaDegree, numel(polya.Constraints)]
putinarState = [putinar.PutinarOrder, numel(putinar.Constraints)]
fullBoxState = [box.FullBoxOrder, numel(box.Constraints)]
```

```
directConstraintCount = 3
polyaState = 1×2 double
     1     4
putinarState = 1×2 double
     1     5
fullBoxState = 1×2 double
     1     5
```

Selections rebuild from the stored original `Residual` and `Relation`. Reapplying Pólya, Putinar, SparsePutinar, SparseFullBox, or FullBox replaces the previous selection, so the active state contains exactly one of the five mutually exclusive opt-in families.

B.11 The Implemented SparseFullBox Tensor-Window State

Basic idea. SparseFullBox retains every FullBox parity or generator-mask term and every exact target-coefficient identity, but it covers each term's tensor Bernstein Gram basis by overlapping axis-aligned windows. Every window owns an independent PSD Gram matrix, and all window contributions are added before matching the target coefficients.

Formula. Let $\mathcal{J}$ be the finite set of active FullBox terms. For $J \in \mathcal{J}$ with Gram degree $\boldsymbol{a}_J$, choose the scalar side length w, set $q_{J,s} = \min\{w, a_{J,s}+1\}$, and define the complete start set $\mathcal{U}_J = \prod_{s=1}^{\ell}\{0,\ldots,a_{J,s}-q_{J,s}+1\}$. Every start $\boldsymbol{u} \in \mathcal{U}_J$ defines the basis window

$$\mathcal{W}_{J,\boldsymbol{u}} = \boldsymbol{u} + \prod_{s=1}^{\ell}\{0,\ldots,q_{J,s}-1\}.$$

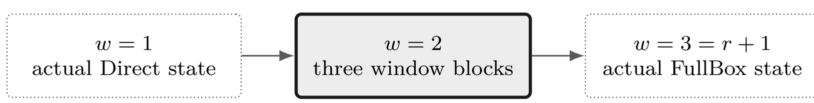
Let $\boldsymbol{z}_{J,\boldsymbol{u}}$ collect the Bernstein basis functions whose labels lie in $\mathcal{W}_{J,\boldsymbol{u}}$. The certificate is

$$S^{(\boldsymbol{c})}(\boldsymbol{\alpha}) = \sum_{J\in\mathcal{J}} g_J(\boldsymbol{\alpha}) \sum_{\boldsymbol{u}\in\mathcal{U}_J} \left(\boldsymbol{z}_{J,\boldsymbol{u}}(\boldsymbol{\alpha}) \otimes I_n\right)^{\top} Q_{J,\boldsymbol{u}} \left(\boldsymbol{z}_{J,\boldsymbol{u}}(\boldsymbol{\alpha}) \otimes I_n\right), \qquad Q_{J,\boldsymbol{u}} \succeq 0.$$

The one-parameter parity terms use the same window rule. A window for an n-by-n semidefinite target has PSD dimension $n\prod_{s=1}^{\ell} q_{J,s}$, and term J contains $\prod_{s=1}^{\ell}(a_{J,s}-q_{J,s}+2)$ independent windows. The reusable Bernstein–Gram map in section B.3 expands each window through its diagonal and paired off-diagonal label contributions, then all active terms and windows are summed into the common target coefficients.

Worked example. For one parameter, take the even assembled residual degree $\boldsymbol{M} = (4)$ and absolute order $\boldsymbol{r} = (2)$. The unweighted term has basis labels $\{0,1,2\}$ and the weighted term has labels $\{0,1\}$. At $w = 2$, the unweighted windows are $\{0,1\}$ and $\{1,2\}$, while the weighted term has one window $\{0,1\}$. The certificate therefore has three independent PSD blocks of dimension two and five exact coefficient identities. The pair $(1,1)$ appears in both unweighted windows, so both diagonal Gram entries contribute to target label two.

Visual conclusion.

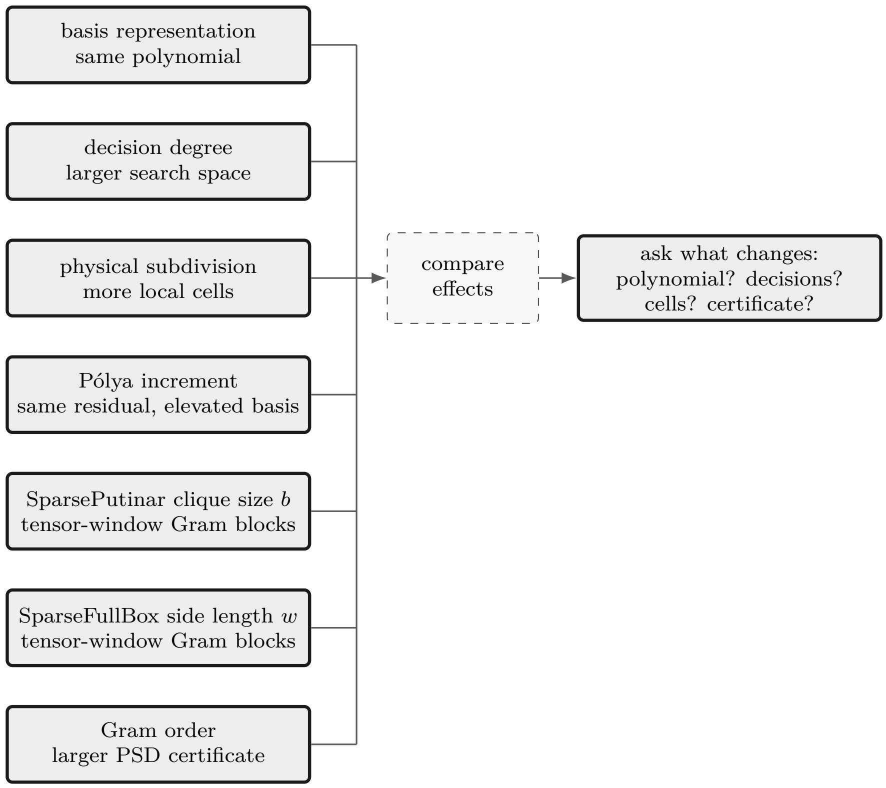


The side length changes the number and size of independent tensor-window blocks, while the target coefficient identities remain exact.

`useSpBox(w,r)` uses one scalar window side length w, default two, and the same dimension-dependent absolute-order minimum as FullBox. The minimum is $\boldsymbol{r} = (\lfloor M_1/2 \rfloor)$ when $\boldsymbol{M} = (M_1)$ and componentwise $\lceil \boldsymbol{M}/2 \rceil$ otherwise. The selected-state property remains `BandWidth`, but it stores the tensor-window side length. Width one canonicalizes to actual Direct state after order validation. A width that spans every order axis canonicalizes to actual FullBox state. Intermediate widths retain SparseFullBox state. Every physical cell, stored rate row, matrix entry in entry-wise mode, active term, and window owns fresh Gram variables. The method uses a zero implicit margin and leaves solver invocation to the caller.

B.12 Seven Different Modeling Choices

These choices act on different parts of the problem.



Choice	Changes	Preserves or leaves separate
basis conversion	coefficient coordinates	existing decision freedom
decision degree	independent decision coefficients	certificate order as a separate control
subdivision	physical cells and local hulls	represented polynomial and its degree
Pólya increment d	elevated residual coefficients	represented residual and decision freedom
Putinar/full-box order	Gram bases and PSD block sizes	decision degree
SparsePutinar clique size b	tensor-window basis size and number of independent PSD blocks	target degree and coefficient-equality count
SparseFullBox side length w	tensor-window basis size and number of independent PSD blocks	target degree and coefficient-equality count

Failure means the selected sufficient certificate is inconclusive about continuous infeasibility. Decide separately whether the model needs a richer direction-wise decision degree $\boldsymbol{m}$, a finer physical grid, a larger direction-wise Pólya increment $\boldsymbol{d}$, a different SparsePutinar clique size b, a different SparseFullBox window side length w, or a higher direction-wise absolute Gram order $\boldsymbol{r}$.

B.13 Implemented Boundary

Implemented public behavior	Background only
direct coefficient assembly	general full-space SOS parser
`usePolya([d])` with scalar or direction-wise increment	automatic Pólya search
`usePutinar([r])` with scalar or direction-wise absolute order	general-domain/general-generator Putinar interface
`useSpPut([b[,r]])` with scalar b and scalar or direction-wise r	sparsity-graph inference, chordal completion, or adaptive clique selection
`useSpBox([w[,r]])` with scalar w and scalar or direction-wise r	automatic or adaptive sparse-pattern selection
`useFullBox([r])` with scalar or direction-wise absolute order	general-domain Schmüdgen parser
exact one-dimensional parity forms	automatic or adaptive Gram-order search
all cells, active rate rows, and column-major entries	cross-cell/rate/entry Gram sharing
explicit user margins	package-owned strictness or solver policy

The consolidated assembly paths preserve the represented functions, existing YALMIP variables, certificate families, constraint order, PSD block dimensions, and cone partition, apart from ordinary floating-point summation differences. Numeric elevation, product, and Gram-map data may be reused because they are purely numeric. Each matrix entry, physical cell, rate row, active term, and tensor-window block owns its Gram decision variables independently.

B.14 Further Reading

The primary papers by Putinar [12] and Schmüdgen [13] remain the mathematical authority for their respective theorems. YALMIP maintains an SOS tutorial for implementation-oriented background, while the Positivstellensatz overview gives a compact map of the surrounding result family. Zheng and Fantuzzi [17] study structural-matrix chordal SOS decomposition, which is cited here to distinguish that graph-based construction from the package's tensor windows. None of these sources changes the fixed-order, box-specific package boundary described above.

Bibliography

[1] C. W. Scherer, "Mixed H_2/H_∞ control for time-varying and linear parametrically-varying systems," *International Journal of Robust and Nonlinear Control*, vol. 6, nos. 9–10, pp. 929–952, 1996. publisher DOI record.

[2] F. H. Clarke, *Optimization and Nonsmooth Analysis.* Philadelphia, PA, USA: SIAM, 1990. doi:10.1137/1.9781611971309.

[3] R. T. Farouki, "The Bernstein polynomial basis: A centennial retrospective," *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. doi:10.1016/j.cagd.2012.03.001.

[4] M. Zettler and J. Garloff, "Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion," *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. doi:10.1109/9.661615.

[5] I. Masubuchi, A. Kume, and E. Shimemura, "Spline-type solution to parameter-dependent LMIs," in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. doi:10.1109/CDC.1998.758549.

[6] Y. Xu and F. Jabbari, "Spline-based parameter varying output feedback synthesis with improved L_2 gain," in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. doi:10.1109/CDC56724.2024.10886461.

[7] G. Chesi, "LMI techniques for optimization over polynomials in control: a survey," *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. doi:10.1109/TAC.2010.2046926.

[8] F. Lukács, "Verschärfung des ersten Mittelwertsatzes der Integralrechnung für rationale Polynome," *Mathematische Zeitschrift*, vol. 2, pp. 295–305, 1918. EuDML record.

[9] E. de Klerk, R. Hess, and M. Laurent, Improved convergence rates for Lasserre-type hierarchies of upper bounds for box-constrained polynomial optimization, *SIAM Journal on Optimization*, vol. 27, no. 1, pp. 347–367, 2017. doi:10.1137/16M1065264.

[10] E. M. Aylward, S. M. Itani, and P. A. Parrilo, Explicit SOS decompositions of univariate polynomial matrices and the Kalman–Yakubovich–Popov lemma, in *Proceedings of the 46th IEEE Conference on Decision and Control*, pp. 5660–5665, 2007. author manuscript.

[11] C. W. Scherer and C. W. J. Hol, Matrix sum-of-squares relaxations for robust semi-definite programs, *Mathematical Programming*, vol. 107, pp. 189–211, 2006. doi:10.1007/s10107-005-0684-2.

[12] M. Putinar, "Positive polynomials on compact semi-algebraic sets," *Indiana University Mathematics Journal*, vol. 42, no. 3, pp. 969–984, 1993. JSTOR 24897130.

[13] K. Schmüdgen, "The K-moment problem for compact semi-algebraic sets," *Mathematische Annalen*, vol. 289, pp. 203–206, 1991. doi:10.1007/BF01446568.

[14] V. Powers and B. Reznick, "A new bound for Pólya's theorem with applications to polynomials positive on polyhedra," *Journal of Pure and Applied Algebra*, vol. 164, nos. 1–2, pp. 221–229, 2001. doi:10.1016/S0022-4049(00)00155-9.

[15] M. T. Harris and P. A. Parrilo, "Improved nonnegativity testing in the Bernstein basis via geometric means," arXiv:2309.10675, 2023. doi:10.48550/arXiv.2309.10675.

[16] J. Gouveia, A. Kovačec, and M. Saee, "On sums of squares of k-nomials," *Journal of Pure and Applied Algebra*, vol. 226, no. 1, art. 106820, 2022. doi:10.1016/j.jpaa.2021.106820.

[17] Y. Zheng and G. Fantuzzi, "Sum-of-squares chordal decomposition of polynomial matrix inequalities," *Mathematical Programming*, vol. 197, pp. 71–108, 2023. doi:10.1007/s10107-021-01728-w.

[18] Y. Xu and F. Jabbari, "GriD-LMIA: A Gridding-Based Assembler for Solving Differentiable Parameter-Dependent Linear Matrix Inequalities," arXiv:2608.03175, 2026. arXiv:2608.03175.

Index